

Tập luận văn đội tuyển quốc gia Trung Quốc năm 2018

Huấn luyện viên: Zhang Ruizhe

China Computer Federation, tháng 4 năm 2018

Nguồn gốc: Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2018

IOI China candidate team papers / National training team 2018 collection.pdf

Tóm tắt

PDF nguồn là tập hợp luận văn đội tuyển quốc gia Trung Quốc năm 2018. Khác với một số tập trước, lớp văn bản của tệp này trích xuất được khá tốt bằng pdftotext: phần bìa, mục lục và các trang mở đầu của từng bài đều đọc được. Bản dịch này chuyển ngữ phần đầu sách và mục lục, đồng thời ghi lại các chủ đề chính có thể xác nhận từ mục lục và phần mở đầu của các bài. Bản hiện tại đã bổ sung bản dịch đầy đủ bảy bài đầu tiên: bài về hàm sinh trong các bài toán gieo xúc xắc, báo cáo ra đề *Số nút của cây hậu tố*, bài về hồi quy bảo toàn thứ tự, báo cáo ra đề *Fim 4*, và bài về các kỹ thuật xử lý khối liên thông trên cây, báo cáo ra đề *Jellyfish* và khảo cứu mở rộng, cùng bài về *Leafy Tree* và cây cân bằng có trọng số.

1 Thông tin đầu sách

Tập sách có tiêu đề *Tập luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2018*. Huấn luyện viên ghi trên bìa là Zhang Ruizhe. Đơn vị xuất bản là China Computer Federation; thời điểm ghi trên bìa là tháng 4 năm 2018.

2 Mục lục đã dịch

Trang	Bài viết	Tác giả / trường
1	Bàn sơ lược về ứng dụng hàm sinh trong bài toán gieo xúc xắc	Yang Maolong, Trường Trung học Changjun, Trường Sa
11	Báo cáo ra đề <i>Số nút của cây hậu tố</i> và tối ưu hóa một lớp bài toán đoạn	Chen Jianglun, Trường Trung học Changjun, Trường Sa
23	Bàn sơ lược về bài toán hồi quy bảo toàn thứ tự	Gao Ruiquan, Trường Ngoại ngữ Nam Kinh
34	Báo cáo ra đề <i>Fim 4</i>	Wu Jinzhao, Trường Trung học số 2 Hàng Châu, Chiết Giang
45	Một số kỹ thuật và công cụ giải bài toán khối liên thông trên cây	Ren Xuandi, Trường Trung học số 1 Thiệu Hưng
58	Báo cáo ra đề <i>Jellyfish</i> và khảo cứu mở rộng	Liang Yancheng, Trường Trung học Trần Hải, Ninh Ba
75	<i>Leafy Tree</i> và cây cân bằng có trọng số được hiện thực bằng nó	Wang Siqi, Trường Trung học số 7 Thành Đô
87	Báo cáo ra đề <i>Tiểu H thích tô màu</i>	Chen Jiale, Trường Trung học số 2 Hàng Châu, Chiết Giang
92	Một số bài toán tổng hàm số học đặc biệt	Zhu Zhenting, THPT trực thuộc Đại học Sư phạm An Huy
113	Bàn sơ lược về ứng dụng DFT trong thi tin học	Liu Chengao, THPT trực thuộc Đại học Công nghiệp Tây Bắc
132	Báo cáo ra đề <i>Hàng đợi hoàn hảo</i>	Lin Xuheng, THPT trực thuộc Đại học Sư phạm Phúc Kiến
143	Bàn sơ lược về một số mở rộng của matroid và ứng dụng	Yang Qianlan, Trường Trung học Hoài Âm, Giang Tô
164	Bàn sơ lược về tính chất của Splay và Treap cùng ứng dụng	Dong Weijun, Trường Trung học số 6 Quảng Châu
180	Báo cáo ra đề <i>Cây khung phương sai nhỏ nhất</i>	He Zhongtian, Trường Trung học số 80 Bắc Kinh
192	Khảo cứu các bài toán sinh và đếm liên quan tới đồ thị Euler	Chen Tong, Trường Thực nghiệm trực thuộc Đại học Sư phạm Bắc Kinh

3 Bài 1: Bàn sơ lược về ứng dụng hàm sinh trong bài toán gieo xúc xắc

Yang Maolong, Trường Trung học Changjun, Trường Sa

Tóm tắt

Bài toán gieo xúc xắc là một lớp bài toán thường gặp trong thi thuật toán. Sau khi nghiên cứu kỹ lớp bài toán này, tác giả nhận thấy chúng đều có thể được giải bằng hàm sinh. Vì vậy, bài viết này xoay quanh chuỗi bài toán gieo xúc xắc, trình bày cách dùng hàm sinh để từng bước giải quyết vấn đề, cũng như ưu thế của cách giải bằng hàm sinh so với phương pháp truyền thống.

Từ khóa: hàm sinh, bài toán gieo xúc xắc, xác suất và kỳ vọng.

1 Dẫn nhập

Bài toán gieo xúc xắc là một lớp bài toán thường gặp trong thi thuật toán, và đã nhiều lần xuất hiện trong các kỳ thi những năm gần đây.

Mặt khác, hàm sinh là một công cụ hữu hiệu để giải lớp bài toán này. So với phương pháp truyền thống, nó có các ưu điểm như dễ tính toán và khả năng mở rộng mạnh. Tuy nhiên, hiện nay trong giới OI vẫn có rất ít nghiên cứu về phương pháp này.

Mục 2 quy ước một số ký hiệu. Mục 3 giới thiệu định nghĩa của hàm sinh xác suất và một số tính chất. Mục 4 giới thiệu ứng dụng cơ bản của thuật toán này. Mục 5 và 6 kết hợp với các đề bài để giới thiệu một số ứng dụng phức tạp hơn.

2 Quy ước ký hiệu

1. A_i biểu thị số thứ i của dãy A .
2. $A[l, r]$ biểu thị dãy gồm các số thứ l đến thứ r của dãy A .
3. $f^{(k)}(x)$ là đạo hàm cấp k của $f(x)$.
4. $[P]$ là ngoặc Iverson: nếu điều kiện trong ngoặc vuông đúng thì giá trị bằng 1, nếu không thì bằng 0.

3 Kiến thức chuẩn bị

3.1 Hàm sinh thông thường

Hàm sinh thông thường của dãy $a_0, a_1, a_2, \dots$ là

$$A(x) = \sum_{i=0}^{\infty} a_i x^i.$$

3.2 Hàm sinh xác suất

Nếu với dãy $a_0, a_1, a_2, \dots$ tồn tại một biến ngẫu nhiên rời rạc X nào đó thỏa $\Pr(X = i) = a_i$, thì hàm sinh thông thường của a_n ($n \in \mathbb{N}$) được gọi là hàm sinh xác suất của X .

Nói cách khác, nếu X là biến ngẫu nhiên rời rạc trên tập số nguyên không âm $\mathbb{N}$, thì hàm sinh xác suất của X là

$$F(z) = \mathbb{E}(z^X) = \sum_{i=0}^{\infty} \Pr(X = i) z^i.$$

3.2.1 Tính chất của hàm sinh xác suất

Vì X là biến ngẫu nhiên rời rạc trên $\mathbb{N}$, ta nhất định có

$$F(1) = \sum_{i=0}^{\infty} \Pr(X = i) = 1.$$

Lấy đạo hàm $F(z)$, ta được

$$F'(z) = \sum_{i=0}^{\infty} i \Pr(X = i) \cdot z^{i-1}.$$

Do đó kỳ vọng của X là

$$\mathbb{E}(X) = F'(1) = \sum_{i=0}^{\infty} i \Pr(X = i).$$

Suy luận thêm có thể nhận được

$$\mathbb{E}(X^k) = F^{(k)}(1), \quad k \neq 0.$$

Vì vậy phương sai của X là

$$\text{Var}(X) = F''(1) + F'(1) - (F'(1))^2.$$

3.3 Border

Với một dãy A có độ dài L , nếu

$$A[1, i] = A[L - i + 1, L],$$

thì gọi $A[1, i]$ là một *border* của A .

4 Dạng bài cơ bản

Ví dụ 1. Vương quốc ca hát Nguồn: CTSC2006.

Cho một dãy A có độ dài L . Sau đó, mỗi lần gieo một con xúc xắc công bằng ghi các số từ 1 đến m , rồi đưa số gieo được vào cuối dãy B ban đầu rỗng. Nếu trong dãy B đã xuất hiện dãy A cho trước, tức A là một dãy con liên tiếp của B , thì dừng lại. Hãy tính độ dài kỳ vọng của dãy B . $L \leq 10^5$.

4.1 Phân tích

Dạng cơ bản của bài toán gieo xúc xắc là: cho một dãy và một con xúc xắc, sau đó liên tục gieo xúc xắc để sinh ra một dãy ngẫu nhiên, cho đến khi dãy đã cho xuất hiện trong dãy ngẫu nhiên thì dừng, và cần tính số lần gieo kỳ vọng.

4.2 Một phương pháp không dùng hàm sinh

Đặt E_i là số lần gieo xúc xắc kỳ vọng từ trạng thái hiện tại thỏa $A[1, i]$ là hậu tố của B cho đến lần đầu tiên thỏa $A[1, i + 1]$ là hậu tố của B . Khi đó đại lượng cần tìm là

$$\sum_{i=0}^{n-1} E_i.$$

Định nghĩa $f_{i,j}$ là độ dài border dài nhất không phải chính nó của dãy mới thu được bằng cách thêm số j vào sau $A[1, i]$. Khi đó có phương trình

$$E_i = 1 + \frac{1}{m} \sum_{j=1}^m [j \neq A_{i+1}] \sum_{k=f_{i,j}}^i E_k.$$

Giải phương trình có thể nhận được

$$E_i = m + (m - 1) \sum_{k=1}^{i-1} E_k - \sum_{j=1}^m [j \neq A_{i+1}] \sum_{k=1}^{f_{i,j}-1} E_k.$$

Độ phức tạp thời gian như vậy là $O(nm)$. Vấn đề chính nằm ở việc tính

$$\sum_{j=1}^m [j \neq A_{i+1}] \sum_{k=1}^{f_{i,j}-1} E_k,$$

và phần này có thể tối ưu. Dựa trên công thức so sánh $A[1, i]$ với border dài nhất không phải chính nó, có thể thấy giữa hai công thức chỉ có $O(1)$ giá trị $f_{i,j}$ khác nhau, nên mỗi lần chỉ cần bảo trì lại $O(1)$ giá trị này. Khi đó độ phức tạp thời gian giảm xuống $O(n)$.

Có thể thấy phương pháp này rất phức tạp và khả năng mở rộng thấp. Phương pháp hàm sinh được giới thiệu dưới đây ưu việt hơn phương pháp này khá nhiều.

4.3 Phương pháp hàm sinh

Đặt a_i biểu thị $A[1, i]$ có phải là một border của A hay không: $a_i = 1$ nghĩa là có, còn $a_i = 0$ nghĩa là không.

Đặt f_i là xác suất để khi kết thúc, độ dài dãy ngẫu nhiên bằng i ; hàm sinh xác suất của nó là $F(x)$. Đặt dãy phụ trợ g_i là xác suất để độ dài dãy ngẫu nhiên đạt đến i mà vẫn chưa kết thúc; hàm sinh thông thường của nó là $G(x)$.

Đại lượng cần tìm là $F'(1)$. Qua phân tích, ta nhận được các đẳng thức sau:

$$F(x) + G(x) = 1 + G(x) \cdot x, \quad (1)$$

$$G(x) \cdot \left(\frac{1}{m}x\right)^L = \sum_{i=1}^L a_i \cdot F(x) \cdot \left(\frac{1}{m}x\right)^{L-i}. \quad (2)$$

Ý nghĩa của (1) là: sau khi thêm một số vào một dãy chưa kết thúc, dãy có thể kết thúc hoặc vẫn chưa kết thúc; 1 là trường hợp ban đầu khi dãy ngẫu nhiên rỗng.

Ý nghĩa của (2) là: sau khi thêm dãy đã cho vào sau một dãy chưa kết thúc, chắc chắn sẽ kết thúc, nhưng có thể đã kết thúc trước khi thêm đủ L số; khi đó dãy đã thêm nhất định phải là một border của dãy đã cho.

Lấy đạo hàm (1) rồi thay $x = 1$, ta được

$$F'(x) + G'(x) = G'(x) \cdot x + G(x), \quad (1)$$

$$F'(1) = G(1). \quad (3)$$

Tiếp tục thay $x = 1$ vào (2), ta được

$$G(1) \cdot \left(\frac{1}{m}\right)^L = \sum_{i=1}^L a_i \cdot F(1) \cdot \left(\frac{1}{m}\right)^{L-i}, \quad (2)$$

$$G(1) = \sum_{i=1}^L a_i \cdot m^i. \quad (4)$$

Từ (3)(4), suy ra

$$F'(1) = \sum_{i=1}^L a_i \cdot m^i.$$

Dùng thuật toán KMP hoặc hash là có thể tìm a_i và tính $F'(1)$ trong độ phức tạp thời gian $O(L)$.

4.4 Suy nghĩ thêm

Ta có thể tiến thêm một bước để nhận được phương pháp tính phương sai.

Theo

$$\text{Var}(X) = F''(1) + F'(1) - (F'(1))^2,$$

có thể thấy chỉ cần biết $F'(1)$ và $F''(1)$. $F'(1)$ đã biết, vì vậy cần tính $F''(1)$.

Lấy đạo hàm cấp hai của (1) rồi thay $x = 1$, ta được

$$F''(x) + G''(x) = G'(x) + G''(x) \cdot x + G'(x), \quad (3)$$

$$F''(1) = 2G'(1). \quad (4)$$

Tiếp tục lấy đạo hàm (2) rồi thay $x = 1$, ta được

$$G(x) = \sum_{i=1}^L a_i \cdot m^i \cdot F(x) \cdot x^{-i}, \quad (5)$$

$$G'(x) = \sum_{i=1}^L a_i \cdot m^i \cdot (F'(x) \cdot x^{-i} - i \cdot F(x) \cdot x^{-i-1}), \quad (6)$$

$$G'(1) = \sum_{i=1}^L a_i \cdot m^i \cdot (F'(1) - i). \quad (7)$$

Vì vậy

$$F''(1) = 2 \cdot \sum_{i=1}^L a_i \cdot m^i \cdot (F'(1) - i).$$

Nếu tiếp tục suy luận và quy nạp, có thể nhận được

$$F^{(k)}(1) = k \cdot \sum_{i=1}^L a_i \cdot m^i \sum_{j=0}^{k-1} (-1)^j \binom{k-1}{j} \frac{(i+j-1)!}{(i-1)!} F^{(k-1-j)}(1).$$

4.5 Tiểu kết

Phương pháp dùng hàm sinh để giải lớp bài toán này thường bắt đầu bằng cách định nghĩa một hàm sinh xác suất $F(x)$ và một hàm sinh phụ trợ $G(x)$. Sau đó xét việc thêm một số hoặc một dãy cho trước vào trạng thái chưa kết thúc, rồi dựa vào tình huống thực tế để lập phương trình. Cuối cùng, thông qua thay giá trị và lấy đạo hàm, ta giải ra $F'(1)$ cần tìm.

5 Dạng bài phức tạp

Ví dụ 2. Trò chơi gieo xúc xắc Cho n dãy A_i có độ dài lần lượt là L_i . Lại cho một con xúc xắc ghi các số từ 1 đến m , trong đó xác suất gieo ra i là P_i . Sau đó, mỗi lần gieo xúc xắc một lần và đưa số trên xúc xắc vào cuối dãy B ban đầu rỗng. Nếu cả n dãy đã cho đều là dãy con liên tiếp của B , thì dừng lại. Hãy tính độ dài kỳ vọng của dãy B . Bảo đảm $n \leq 15$, $L \leq 20000$, $m \leq 10^5$.

5.1 Phân tích

Về bản chất, bài toán này vẫn là bài toán gieo xúc xắc, nhưng đã biến thành trường hợp cần n dãy đều xuất hiện trong dãy ngẫu nhiên.

5.2 Lời giải

Đặt T_i là độ dài của dãy ngẫu nhiên khi A_i xuất hiện lần đầu. Khi đó đại lượng cần tìm là

$$\mathbb{E} \left(\max_{i \in \{1, 2, \dots, n\}} T_i \right).$$

Nếu một dãy A_i là dãy con liên tiếp của một dãy khác A_j , thì nhất định có $T_i \leq T_j$, nên có thể loại bỏ A_i .

Giá trị lớn nhất không dễ tính, nên dùng bao hàm–loại trừ Min–Max:

$$\max_{i=1}^n T_i = \sum_{S \subseteq \{1, 2, \dots, n\}} (-1)^{|S|+1} \min_{i \in S} T_i.$$

Theo tính tuyến tính của kỳ vọng, bài toán có thể chuyển thành tính $\mathbb{E}(\min_{i \in S} T_i)$, tức hể một dãy xuất hiện thì kết thúc. Không mất tính tổng quát, giả sử $S = \{1, 2, \dots, n\}$.

Định nghĩa

$$P(A) = \prod_{i \in A} P_i.$$

Đặt

$$a_{i,j,k} = [A_i[1, k] = A_j[L_j - k + 1, L_j]].$$

Đặt $f_{i,j}$ là xác suất để dãy xuất hiện đầu tiên là A_i và độ dài dãy ngẫu nhiên bằng j ; hàm sinh thông thường của nó là $F_i(x)$. Đặt dãy phụ trợ g_i là xác suất để độ dài dãy ngẫu nhiên đạt đến i mà vẫn chưa kết thúc; hàm sinh thông thường của nó là $G(x)$.

Ta có các đẳng thức sau:

$$\sum_{i=1}^n F_i(x) + G(x) = 1 + G(x) \cdot x, \quad (5)$$

$$G(x) \cdot P(A_i)x^{L_i} = \sum_{j=1}^n \sum_{k=1}^{L_i} a_{i,j,k} \cdot F_j(x) \cdot P(A_i[k+1, L_i]) \cdot x^{L_i-k}, \quad \forall i \in S. \quad (6)$$

Đại lượng cần tính là

$$\sum_{i=1}^n F'_i(1).$$

Lấy đạo hàm (5) rồi thay $x = 1$, ta được

$$\sum_{i=1}^n F'_i(1) = G(1).$$

Thay $x = 1$ vào (6), ta được

$$G(1) = \sum_{j=1}^n \left(\sum_{k=1}^{L_i} a_{i,j,k} \cdot \frac{1}{P(A_i[1, k])} \right) \cdot F_j(1), \quad \forall i \in S.$$

Hiện có $n+1$ ẩn, nhưng chỉ có n phương trình. Có thể thấy $\sum_{i=1}^n F_i(x)$ là hàm sinh xác suất của độ dài dãy ngẫu nhiên, nên

$$\sum_{i=1}^n F_i(1) = 1.$$

Như vậy có thể dùng khử Gauss để tìm $G(1)$. Đồng thời cũng có thể nhận được $F_i(1)$, tức xác suất để dãy xuất hiện đầu tiên là dãy thứ i ; chẳng hạn bài [SDOI2017] Trò chơi đồng xu yêu cầu tính $F_i(1)$.

Dùng hash, có thể tìm mảng a trong độ phức tạp thời gian $O(n^2L)$, và với mỗi cặp (i, j) tính

$$\sum_{k=1}^{L_i} a_{i,j,k} \cdot \frac{1}{P(A_i[1, k])}.$$

Với mỗi S , có thể dùng khử Gauss trong $O(n^3)$ để giải $G(1)$. Vì vậy tổng độ phức tạp thời gian là $O(n^2L + 2^n n^3)$.

6 Một số ứng dụng mới

6.1 Khớp mơ hồ

Ví dụ 3. Dice Nguồn: 2013 Multi-University Training Contest 5.

Với một con xúc xắc công bằng m mặt, hãy tính số lần gieo kỳ vọng để dừng khi n kết quả cuối cùng giống nhau, hoặc để dừng khi n kết quả cuối cùng đôi một khác nhau. Bảo đảm $n, m \leq 10^6$, và với câu hỏi thứ hai bảo đảm $n \leq m$.

6.1.1 Phương pháp sai phân truyền thống

Câu hỏi thứ nhất Đặt f_i là số lần gieo xúc xắc kỳ vọng từ trạng thái hiện tại đến khi kết thúc, trong đó trạng thái hiện tại thỏa i lần cuối cùng giống nhau. Khi đó đại lượng cần tìm là f_0 .

Có thể lập công thức chuyển:

$$f_i = \frac{1}{m}f_{i+1} + \frac{m-1}{m}f_1 + 1.$$

Trước hết sai phân, ta được

$$f_{i-1} - f_i = \frac{1}{m}(f_i - f_{i+1}).$$

Vì $f_0 - f_1 = 1$, nên $f_{i-1} - f_i = m^{i-1}$. Đồng thời $f_n = 0$, vì vậy

$$f_0 = \sum_{i=0}^{n-1} m^i = \frac{m^n - 1}{m - 1}.$$

Câu hỏi thứ hai Đặt g_i là số lần gieo xúc xắc kỳ vọng từ trạng thái hiện tại đến khi kết thúc, trong đó trạng thái hiện tại thỏa i lần cuối cùng đôi một khác nhau. Khi đó đại lượng cần tìm là g_0 .

Công thức chuyển là

$$g_i = \frac{m-i}{m}g_{i+1} + \frac{1}{m} \sum_{j=1}^i g_j + 1.$$

Trước hết sai phân:

$$g_{i-1} - g_i = \frac{m-i}{m}(g_i - g_{i+1}).$$

Tương tự có $g_0 - g_1 = 1$, $g_n = 0$, nên cũng có thể tìm g_0 trong thời gian $O(n)$.

6.1.2 Cách làm bằng hàm sinh

Phân tích Tuy có thể xem đơn giản bài này là một bài toán gieo xúc xắc nhiều dãy, số lượng dãy hợp lệ lại quá lớn. Nhưng tất cả chúng đều thỏa cùng một điều kiện, vì vậy có thể gom chúng lại với nhau.

Câu hỏi thứ nhất Đặt f_i là xác suất để kết thúc ở lần thứ i ; hàm sinh xác suất của nó là $F(x)$. Đặt g_i là xác suất để đã gieo i lần mà vẫn chưa kết thúc; hàm sinh thông thường của nó là $G(x)$.

Khi đó nhận được

$$F(x) + G(x) = G(x) \cdot x + 1, \tag{7}$$

$$G(x) \cdot \left(\frac{1}{m}x\right)^{n-1} = \sum_{i=1}^n F(x) \cdot \left(\frac{1}{m}x\right)^{n-i}. \tag{8}$$

Áp dụng cách giải tương tự phần trước, ta có thể giải ra

$$F'(1) = \frac{m^n - 1}{m - 1}.$$

Câu hỏi thứ hai Đặt f_i là xác suất để kết thúc ở lần thứ i ; hàm sinh xác suất của nó là $F(x)$. Đặt g_i là xác suất để đã gieo i lần mà vẫn chưa kết thúc; hàm sinh thông thường của nó là $G(x)$.

Khi đó nhận được

$$F(x) + G(x) = G(x) \cdot x + 1, \quad (9)$$

$$G(x) \cdot \left(\frac{1}{m}x\right)^n \cdot \frac{m!}{(m-n)!} = \sum_{i=1}^n F(x) \cdot \left(\frac{1}{m}x\right)^{n-i} \cdot \frac{(m-i)!}{(m-n)!}. \quad (10)$$

Có thể giải ra

$$F'(1) = \sum_{i=1}^n m^i \frac{(m-i)!}{m!}.$$

6.1.3 So sánh

Tuy hai phương pháp đều có thể cho cùng kết quả, cách làm bằng hàm sinh rõ ràng đơn giản hơn về mặt tính toán so với phương pháp truyền thống, và phương pháp cũng dễ hiểu hơn. Đồng thời, với các đề bài khác nhau, phần cần sửa đổi là rất ít.

6.2 Trường hợp phức tạp hơn

Ví dụ 4. Một bài xác suất nhỏ thú vị Nguồn: <http://roosephu.github.io/2017/12/31/condexp/>.

Một con xúc xắc n mặt, mỗi mặt được đánh số từ 1 đến n . Có một bộ đếm ban đầu bằng 0. Mỗi lần tung xúc xắc, nếu kết quả là số lẻ thì bộ đếm bị xóa về 0; nếu không thì bộ đếm tăng thêm 1, và nếu kết quả là n thì kết thúc. Hỏi kỳ vọng của bộ đếm khi kết thúc. Bảo đảm n là số chẵn.

6.2.1 Lời giải

Nhìn qua thì đáp án là $\frac{n}{2}$, nhưng thật ra không phải.

Tương tự, đặt f_i là xác suất để kết thúc khi bộ đếm bằng i ; hàm sinh xác suất của nó là $F(x)$. Nhưng định nghĩa của mảng phụ trợ cần đổi một chút: đặt g_i là số lần kỳ vọng để bộ đếm bằng i ; hàm sinh thông thường của nó là $G(x)$.

Ta có thể nhận được các phương trình

$$F(x) + G(x) = G(x) \cdot \frac{x}{2} + \frac{G(1)}{2} + 1, \quad (11)$$

$$F(x) = G(x) \cdot \frac{x}{n}. \quad (12)$$

Ý nghĩa của phương trình thứ nhất là: với mỗi trạng thái của G , có xác suất $\frac{1}{2}$ để bộ đếm tăng thêm 1, và có xác suất $\frac{1}{2}$ để bộ đếm bị xóa về 0.

Vì $F(1) = 1$, nên $G(1) = n$. Phương trình (11) có thể biến thành

$$F(x) + G(x) = G(x) \cdot \frac{x}{2} + \frac{n}{2} + 1. \quad (13)$$

Thay (12) vào (13), ta được

$$G(x) \cdot \left(1 + \frac{x}{n} - \frac{x}{2}\right) = \frac{n}{2} + 1, \quad (8)$$

$$G(x) = \frac{\frac{n}{2} + 1}{1 + \frac{x}{n} - \frac{x}{2}}, \quad (9)$$

$$G(x) = \frac{n^2 + 2n}{2n - (n-2)x}. \quad (14)$$

Lại thay (14) ngược vào (12), ta biết

$$F(x) = \frac{(n+2)x}{2n - (n-2)x}, \quad (10)$$

$$F'(x) = \frac{2n \cdot (n+2)}{[2n - (n-2)x]^2}, \quad (11)$$

$$F'(1) = \frac{2n}{n+2}. \quad (12)$$

7 Tổng kết

Với lớp bài toán gieo xúc xắc, ta có thể xây dựng hàm sinh và sử dụng thông tin trong đề bài để lập quan hệ giữa các hàm sinh, rồi giải ra giá trị mong muốn. Dùng phương pháp này có thể tránh phương pháp khử Gauss truyền thống.

Lớp bài toán được nhắc đến trong bài viết này vẫn đáng được nghiên cứu sâu hơn. Vì vậy tác giả hy vọng bài viết có thể đóng vai trò gợi mở, và hy vọng các độc giả quan tâm có thể nghiên cứu tiếp, mở rộng cách làm của tác giả, từ đó nhận được nhiều cách làm và ứng dụng thú vị hơn.

Lời cảm ơn

- Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu.
- Cảm ơn sự hướng dẫn của huấn luyện viên đội tuyển quốc gia Zhang Ruizhe.
- Cảm ơn thầy Xie Qiufeng đã quan tâm và hướng dẫn.
- Cảm ơn bạn Chen Jianglun và đàn anh Guo Xiaoxu đã cung cấp ý tưởng cho tôi.
- Cảm ơn bạn Chen Jianglun, học trưởng Luo Yuping, đàn anh Guo Xiaoxu và bạn Wang Xiuhua đã đọc duyệt bài viết này.
- Cảm ơn gia đình đã quan tâm và ủng hộ tôi.

Tài liệu tham khảo

1. Wikipedia, *Probability-generating function*.
2. Ronald L. Graham, Donald E. Knuth, Oren Patashnik. *Concrete Mathematics*.
3. Jin Ce, *Phép toán trên hàm sinh và bài toán đếm tổ hợp*, luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2015.
4. Hu Yuanming, *Phân tích cơ sở và ứng dụng của xác suất trong thi tin học*, luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2013.
5. Wei Taiyun, *Các cách giải khác nhau và so sánh cho một bài toán tung đồng xu*.

4 Bài 2: Báo cáo ra đề Số nút của cây hậu tố và tối ưu hóa một lớp bài toán đoạn

Chen Jianglun, Trường Trung học Changjun, Trường Sa

Tóm tắt

Cấu trúc dữ liệu hậu tố là một hệ thống hiện đang được ứng dụng rộng rãi trong thi thuật toán. Nó bao gồm các nội dung quen thuộc như mảng hậu tố, automaton hậu tố, cây hậu tố, v.v. Những cấu trúc dữ liệu hậu tố này có thể xử lý hiệu quả rất nhiều bài toán.

Tác giả đã nghiên cứu các thuật toán liên quan tới cấu trúc dữ liệu hậu tố xuất hiện trong những năm gần đây, rồi ra đề *Số nút của cây hậu tố*. Dựa vào tính chất của cây hậu tố và automaton hậu tố, tác giả nhận được một cách làm sơ bộ. Trong quá trình trao đổi ở bài tập đội tuyển và trại mùa đông, tác giả đã thảo luận cách làm này với các bạn khác, rồi cùng họ nghiên cứu ra các thuật toán hiệu quả và ngắn gọn hơn.

Khi nghiên cứu thuật toán cho bài này, tác giả nhận thấy rằng với lớp bài toán chuỗi dùng cây động để bảo trì cây hậu tố, có thể dùng hợp nhất theo heuristic để thay cây động, từ đó làm cho mô hình này được cài đặt đơn giản hơn. Đồng thời, phương pháp này còn có thể mở rộng sang nhiều bài toán đoạn hơn.

Bài viết này trình bày chi tiết nguồn gốc ý tưởng ra đề và quá trình tối ưu từng bước, đồng thời đưa ra một phương án tối ưu cho một lớp bài toán đoạn. Tác giả hy vọng bài viết có thể giúp nhiều người hiểu sâu hơn các tính chất liên quan của cấu trúc dữ liệu hậu tố, cũng như học được phương pháp tối ưu cho một lớp bài toán đoạn.

1 Dẫn nhập

Trong một kỳ ACM-ICPC khu vực năm 2016, đã xuất hiện một mô hình mới mẽ dùng cây động để bảo trì cây hậu tố. Mô hình này có thể giải hiệu quả một lớp bài toán như nhiều lần hỏi số xâu con phân biệt về bản chất của một xâu con trong một xâu, hoặc độ dài xâu con dài nhất xuất hiện ít nhất hai lần trong một xâu con. Điều đó khiến mọi người chú ý tới hướng này.

Trong kỳ tự kiểm tra đội tuyển quốc gia năm 2017, phiên bản tăng cường của bài toán này xuất hiện: đếm số xâu con phân biệt về bản chất trong một xâu con, với điều kiện bắt đầu bằng một ký tự cố định và kết thúc bằng một ký tự cố định.

Đồng thời, những năm gần đây cũng thường xuất hiện một dạng bài: đã biết một thuật toán giải một bài toán nào đó, hãy tính một số thông tin trong quá trình chạy thuật toán ấy. Chẳng hạn trong ZJOI2017 Day1 T2, đã biết một thuật toán hỗ trợ cộng điểm đơn và hỏi tổng đoạn; yêu cầu của đề không phải là đáp án mà thuật toán đó trả về, mà là xác suất thuật toán tính đúng kết quả theo đúng quá trình của nó.

Cây hậu tố là một công cụ mạnh để giải bài toán chuỗi. Từ đó tác giả nghĩ rằng: nếu không yêu cầu dùng cây hậu tố để giải bài toán chuỗi nói chung, mà hỏi số nút nhận được sau khi dựng cây hậu tố cho một xâu thì sao?

Từ đó tác giả bắt đầu nghiên cứu vấn đề này và ra đề bài này.

Cấu trúc bài viết như sau:

- Mục 2 đến 4 là một số định nghĩa cơ bản, mô tả thông tin đề bài và tình hình điểm số.
- Mục 5 đến 10 giới thiệu 3 thuật toán ăn điểm từng phần và 3 thuật toán đạt điểm tối đa.
- Mục 11 đề xuất phương án dùng hợp nhất theo heuristic để thay cây động, qua đó tối ưu độ phức tạp mã nguồn của lớp bài toán đoạn như dùng cây động để bảo trì cây hậu tố, đồng thời trình bày một số ứng dụng của phương án này.

2 Mô tả đề bài

Cho một xâu P độ dài n . Có m truy vấn; mỗi truy vấn cho hai tham số l, r , hỏi số nút của cây hậu tố tạo bởi xâu con $P[l, r]$.

2.1 Phạm vi dữ liệu

Trong bài này dùng các số để biểu thị xâu. Các số khác nhau đại diện cho các ký tự khác nhau, số giống nhau đại diện cho cùng một ký tự; giá trị các số nằm trong đoạn $[0, n]$.

- Với 20% dữ liệu: $n \leq 500$, $m \leq 500$.
- Với 40% dữ liệu: $n \leq 3000$, $m \leq 3000$.
- Với 50% dữ liệu: $n \times m \leq 3 \times 10^8$.
- Với 10% dữ liệu khác: xâu toàn là 0.
- Với 20% dữ liệu khác: mỗi ký tự là một số nguyên không âm ngẫu nhiên trong $[0, n]$, và l, r của mỗi truy vấn đều được sinh ngẫu nhiên.
- Với 100% dữ liệu: $n \leq 10^5$, $m \leq 3 \times 10^5$, mỗi số trong xâu đều không vượt quá n .

2.2 Giới hạn tài nguyên

Giới hạn thời gian 3 giây, giới hạn bộ nhớ 1 GB.

2.3 Định dạng nhập

Dòng đầu gồm hai số nguyên dương n, m , biểu thị độ dài xâu và số truy vấn.

Dòng thứ hai gồm n số nguyên không âm cách nhau bởi dấu cách, biểu thị xâu đó.

Tiếp theo là m dòng, mỗi dòng gồm hai số nguyên dương l, r ($l \leq r$), biểu thị truy vấn xâu con $[l, r]$.

2.4 Định dạng xuất

In m dòng, mỗi dòng một số nguyên dương biểu thị đáp án.

2.5 Ví dụ nhập

```
7 1
2 1 3 1 3 1 4
1 7
```

2.6 Ví dụ xuất

```
10
```

Khi tính bài này, bỏ qua nút gốc.

2.7 Giải thích ví dụ

Cho số 1 biểu thị ký tự a , số 2 biểu thị ký tự b , số 3 biểu thị ký tự n , số 4 biểu thị ký tự s . Khi đó cây hậu tố của xâu trong ví dụ chính là cây hậu tố của bananas. Hình trong bản gốc vẽ cây này với các cạnh lần lượt gắn các nhãn như bananas, a, na, s, na, s, nas, v.v., và tính đáp án sau khi bỏ qua gốc.

3 Phân bố điểm

Có 59 người tham gia lần huấn luyện này. Trong đội tuyển và nhóm bồi dưỡng tinh anh, có 4 người đạt điểm tối đa, 1 người đạt 60 điểm, 1 người đạt 50 điểm, 2 người đạt 40 điểm, 2 người đạt 30 điểm, và 1 người đạt 20 điểm.

4 Các khái niệm liên quan trong bài viết

Với một xâu S , dùng $S[i]$ hoặc S_i biểu thị ký tự thứ i của S , dùng $S[l, r]$ biểu thị xâu con của S nằm trong đoạn $[l, r]$, và dùng $|S|$ biểu thị độ dài xâu S .

Định nghĩa ký tự kế tiếp của một xâu con $S[a, b]$ của S là $S[b + 1]$. Đặc biệt, nếu $S[b]$ là cuối xâu thì ký tự kế tiếp của $S[a, b]$ không được định nghĩa.

4.1 Hash

Hash là một công cụ thường dùng để xử lý bài toán chuỗi; nó thực hiện một ánh xạ nén. Trong bài viết này sẽ dùng hash để ánh xạ mỗi xâu vào một số nguyên, biến bài toán xét hai xâu có bằng nhau hay không thành bài toán xét hai số nguyên có bằng nhau hay không.

4.2 Cây trie hậu tố

Cây trie hậu tố của một xâu S có $|S|$ ký tự là một cây có hướng chứa nút gốc. Mỗi nút đại diện cho một xâu; các nút khác nhau đại diện cho các xâu khác nhau; mỗi nút không phải gốc đại diện cho một xâu con của S ; nút gốc biểu thị xâu rỗng. Trên cây, xâu mà một tổ tiên đại diện là một tiền tố của xâu mà nút đang xét đại diện.

4.3 Cây hậu tố

Cây hậu tố là một dạng nén của cây trie hậu tố.

Trong cây trie hậu tố của S , nếu xâu mà một nút đại diện là một hậu tố của S , thì gọi nút đó là nút hậu tố.

Trên cơ sở cây trie hậu tố, sau khi gộp mỗi nút không phải nút hậu tố và có đúng một con với nút con của nó, ta nhận được cây hậu tố.

Trong cây hậu tố, dùng fa_k biểu thị cha của nút k trên cây. Dùng $last_k$ biểu thị vị trí xuất hiện cuối cùng của xâu mà nút k đại diện.

4.4 Automaton hậu tố

Automaton hậu tố của một xâu S là một automaton hữu hạn có thể chấp nhận mọi hậu tố của S .

Định nghĩa và phương pháp xây dựng automaton hậu tố đã rất phổ biến trong thi thuật toán, nên bài viết này không trình bày lại. Độc giả chưa quen có thể xem tài liệu tham khảo [1].

Cấu trúc cây tạo bởi tất cả các nút của automaton hậu tố được gọi là cây *parent*.

4.5 Cây động

Cây động là một cấu trúc dữ liệu thường dùng trong thi thuật toán hiện nay, dùng để bảo trì động phân rã đường của một cây.

Nội dung cơ bản của cây động đã rất phổ biến; độc giả chưa quen có thể xem tài liệu tham khảo [6].

5 Thuật toán một

Dựa vào cây trie hậu tố để xây dựng cây hậu tố: trước hết dùng mọi hậu tố của một xâu để xây dựng một cây trie, sau đó gộp mọi nút không phải nút hậu tố và có đúng một con với nút con của nó.

Độ phức tạp thời gian của thuật toán này là $O(n^2m)$, độ phức tạp không gian là $O(n^3)$. Lý do là kích thước bảng chữ cái bằng n , còn số nút trong cây trie của mọi hậu tố là $O(n^2)$.

Điểm kỳ vọng: 20 điểm.

6 Thuật toán hai

Với dữ liệu $n, m \leq 3000$, có thể độc lập tính đáp án của từng xâu trong mỗi truy vấn với độ phức tạp tương đối tốt.

Bổ đề 6.1. Cây *parent* của automaton hậu tố là cây hậu tố của xâu đảo.

Tính chất này thường được dùng trong bài toán chuỗi và cũng được nhắc tới trong tài liệu tham khảo [2].

Dựa vào tính chất này, có thể đảo xâu rồi xây dựng automaton hậu tố. Việc xây dựng automaton hậu tố có thể dùng phương pháp tăng dần tuyến tính.

Thuật toán này có độ phức tạp thời gian $O(nm \log n)$, độ phức tạp không gian $O(n)$. Vì kích thước bảng chữ cái của bài này rất lớn, cần dùng cấu trúc dữ liệu, chẳng hạn map trong C++, để bảo trì các cạnh đi ra của mỗi điểm; do đó độ phức tạp thời gian có thêm một nhân tử log.

Điểm kỳ vọng: 40 điểm.

7 Thuật toán ba

Thực ra việc lưu các cạnh chuyển của automaton hậu tố có thể dùng hash để tối ưu cấu trúc dữ liệu của thuật toán hai. Khi đó đạt được độ phức tạp thời gian $O(nm)$, còn độ phức tạp không gian vẫn là $O(n)$.

Điểm kỳ vọng: 50 điểm.

8 Thuật toán bốn

8.1 Phân tích sơ bộ

Ký hiệu số nút cây hậu tố của xâu cần xét S là $A(S)$.

Các nút của cây hậu tố có thể chia làm hai loại: nút hậu tố và nút không phải hậu tố. Số nút hậu tố đúng bằng $|S|$; phần này chỉ liên quan tới độ dài xâu.

Vì nút lá của cây trie hậu tố nhất định là nút hậu tố, nên một nút không phải hậu tố nhất định có ít nhất một con. Nếu một nút không phải hậu tố có đúng một con thì trong cây hậu tố nút này sẽ bị gộp với con của nó. Do đó, những nút cần được tính vào đáp án phải thỏa có ít nhất hai con.

Một nút trên cây trie hậu tố đại diện cho một xâu con của S . Nếu nút k có ít nhất hai con, điều đó có nghĩa là tồn tại ít nhất hai ký tự khác nhau c_1, c_2 , sao cho xâu con t mà nút k đại diện thỏa: tc_1 và tc_2 cũng đều là xâu con của S .

8.2 Cách làm phức tạp

Sau khi phát hiện tính chất trên, tác giả nhận được một cách làm có độ phức tạp thời gian $O(n \log^3 n + m \log^2 n)$. Tuy nhiên, cách làm này quá phức tạp, hiệu suất cũng thấp, rất khó hiểu. Đồng thời phương pháp này không ảnh hưởng tới việc hiểu các cách làm phía sau, nên bài viết này không trình bày lại. Độ giả quan tâm có thể tìm phần giới thiệu chi tiết về cách làm này trong tài liệu tham khảo [5].

Điểm kỳ vọng: 100 điểm.

9 Thuật toán năm

9.1 Ứng dụng mô hình cây động bảo trì cây hậu tố

Điểm khó của bài này nằm ở nhiều nhóm truy vấn, mỗi nhóm hỏi một xâu con $P[l, r]$ của xâu nhập P . Xét việc liệt kê đầu trái l của truy vấn từ phải sang trái; với mỗi r , ta bảo trì động số nút cây hậu tố của xâu $P[l, r]$.

Khi l dịch sang trái một vị trí, với mỗi xâu $P[l, r]$, xét các thay đổi giữa $P[l, r]$ và $P[l + 1, r]$.

Dựa vào tính chất vừa phát hiện, xét một tiền tố $P[l, v]$ của $P[l, r]$, gọi xâu này là t . Nếu trong $[l, r]$ tồn tại hai ký tự c_1, c_2 sao cho tc_1 và tc_2 đều là xâu con của $P[l, r]$, còn xâu $P[l + 1, r]$ không thỏa điều kiện này, thì xâu t sẽ tạo thêm đóng góp 1 cho đáp án của $P[l, r]$. Sau khi đầu trái chuyển từ $l + 1$ sang l , cần lập tức tìm các v thỏa yêu cầu này.

Vì tc_1 và tc_2 đều là xâu con của $P[l, r]$, nên xâu t chắc chắn đã xuất hiện trong $P[l + 1, r]$. Do trong $P[l + 1, r]$, t chưa tạo đóng góp cho đáp án, nên với mọi vị trí xuất hiện $P[a, b]$ của t trong $P[l + 1, r]$ ($b - a + 1 = |t|$, $P[a, b] = t$), hoặc $b = r$, hoặc các ký tự kế tiếp của những xâu này, tức $P[b + 1]$, đều giống nhau.

Hình trong bản gốc minh họa trường hợp trên bằng các lần xuất hiện của t trong đoạn $[l, r]$, kèm các ký tự kế tiếp c_1, c_2, c_2 .

Mọi tiền tố $P[l, v]$ ($l \leq v \leq r$) của xâu $P[l, r]$ tương ứng trên cây trie hậu tố với một đường đi từ gốc tới một nút nào đó. Giả sử các nút trên đường này lần lượt là $x_1, x_2, \dots, x_{r-l+1}$. Cần thống kê có bao nhiêu nút x_i thỏa x_i tồn tại một con khác x_{i+1} . Nếu tồn tại, vì x_{i+1} là con của x_i , nên x_i đã có ít nhất hai con.

Mỗi lần thêm mọi tiền tố bắt đầu ở l hiện tại, ta đánh dấu đã thăm cho tất cả $x_1, x_2, \dots, x_{r-l+1}$. Định nghĩa *con thực* của một nút là nút con được thăm cuối cùng trong tất cả các con của nó. Nếu sau một thao tác, con thực của một nút k không thay đổi, điều đó cho thấy ký tự kế tiếp của xâu t mà nút k đại diện giống với ký tự kế tiếp trong lần xuất hiện trước của xâu này. Trong hình của bản gốc, ký tự kế tiếp của xâu hiện tại $t = P[l, l + |t| - 1]$ và ký tự kế tiếp của lần xuất hiện trước của t đều là c_1 .

Chú ý rằng điều kiện để xét một xâu có tạo đóng góp mới cho đáp án hay không là: hiện tại đã xuất hiện ít nhất hai loại ký tự kế tiếp, còn trước đó chỉ xuất hiện một loại ký tự kế tiếp. Vì vậy, nếu ký tự kế tiếp của một tiền tố của $P[l, r]$ bằng ký tự kế tiếp của lần xuất hiện cuối cùng, thì nó không thể tạo đóng góp mới cho đáp án.

Nói cách khác, chỉ những nút có con thực thay đổi mới có khả năng tạo đóng góp mới cho đáp án. Chú ý rằng định nghĩa con thực ở đây giống con thực trong cây động, và mỗi lần đánh dấu tương đương với thao tác access của cây động. Theo phân tích độ phức tạp của cây động, số lần chuyển con thực là $O(\log n)$ khấu hao, vì vậy điều này liên hệ với mô hình cổ điển dùng cây động để bảo trì cây hậu tố.

Cụ thể, dùng $last_t$ biểu thị vị trí xuất hiện trước của xâu t , và dùng $diff_t$ biểu thị vị trí xuất hiện trước của xâu t với ký tự kế tiếp khác. Hình trong bản gốc minh họa $last[t]$ và $diff[t]$ bằng bốn lần xuất hiện của t có các ký tự kế tiếp c_1, c_2, c_2, c_3 .

Với xâu con $P[l + 1, r]$, khi $r \geq diff_t + |t|$, t tạo đóng góp cho đáp án của $P[l + 1, r]$; khi $r < diff_t + |t|$, t chưa tạo đóng góp cho đáp án.

Với xâu con $P[l, r]$, khi $r \geq last_t + |t|$, t tạo đóng góp cho đáp án của $P[l, r]$; khi $r < last_t + |t|$, t chưa tạo đóng góp cho đáp án.

Do đó, khi chuyển từ $P[l + 1, r]$ sang $P[l, r]$, nếu r nằm trong đoạn $[last_t + |t|, diff_t + |t|)$, đáp án sẽ mới tăng thêm 1. Đây là một thao tác cộng đoạn.

Như vậy, có thể dùng cây động và cây đoạn để hỗ trợ access, tìm các nút có con thực thay đổi, và thực hiện thao tác cộng đoạn.

9.2 Tiểu kết

Nội dung trên chính là ứng dụng của một mô hình cổ điển trong bài này.

Nhưng đó chỉ là một phần của bài. Tiếp theo còn phải đối mặt với một vấn đề khác.

9.3 Xử lý đáp án bị tính lặp

Khi tính số nút cây hậu tố, ta chia toàn bộ nút thành hai loại: nút hậu tố và nút không phải hậu tố. Phần trước đã thống kê “nút hậu tố + nút trên cây trie hậu tố có ít nhất hai con”. Trong đó, các nút vừa là nút hậu tố vừa có hai con đã bị tính hai lần. Theo nguyên lý bù trừ, chỉ cần trừ số nút đồng thời thỏa hai điều kiện này là nhận được đáp án cuối cùng.

Khi hỏi xâu $P[l, r]$, nếu tồn tại một v sao cho xâu $P[v, r]$ xuất hiện trong $P[l, r]$ với ít nhất hai ký tự kế tiếp khác nhau, thì xâu này sẽ bị tính lặp.

9.4 Tính chất then chốt

Sau khi phân tích, có thể phát hiện một tính chất then chốt: nếu một xâu t có hai ký tự kế tiếp khác nhau trong $P[l, r]$, thì mọi hậu tố của t đều có ít nhất hai ký tự kế tiếp khác nhau.

Dựa vào tính chất này, độ dài các xâu bị tính lặp thỏa tính đơn điệu, nên có thể dùng chặt nhị phân để tính độ dài lớn nhất của xâu bị tính lặp.

Sau khi chặt nhị phân, bài toán chuyển thành: cho một xâu, hãy xét xâu này có hai ký tự kế tiếp khác nhau trong một đoạn hay không.

9.5 Hợp nhất cây đoạn

Có hai ký tự kế tiếp khác nhau nghĩa là trên cây trie hậu tố có hai con khác nhau. Vì vậy cần hỏi một nút có tồn tại hai cây con của hai con chứa nút hậu tố nằm trong $[l, r]$ hay không.

Vì con thực là nút con được thăm cuối cùng trong các con của một nút, nếu xâu mà con thực đại diện không xuất hiện trong đoạn $[l, r]$, điều đó cho thấy xâu mà nút hiện tại đại diện không có bất kỳ ký tự kế tiếp nào trong $[l, r]$, chắc chắn không thỏa điều kiện. Nếu xâu mà con thực đại diện xuất hiện trong đoạn $[l, r]$, tiếp theo cần xét trong cây con của các con khác có tồn tại một nút hậu tố nằm trong $[l, r]$ hay không; vấn đề này có thể giải bằng hợp nhất cây đoạn.

Dùng hợp nhất cây đoạn để bảo trì tập vị trí bắt đầu của các xâu do nút hậu tố trong mỗi cây con đại diện, ta có thể truy vấn trong $O(\log n)$ số nút hậu tố của một cây con nằm trong đoạn $[l, r]$. Xét ngoài con thực ra còn có cây con của con nào khác tồn tại nút hậu tố nằm trong $[l, r]$ hay không, tương đương với truy vấn xem hiệu giữa số nút hậu tố trong cây con của nút hiện tại nằm trong $[l, r]$ và số nút hậu tố trong cây con của con thực nằm trong $[l, r]$ có bằng 0 hay không.

Trên đây là toàn bộ nội dung của thuật toán năm. Thuật toán dùng cây động + cây đoạn và chặt nhị phân + hợp nhất cây đoạn; tổng độ phức tạp là $O(n \log^2 n + m \log^2 n)$.

Điểm kỳ vọng: 100 điểm.

10 Thuật toán sáu

Bài này thực ra còn có thể tối ưu thêm. Chú ý rằng quá trình xử lý là liệt kê đầu trái l của truy vấn, rồi tính đáp án cho mọi truy vấn có đầu trái hiện tại. Khi l cố định, với một xâu t , chỉ cần tìm một r nhỏ nhất sao cho t xuất hiện trong $[l, r]$ với ký tự kế tiếp khác nhau. Giá trị r này chính là $\text{diff}_{\text{node}(t)} + |t|$ đã được bảo trì trước đó, trong đó $\text{node}(t)$ là nút tương ứng với xâu t trên cây hậu tố.

Sau khi chèn nhị phân một xâu, có thể dùng bảng hash để truy vấn trong $O(1)$ vị trí của một xâu trên cây hậu tố. Như vậy có thể phán đoán trong $O(1)$ xem một xâu có xuất hiện trong một đoạn với ký tự kế tiếp khác nhau hay không.

Nhờ đó độ phức tạp thời gian được tối ưu thành $O(n \log^2 n + m \log n)$.

Điểm kỳ vọng: 100 điểm.

11 Tối ưu hóa một lớp bài toán đoạn

Khi nghiên cứu mô hình cây động bảo trì cây hậu tố, tác giả phát hiện bản chất của việc chuyển con thực là: với một xâu t , tìm mọi vị trí xuất hiện của nó và sắp xếp thành một dãy $\text{pos}_1, \text{pos}_2, \dots, \text{pos}_v$ theo thứ tự tăng dần của đầu phải vị trí xuất hiện. Với hai phần tử kế nhau bất kỳ pos_i và pos_{i+1} của dãy này, nếu ký tự kế tiếp của hai vị trí này khác nhau, điều đó cho thấy các nút lá hậu tố tương ứng với hai xâu này nằm trong cây con của hai con khác nhau của nút tương ứng với t trên cây hậu tố. Khi đầu trái truy vấn được liệt kê từ pos_{i+1} tới pos_i , con thực trên cây hậu tố sẽ chuyển một lần.

Một khi đã bảo trì dãy như vậy, ta cũng có thể tính được last và diff mà không cần cây động.

Dùng hợp nhất theo heuristic để tính dãy pos của mỗi cây con. Đồng thời, hợp nhất theo heuristic cũng có thể chứng minh độ phức tạp số lần chuyển con thực trong bài này. Độ phức tạp chính là số lần xuất hiện tình huống pos_i và pos_{i+1} có ký tự kế tiếp khác nhau.

Định nghĩa con nặng là con có số nút hậu tố trong cây con lớn nhất trong tất cả các con, các con khác là con nhẹ. Nếu trước hết chỉ xét con nặng, ký tự kế tiếp của pos_1 tới pos_v đều giống nhau. Sau đó thêm từng nút hậu tố của con nhẹ vào; mỗi lần thêm một nút vào dãy pos , nhiều nhất tạo ra 2 điểm đứt mới với hai phần tử kế trước và sau. Vì mỗi số nhiều nhất được chen $O(\log n)$ lần với tư cách thuộc con nhẹ, nên tổng độ phức tạp là $O(n \log n)$ nhân với độ phức tạp của phép cộng đoạn.

Do ngay từ bước đầu tác giả đã nghĩ tới việc dùng hợp nhất theo heuristic để thay cây động, nên đã không nhận ra rằng phần sau có thể trực tiếp dùng phương pháp trong thuật toán sáu để phán đoán. Dĩ nhiên, vì hợp nhất theo heuristic cũng có thể tính được diff và last tương ứng, thuật toán sáu vẫn có thể được cài đặt bằng hợp nhất theo heuristic.

Phương pháp như vậy không chỉ dùng được cho bài này, mà còn có thể dùng trong nhiều bài toán khác.

11.1 Ví dụ một

11.1.1 Mô tả đề bài và phạm vi dữ liệu Cho một xâu $P[1, n]$, có m truy vấn; mỗi truy vấn hỏi số xâu con phân biệt về bản chất trong một xâu con $[l, r]$.

$$n, m \leq 10^5.$$

11.1.2 Phân tích Dựa vào ý tưởng được cung cấp trong bài viết này, liệt kê đầu trái l của truy vấn từ phải sang trái, rồi bảo trì đáp án cho mỗi đầu phải.

Vấn xét lượng thay đổi của đáp án khi đầu trái truy vấn chuyển từ $l + 1$ thành l . Với một xâu $P[l, v]$, nếu đầu phải của vị trí xuất hiện trước của xâu này lớn hơn r , còn đầu phải v của vị trí xuất hiện hiện tại không vượt quá r , thì nó tạo đóng góp mới 1 cho đáp án. Tương tự, dùng $last_t$ biểu thị vị trí xuất hiện cuối cùng của xâu t . Khi đó với mọi $v \in [l, n]$, đáp án của các truy vấn có đầu phải nằm trong $[v, last_{P[l, v]} - 1]$ cần tăng thêm 1.

Tiếp theo, nếu xâu $P[l, v]$ và xâu $P[l, v + 1]$ có $last$ giống nhau, thì có thể xử lý chung, tương đương với cộng một cấp số cộng có công sai 1 lên một đoạn.

Theo tính chất của cây động, mọi điểm nằm trên cùng một đường thực có thời điểm được thăm cuối cùng giống nhau; tức là $last$ của các xâu mà những nút này đại diện giống nhau. Do đó, các điểm trên cùng một đường thực có thể được xử lý cùng lúc. Mỗi lần access, số đường thực được thăm là $O(\log n)$ khẩu hao, nên tổng độ phức tạp thời gian là $O(n \log^2 n)$, độ phức tạp không gian $O(n)$.

Khi access, ta sẽ tìm được mọi đường thực từ một nút tới gốc. Điểm đổi giữa hai đường thực chính là nơi xảy ra một lần chuyển con thực. Phần trước đã nói rằng hợp nhất theo heuristic có thể truy vấn mọi sự kiện chuyển con thực, nên có thể dùng hợp nhất theo heuristic để thay cây động.

11.2 Tiểu kết

Các bài toán chuỗi dùng được tối ưu này thường có các đặc điểm sau:

1. Có một xâu mẫu lớn được nhập vào, mỗi truy vấn hỏi thông tin của một xâu con của xâu này.
2. Có thể biết toàn bộ xâu mẫu lớn trước khi trả lời truy vấn, không phải bắt buộc trực tuyến thêm ký tự vào cuối xâu mẫu.
3. Đáp án cần tìm liên quan tới mỗi xâu con phân biệt về bản chất và tiền nhiệm, hậu nhiệm của nó trong xâu mẫu.

11.3 Mở rộng

Khi nghiên cứu sâu hơn quá trình này, tác giả phát hiện phương pháp này có thể mở rộng sang nhiều bài toán đoạn hơn.

11.4 Ví dụ hai

11.4.1 Mô tả đề bài và phạm vi dữ liệu Cho một cây n nút, có m nhóm truy vấn. Mỗi lần hỏi số nút sau khi xây dựng cây ảo trên tất cả các nút có số hiệu nằm trong $[l, r]$.

$$n, m \leq 10^5.$$

11.4.2 Phân tích Với truy vấn $[l, r]$, xét xem mỗi điểm k có tạo đóng góp cho đáp án hay không. k nằm trong cây ảo nếu $l \leq k \leq r$, hoặc nếu trong cây con của ít nhất hai con của k đều tồn tại một nút nằm trong đoạn $[l, r]$.

Vấn dùng ý tưởng tương tự: liệt kê đầu trái l từ phải sang trái, rồi bảo trì động đáp án cho mỗi đầu phải r . Mỗi lần thực hiện một thao tác access với đầu trái hiện tại l ; việc chuyển cạnh ảo-thực tương ứng với việc đáp án của các truy vấn có đầu phải r trong một đoạn sẽ mới tăng thêm 1.

Cũng có thể dùng hợp nhất theo heuristic: sắp xếp mọi nút trong cây con của k theo số hiệu. Khi $[l, r]$ chứa hai nút đến từ cây con của hai con khác nhau, k sẽ nằm trong cây ảo. Ở đầu mục này đã chứng minh tổng số đoạn là $O(n \log n)$. Vì vậy bài này có thể được giải trong thời gian $O(n \log^2 n) + O(m \log n)$.

12 Tổng kết

Bài này là một bài toán lấy số nút cây hậu tố làm mô hình, khảo sát năng lực phân tích tính chất cây hậu tố của thí sinh và năng lực vận dụng mô hình cổ điển. Các cấu trúc liên quan gồm automaton hậu tố, cây hậu tố, cây đoạn, cây động và hợp nhất theo heuristic, đều thuộc phạm vi kiến thức của thí sinh NOI.

Tuy nhiên, bài này chú trọng kiểm tra khả năng phân tích tính chất của cấu trúc dữ liệu hậu tố. Chẳng hạn so với cây hậu tố, mỗi nút không bị nén trên cây trie hậu tố tương ứng với một xâu con trong xâu gốc; xâu con đó hoặc là một hậu tố, hoặc có hai ký tự kế tiếp khác nhau.

Bài này gồm hai phần, mỗi phần đều cần suy nghĩ sâu mới nhận được phương án giải hiệu quả; đối với thí sinh, độ khó là rất lớn.

Ban đầu bài này được tạo ra khi nghiên cứu tính chất của automaton hậu tố. Lúc đầu tác giả đặt trực tiếp vấn đề lên automaton hậu tố và nhận được thuật toán bốn rất phức tạp. Đưa bài này vào bài tập đội tuyển là để thử tìm cách làm đơn giản hơn.

Sau đó, bạn Zhu Zhenting từ THPT trực thuộc Đại học Sư phạm An Huy phát hiện một tính chất quan trọng, tức tính chất được nhắc tới trong bài viết này: độ dài các xâu bị tính lặp thỏa tính đơn điệu. Qua chắt nhai phân, cách làm được đơn giản hóa rất nhiều. Trong buổi trao đổi của trại mùa đông năm nay, bạn ấy cùng bạn Wang Xiuhuan từ Trường Trung học số 7 Thành Đô đã chia sẻ ý tưởng này. Đó chính là thuật toán nằm trong bài viết này.

Sau buổi trao đổi, tác giả và bạn Zhang Yubo từ Trường Trung học số 80 Bắc Kinh đã thảo luận sâu hơn, phát hiện chỉ cần dùng *last*, *diff* và bảng hash là có thể phán đoán trong $O(1)$ xem một xâu có từng xuất hiện trong một đoạn với ký tự kế tiếp khác nhau hay không. Đó chính là thuật toán sáu trong bài viết này.

Trong quá trình nghiên cứu các thuật toán này, tác giả phát hiện bản chất của cây động bảo trì cây hậu tố là tìm mọi vị trí xuất hiện của một xâu, rồi chia chúng theo ký tự kế tiếp: các xâu kế nhau và có ký tự kế tiếp giống nhau được chia vào cùng một đoạn; giữa các đoạn khác nhau sẽ phát hiện thay đổi của *last* và *diff*. Vì vậy có thể dùng hợp nhất theo heuristic để thay cây động, làm đơn giản độ phức tạp mã nguồn. Đồng thời tác giả phát hiện phương pháp này có thể mở rộng sang một số bài toán đoạn khác.

Hy vọng thông qua bài này, mọi người có thể hiểu sâu hơn về tính chất của automaton hậu tố và cây hậu tố, đồng thời thành thạo cách giải tinh tế cho một lớp bài toán đoạn.

13 Lời cảm ơn

- Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu.
- Cảm ơn sự hướng dẫn của huấn luyện viên đội tuyển quốc gia Zhang Ruizhe.
- Cảm ơn thầy Xie Qiufeng của Trường Trung học Changjun đã quan tâm và hướng dẫn.
- Cảm ơn Wang Xiuhuan, Zhu Zhenting và Zhang Yubo đã giúp đỡ về ý tưởng tối ưu của bài này.
- Cảm ơn Luo Yuping, Guo Xiaoxu, Yang Maolong, Liu Chengao, Zhang Qingchuan và Wu Hang đã đọc duyệt bài viết này.
- Cảm ơn gia đình đã quan tâm và ủng hộ tôi.

Tài liệu tham khảo

1. Chen Lijie, *Automaton hậu tố*, trao đổi trại mùa đông năm 2012.
2. Zhang Tianyang, *Automaton hậu tố và ứng dụng*, tập luận văn ứng viên đội tuyển quốc gia Trung Quốc năm 2015.
3. Hong Huadun, *Báo cáo ra đề Tái cấu trúc hệ gene*, tập luận văn ứng viên đội tuyển quốc gia Trung Quốc năm 2017.
4. *String*, ACM/ICPC Asia Regional Qingdao Online 2016.
5. Chen Jianglun, *Báo cáo lời giải đề tự chọn trong bài tập đội tuyển quốc gia IOI2018*.
6. Huang Zhi'ao, *Bàn sơ lược về các vấn đề liên quan tới cây động và một số mở rộng đơn giản*, tập luận văn ứng viên đội tuyển quốc gia Trung Quốc năm 2014.

5 Bài 3: Bàn sơ lược về bài toán hồi quy bảo toàn thứ tự

Gao Ruiquan, Trường Ngoại ngữ Nam Kinh

Tóm tắt

Bài viết này là luận văn đội tuyển quốc gia Tin học Trung Quốc năm 2018 của tác giả. Bài viết chủ yếu giới thiệu bài toán hồi quy bảo toàn thứ tự. Trước hết, thông qua các trường hợp đặc biệt, bài viết giới thiệu những cách làm thường gặp trong thi tin học cho bài toán L_p ($1 \leq p < \infty$); tiếp đó giới thiệu một ý tưởng mới dùng chia đôi toàn cục để giải bài toán tổng quát và các tối ưu của nó; cuối cùng giới thiệu sơ lược cách giải và mở rộng của bài toán L_∞ .

1 Dẫn nhập

Bài toán hồi quy bảo toàn thứ tự là một lớp bài toán ít được thảo luận trong thi tin học, có độ khó cao và có thể xây dựng lời giải bằng nhiều hướng tư duy khác nhau. Trong đời sống thực tế, bài toán hồi quy bảo toàn thứ tự thường xuất hiện trong các bài toán thống kê như dự đoán ngộ độc thuốc, thiết lập cỡ quần áo, v.v.

Bài toán hồi quy bảo toàn thứ tự được thảo luận trong bài này phụ thuộc vào những kết luận toán học khá mạnh. Một loại kết luận được suy ra bằng cách kết hợp hình học và đại số; một loại khác được giải quyết bằng cách xây dựng bài toán mới rồi dùng chia đôi toàn cục. Hướng tư duy đó vượt khỏi cách nghĩ thông thường cho các bài toán giá trị tối ưu trong thi lập trình, và có thể đem lại nhiều gợi mở cho thí sinh.

Mục 2 giới thiệu các khái niệm cơ bản của bài toán hồi quy bảo toàn thứ tự (L_p). Các mục 3 đến 5 trình bày chi tiết cách làm cho L_p ($1 \leq p < \infty$): mục 3 giới thiệu một số lời giải trong các tình huống đặc biệt từ góc nhìn tham lam và bảo trì đường gấp khúc; mục 4 trình bày chi tiết cách dùng chia đôi toàn cục để giải bài toán hồi quy bảo toàn thứ tự tổng quát; mục 5 giới thiệu các tối ưu của cách làm trên những cấu trúc thứ tự bộ phận đặc biệt. Mục 6 giới thiệu sơ lược cách giải và mở rộng của L_∞ .

2 Bài toán hồi quy bảo toàn thứ tự

2.1 Quan hệ thứ tự bộ phận

Định nghĩa 2.1. Giả sử R là một quan hệ hai ngôi trên tập S . Nếu R thỏa:

- tính phản xạ: $\forall x \in S$, có xRx ;
- tính phản đối xứng: $\forall x, y \in S$, nếu xRy và yRx thì $x = y$;
- tính bắc cầu: $\forall x, y, z \in S$, nếu xRy và yRz thì xRz ,

thì gọi R là một quan hệ thứ tự bộ phận không nghiêm ngặt trên S , ký hiệu là $\leq$.

2.2 Mô tả bài toán

Cho số nguyên dương p , một đồ thị có hướng không chu trình G với tập đỉnh $V = \{v_1, v_2, \dots, v_n\}$, tập cạnh E ($|E| = m$), và hàm chi phí (y, w) ($\forall i, w_i > 0$). Nếu trong G có một đường đi có hướng từ v_i đến v_j , thì ta có quan hệ thứ tự bộ phận $v_i \leq v_j$.

Cần tìm dãy giá trị đỉnh f , thỏa với mọi $v_i \leq v_j$ ($i, j \in [1, n]$) đều có $f_i \leq f_j$, sao cho chi phí hồi quy là nhỏ nhất:

$$\sum_{i=1}^n w_i |f_i - y_i|^p, \quad 1 \leq p < \infty,$$

$$\max_{i=1}^n w_i |f_i - y_i|, \quad p = \infty.$$

Với cùng một p , lớp bài toán này được gọi chung là bài toán L_p .

2.3 Một số quy ước

Định nghĩa 2.2. Với dãy z , thao tác biến các phần tử không vượt quá a thành a , và biến các phần tử không nhỏ hơn b thành b , được gọi là làm tròn dãy z về tập $S = \{a, b\}$.

Định nghĩa 2.3. Giá trị trung bình L_p của tập đỉnh U là giá trị k làm nhỏ nhất

$$\sum_{v_i \in U} w_i |y_i - k|^p \quad (1 \leq p < \infty)$$

hoặc

$$\max_{v_i \in U} w_i |y_i - k| \quad (p = \infty).$$

3 Thuật toán trong các trường hợp đặc biệt

3.1 Một thuật toán tham lam

Ví dụ 1. Approximation Nguồn đề: phỏng theo ASC38 Problem A; trong đề gốc mọi w_i đều bằng 1.

Cho các dãy số nguyên dương y, w . Hãy tìm dãy số thực đơn điệu không giảm f , sao cho

$$\sum_{i=1}^n w_i (f_i - y_i)^2$$

nhỏ nhất. $n \leq 200000$.

Bổ đề 3.1. Giá trị trung bình L_2 của tập đỉnh U là trung bình có trọng số

$$\frac{\sum_{v_i \in U} w_i y_i}{\sum_{v_i \in U} w_i}.$$

Chứng minh. Có thể chứng minh đơn giản bằng đạo hàm. ■

Bổ đề 3.2. Với mọi $1 \leq i < n$, nếu $y_i > y_{i+1}$, thì trong nghiệm tối ưu nhất định có $f_i = f_{i+1}$.

Chứng minh. Phản chứng rằng trong nghiệm tối ưu $f_i < f_{i+1}$. Chọn một $\varepsilon > 0$ đủ nhỏ, khi đó đặt

$$f'_i = f_i + \varepsilon w_{i+1}, \quad f'_{i+1} = f_{i+1} - \varepsilon w_i$$

sẽ không ảnh hưởng đến quan hệ thứ tự bộ phận. Khi đó

$$\begin{aligned} & w_i(f'_i - y_i)^2 + w_{i+1}(f'_{i+1} - y_{i+1})^2 - w_i(f_i - y_i)^2 - w_{i+1}(f_{i+1} - y_{i+1})^2 \\ &= -2\varepsilon w_i w_{i+1}(f_{i+1} + y_i - f_i - y_{i+1}) + \varepsilon^2 w_i w_{i+1}(w_i + w_{i+1}) < 0. \end{aligned}$$

Điều này mâu thuẫn với tính tối ưu. ■

Theo Bổ đề 3.1 và Bổ đề 3.2, ta có thể dùng quy trình sau để giải:

1. Duy trì một ngăn xếp đơn điệu. Mỗi phần tử trong ngăn xếp là (S_i, y'_i, w'_i) , trong đó S_i là một tập, còn y'_i, w'_i là các số thực. Giả sử tại mỗi thời điểm kích thước ngăn xếp là m . Ban đầu ngăn xếp rỗng.
2. Xét các i từ trái sang phải, với mỗi i thêm phần tử $(\{i\}, y_i, w_i)$. Nếu tại một thời điểm nào đó $m > 1$ và $y'_{m-1} > y'_m$, thì lấy (S_m, y'_m, w'_m) và $(S_{m-1}, y'_{m-1}, w'_{m-1})$ khỏi ngăn xếp, rồi thêm phần tử

$$\left(S_{m-1} \cup S_m, \frac{y'_{m-1}w'_{m-1} + y'_m w'_m}{w'_{m-1} + w'_m}, w'_{m-1} + w'_m \right).$$

3. Đáp án f_i là y'_j ứng với tập S_j chứa i .

Độ phức tạp thời gian của thuật toán trên là $O(n)$. Thuật toán này có thể mở rộng sang bài toán $1 \leq p < \infty$: chỉ cần thay trung bình có trọng số sau khi gộp bằng giá trị trung bình L_p của tập mới.

Cách tham lam như vậy có thể mở rộng sang cấu trúc thứ tự bộ phận tổng quát hơn không? Câu trả lời là không. Lý do là kết luận “không ảnh hưởng đến quan hệ thứ tự bộ phận” trong Bổ đề 3.2 không còn đúng nữa, nên không thể dùng tiếp thuật toán phía sau.

3.2 Thuật toán bảo trì đường gấp khúc

Thuật toán bảo trì đường gấp khúc là một phương pháp quan trọng để tối ưu quy hoạch động, và có thể mở rộng sang bài toán hồi quy bảo toàn thứ tự khi G tạo thành một cây.

Ví dụ 2. Đồ thị có hướng Bài tập đội tuyển năm 2018, đề liên khảo day 2 T3, phần điểm 65.

Cho đồ thị có hướng liên thông yếu $G = (V, E)$ gồm n đỉnh và $n - 1$ cạnh. Mỗi đỉnh có trọng số đỉnh d_i và thời gian sửa đổi w_i . Với mỗi $i (1 \leq i \leq n)$, mỗi lần sửa đổi có thể tốn w_i thời gian để tăng hoặc giảm d_i đi 1. Hãy tìm tổng thời gian ít nhất để sau khi sửa đổi, với mọi $(u, v) \in E$ đều có $d_u \leq d_v$.

$$n \leq 3 \times 10^5, \quad 1 \leq d_i \leq 10^9, \quad 1 \leq w_i \leq 10^4.$$

Ta dễ nghĩ đến việc dùng quy hoạch động trên cây.

Dễ thấy mọi giá trị cuối cùng nhất định nằm trong tập $D = \{d_1, d_2, \dots, d_n\}$. Gọi D_j là phần tử nhỏ thứ j trong D , và $g_{i,j}$ là chi phí sửa đổi nhỏ nhất trong cây con của đỉnh i khi đỉnh i được chỉnh về D_j . Chuyển trạng thái quy hoạch động có thể được tối ưu bằng giá trị nhỏ nhất tiền tố $L_{i,j}$ và giá trị nhỏ nhất hậu tố $R_{i,j}$.

Với đỉnh lá i , hiển nhiên g_i, L_i, R_i đều là các đường gấp khúc có hệ số góc đơn điệu không giảm.

Với đỉnh bất kỳ i , g_i có thể nhận được bằng cách cộng các L hoặc R của mọi con của nó với đường gấp khúc $y = w_i|x - d_i|$. Còn L_i là đường nhận được bằng cách biến phần có hệ số góc dương của g_i thành $y = \min\{g_{i,j}\}$; R_i là đường nhận được bằng cách biến phần có hệ số góc âm của g_i thành $y = \min\{g_{i,j}\}$. Vì vậy, bằng quy nạp có thể chứng minh rằng mọi g_i, L_i, R_i đều là các đường gấp khúc có hệ số góc đơn điệu không giảm.

Do đó có thể dùng cây đoạn để duy trì biểu thức hàm của từng đoạn trong các đường gấp khúc tạo bởi g, L, R (hiển nhiên $x = D_j \sim D_{j+1}$ là một đoạn): trong tính toán g_i , việc cộng các L và R của con có thể thực hiện bằng gộp cây đoạn¹; việc cộng đường gấp khúc $y = w_i|x - d_i|$ là một thao tác cộng đoạn.

Vì L_i và R_i tương ứng với g_i sẽ không đồng thời ảnh hưởng đến g của tổ tiên, nên có thể tính L_i, R_i bằng cách chặt nhị phân và sửa trực tiếp trên cây đoạn của g_i .

Độ phức tạp thời gian của thuật toán trên là $O(n \log n)$.

Có thể thấy phương pháp bảo trì đường gấp khúc vẫn không thể mở rộng sang cấu trúc thứ tự bộ phận tổng quát. Nút thắt nằm ở chỗ nó cần cấu trúc thứ tự bộ phận của toàn bộ đồ thị hỗ trợ quá trình quy hoạch động từ dưới lên.

4 Thuật toán cho bài toán tổng quát

4.1 Một hướng tư duy khác: bắt đầu từ chia đôi toàn cục

Trong thi tin học, ta thường gặp những bài cần trả lời nhiều truy vấn mà mỗi truy vấn đều có thể trả lời bằng chặt nhị phân. Nếu chặt nhị phân riêng cho từng truy vấn, thường sẽ xuất hiện rất nhiều tính toán lặp. Vì vậy lúc này có thể dùng tư tưởng toàn cục: nhờ cùng một phép tiền xử lý, nhanh chóng truy vấn mọi câu hỏi có đáp án rơi vào một khoảng, rồi chia mỗi truy vấn sang một khoảng đáp án mới. Tư tưởng này cũng áp dụng được cho các bài tương tác mà ta chỉ biết số lượng đáp án trong khoảng.

Vì bài viết này không liên quan đến các cách làm chia đôi toàn cục phức tạp hơn, các mở rộng cụ thể có thể xem ở tài liệu tham khảo [1]. Dưới đây là một ví dụ giúp độc giả hiểu chia đôi toàn cục tốt hơn.

Ví dụ 3. Library Nguồn đề: JOI2018 spring camp day 4 T2.

Trên bàn có n quyển sách xếp thành một hàng từ trái sang phải. Ta chỉ biết các quyển được đánh số $1 \sim n$, nhưng không biết thứ tự của các số. Mỗi lần, người quản lý có thể lấy đi một số quyển sách liên tiếp. Mỗi lần bạn có thể hỏi thư viện tương tác một tập số hiệu S ; thư viện trả về số lần ít nhất để người quản lý lấy đi mọi quyển sách có số hiệu trong S . Yêu cầu dùng các truy vấn để tìm ra thứ tự số hiệu của n quyển sách. Vì không thể phân biệt thứ tự trái-phải qua truy vấn, chỉ cần tìm được dãy số hiệu từ trái sang phải hoặc từ phải sang trái.

$$n \leq 1000, \quad \text{số lần hỏi giới hạn là } 20000.$$

Gọi $\text{ask}(S)$ là đáp án khi hỏi thư viện tương tác với tập S .

Ta có thể tìm thứ tự cuối cùng bằng cách hỏi số hiệu kề với từng quyển sách. Để thấy quyển sách có số hiệu x kề với

$$\text{ask}(S) - \text{ask}(S \cup \{x\}) + 1$$

quyển trong tập S ($x \notin S$).

Vì chỉ cần tìm $n - 1$ quan hệ kề nhau, ta có thể xét việc hỏi một quyển sách x kề với những quyển nào có số hiệu lớn hơn nó. Dùng chia đôi toàn cục để giải: khi chia đến khoảng $[l, r]$, duy trì số lượng quyển

¹Với gộp cây đoạn có lazy tag, chỉ cần cộng trực tiếp các tag và bảo đảm thông tin của mỗi nút trong cây đoạn là đúng nếu không xét lazy tag của tổ tiên.

trong tập $[l, r]$ kề với x ; bằng cách hỏi số quyền kề với x trong tập $[l, \text{mid}]$, với $\text{mid} = (l + r)/2$ (dưới đây cũng dùng ký hiệu này), ta đệ quy tính các khoảng $[l, \text{mid}]$ và $[\text{mid} + 1, r]$ có chứa quyền kề với x .

Có thể thấy mỗi ràng buộc kề nhau chỉ dùng $\lfloor \log n \rfloor$ lần hỏi, và nhiều nhất chỉ có $\lfloor n/2 \rfloor$ quyền làm lãng phí 2 lần hỏi. Vì vậy tối đa khoảng $2n \lfloor \log n \rfloor + n \approx 19000$ lần hỏi là đủ để nhận được đáp án.

4.2 Xây dựng bài toán mới

Xét việc xây dựng bài toán $S = \{a, b\}$ ($a < b$) mới trên cơ sở bài toán L_p : ngoài việc thỏa mọi ràng buộc của bài toán gốc, còn cần thỏa $a \leq f_i \leq b$, và phải cực tiểu hóa chi phí hồi quy.

4.3 Trường hợp $p = 1$

Bổ đề 4.1. Trong bài toán L_1 , nếu mọi y_i đều không nằm trong khoảng (a, b) , và tồn tại một dãy nghiệm tối ưu z sao cho mọi phần tử z_i cũng không nằm trong khoảng (a, b) , thì với một nghiệm tối ưu z_S của bài toán S , nhất định tồn tại một nghiệm tối ưu z của bài toán gốc sao cho làm tròn z về S sẽ được z_S .

Chứng minh. Dùng phản chứng. Giả sử nghiệm tối ưu của bài toán gốc là z^0 , trong đó phải có $z_i^0 \leq a, z_i^S = b$ hoặc $z_i^0 \geq b, z_i^S = a$. Vì hai loại tình huống này tương ứng với các đỉnh hiển nhiên không thể có quan hệ thứ tự bộ phận, nên không mất tính tổng quát ta chứng minh không tồn tại $z_i^0 \leq a, z_i^S = b$.

Đặt

$$y = \max\{z_i^0 \mid z_i^0 \leq a, z_i^S = b\},$$

$$U = \{v_i \mid z_i^0 = y, z_i^S = b\},$$

và giả sử khoảng các giá trị trung bình L_1 của U là $[l, r]^2$. Dễ thấy trên khoảng $(-\infty, l]$, chi phí hồi quy của tập U là hàm đơn điệu giảm; trên khoảng $[r, +\infty)$, nó là hàm đơn điệu tăng.

Nếu $l > a$, thì chỉnh các z_i^0 trong U thành $\min\{l, b\}$ sẽ cho bài toán gốc một nghiệm tốt hơn: vì y là giá trị lớn nhất trong số các vị trí có $z_i^S = b$, nên không tồn tại i, j ($v_i \in U, v_j \notin U, z_j^0 \in [y, a]$) sao cho $v_i \leq v_j$. Vì vậy nghiệm sau khi chỉnh vẫn hợp lệ. Điều này mâu thuẫn với giả thiết nghiệm tối ưu của bài toán gốc.

Nếu $l \leq a, r \geq b$, tương tự trường hợp $l > a$, chỉnh các z_i^0 trong U thành b nhất định sẽ nhận được một nghiệm khả thi của bài toán gốc; và vì $[a, b]$ nằm trong khoảng tối ưu, nghiệm khả thi này không kém hơn z^0 . Sau lần chỉnh này, số vị trí mà kết quả làm tròn của z^0 về S khác z_S giảm đi. Vì vậy sau hữu hạn thao tác, nhất định có thể chỉnh z^0 thành một dãy làm tròn về z_S . Điều này mâu thuẫn với giả thiết không tìm được nghiệm tối ưu z tương ứng.

Nếu $l \leq a, r < b$, đặt

$$U' = \{v_i \mid z_i^0 \leq a, z_i^S = b\}.$$

Vì $z_i^S = b$, nên không tồn tại i, j ($v_i \in U', v_j \notin U', z_j^0 \in [-\infty, a]$) sao cho $v_i \leq v_j$. Nếu $U = U'$, chỉnh các z_i^S trong U' thành a hiển nhiên sẽ cho một dãy tốt hơn z_S , mâu thuẫn với giả thiết z_S là nghiệm tối ưu của bài toán S . Tiếp theo giả sử khoảng các giá trị trung bình L_1 của $U' \setminus U$ là $[l', r']$. Nếu $r' \leq y$, thì sau khi chỉnh các phần tử trong $U' \setminus U$ theo cùng cách, cũng có thể nhận được một nghiệm tốt hơn cho bài toán S , mâu thuẫn với tính tối ưu của z_S . Nếu $r' \geq y$, có thể chỉnh các z_i^0 của $U' \setminus U$ thành y , nhận được một nghiệm khả thi của bài toán gốc không kém hơn z^0 . Nếu nghiệm này tốt hơn, ta mâu thuẫn với giả thiết z^0 là tối ưu; ngược lại, nghiệm tối ưu này nhất định thỏa $U' = U$, và lặp lại quá trình chứng minh trên sẽ tìm được mâu thuẫn. ■

²Giá trị trung bình L_1 có thể không duy nhất, nhưng mọi giá trị đó nhất định tạo thành một khoảng.

Theo Bổ đề 4.1, thông qua một nghiệm tối ưu của bài toán S , ta có thể biết quan hệ lớn nhỏ giữa z_i và a, b trong một nghiệm tối ưu nào đó của bài toán gốc. Với bài toán L_1 , vì z_i trong nghiệm tối ưu nhất định nằm trong tập $\{Y_1, Y_2, \dots, Y_m\}$ tạo bởi các y_i ($\forall i < m, Y_i < Y_{i+1}$), ta chỉ cần thực hiện chia đôi toàn cục trên tập này. Khi chia đến khoảng $Y_{[l,r]}$, chỉ cần tính nghiệm tối ưu của bài toán $S = \{Y_{\text{mid}}, Y_{\text{mid}+1}\}$ tạo bởi các v_i rơi vào khoảng này, rồi dựa vào z_i^S để chia các v_i đó sang các khoảng chọn giá trị mới $Y_{[l,\text{mid}]}$ và $Y_{[\text{mid}+1,r]}$. Như vậy với mỗi đỉnh v_i , chỉ cần qua $O(\log n)$ tầng tính toán là xác định được giá trị được chọn.

4.4 Trường hợp $1 < p < \infty$

Bổ đề 4.2. Khi $1 < p < \infty$, giá trị trung bình L_p của một tập U bất kỳ là duy nhất.

Chứng minh. Đặt

$$g(x) = \sum_{v_i \in U} |x - y_i|^p.$$

Khi đó

$$g(x) = \sum_{v_i \in U, y_i \leq x} (x - y_i)^p + \sum_{v_i \in U, y_i > x} (y_i - x)^p,$$

$$g'(x) = p \left(\sum_{v_i \in U, y_i \leq x} (x - y_i)^{p-1} - \sum_{v_i \in U, y_i > x} (y_i - x)^{p-1} \right).$$

Dễ thấy khi x tăng, tổng

$$\sum_{v_i \in U, y_i \leq x} (x - y_i)^{p-1}$$

đơn điệu tăng, còn tổng

$$\sum_{v_i \in U, y_i > x} (y_i - x)^{p-1}$$

đơn điệu giảm, nên phương trình $g'(x) = 0$ có nhiều nhất một nghiệm. Lại vì khi $x > \max\{y_i\}$ thì $g'(x) > 0$, còn khi $x < \min\{y_i\}$ thì $g'(x) < 0$, nên $g'(x)$ có ít nhất một điểm không. ■

Bổ đề 4.3. Nếu giá trị trung bình L_p của mọi tập con không rỗng của V đều không nằm trong khoảng (a, b) ($a < b$), và tồn tại một nghiệm tối ưu z sao cho mọi phần tử z_i cũng không nằm trong (a, b) . Gọi $\tilde{z}$ là một nghiệm tối ưu của bài toán L_1 trên cùng tập có thứ tự bộ phận, với hàm chi phí (y', w') :

$$(y'_i, w'_i) = \begin{cases} (0, w_i((b - y_i)^p - (a - y_i)^p)), & y_i \leq a, \\ (1, w_i((y_i - a)^p - (y_i - b)^p)), & y_i > a. \end{cases}$$

Khi đó tồn tại một nghiệm tối ưu z của bài toán gốc thỏa $\tilde{z}_i = 0$ khi và chỉ khi $z_i \leq a$.

Chứng minh. Vì đã bảo đảm giá trị trung bình L_p của mọi tập con sẽ không rơi vào (a, b) , nên nghiệm tối ưu của bài toán S ứng với bài toán L_p sẽ không có phần tử nào trong khoảng (a, b) . Do đó bài toán S tương đương với bài toán L_1 ở trên. Phần chứng minh còn lại giống Bổ đề 4.1, xin lược bỏ. ■

Theo Bổ đề 4.2, hợp của các phần tử trong nghiệm tối ưu và mọi giá trị trung bình L_p là một tập hữu hạn. Vì vậy với a bất kỳ, nhất định tồn tại $\varepsilon > 0$ sao cho khoảng $(a, a + \varepsilon)$ không chứa các phần tử trên; đồng thời có thể đổi w'_i trong bài toán L_1 ở Bổ đề 4.3 thành đạo hàm tại vị trí a . Vì trong thi tin học chỉ cần tìm nghiệm tối ưu đến một độ chính xác nhất định, ta vẫn có thể áp dụng cách chia đôi toàn cục ở mục 4.3, chỉ thay việc chia đôi trên tập bằng chia đôi trên số thực.

4.5 Xây dựng mô hình luồng mạng

Với bài toán L_1 trên tập có thứ tự bộ phận tổng quát, bài toán S tương ứng có thể xem là một bài toán quyết định 0/1 đơn giản. Ràng buộc $f_i \leq f_j$ có thể xem là: nếu f_i chọn 1, thì f_j không được chọn 0. Như vậy bài toán trở thành bài toán đồ thị con đóng có trọng số nhỏ nhất kinh điển, có thể giải bằng luồng mạng.

4.6 Bài tập ví dụ

Ví dụ 4. *ExtremeSpanningTrees* Nguồn đề: *Single Round Match 720 Round 1 - Division I, Level Three*.

Cho một đồ thị vô hướng liên thông $G = (V, E)$ có n đỉnh, m cạnh, và hai tập cạnh E_1, E_2 ($E_1, E_2 \subseteq E$). Mỗi cạnh có trọng số ban đầu d_i . Mỗi thao tác có thể tăng hoặc giảm trọng số của một cạnh đi 1. Hỏi cần ít nhất bao nhiêu thao tác để thỏa:

- đồ thị con tạo bởi tập cạnh E_1 là cây khung nhỏ nhất của toàn đồ thị;
- đồ thị con tạo bởi tập cạnh E_2 là cây khung lớn nhất của toàn đồ thị.

$$n \leq 50, \quad m \leq 1000.$$

Ta có thể xây dựng quan hệ thứ tự bộ phận trên m cạnh: tính các quan hệ thứ tự bộ phận giữa cạnh không thuộc cây i và cạnh thuộc đường đi tương ứng trên cây j . Sau đó bài toán trở thành một bài toán L_1 trên tập có thứ tự bộ phận tổng quát.

5 Tối ưu trên cấu trúc thứ tự bộ phận đặc biệt

5.1 Bài toán trên cây

Với bài toán hồi quy bảo toàn thứ tự trên cây ở mục 3.2, ta cũng có thể dựa trên chia đôi toàn cục rồi lồng cùng kiểu quy hoạch động trên cây để giải bài toán S .

Đặc biệt, thuật toán này có thể mở rộng sang xương rồng. Khi quy hoạch động trên mỗi chu trình, chỉ cần liệt kê giá trị 0/1 được chọn của một đỉnh để phá chu trình thành đường đi.

Dù là phiên bản trên cây hay trên xương rồng, độ phức tạp thời gian đều là $O(n \log n)$. Ở đây bỏ qua độ phức tạp tính lũy thừa $p - 1$ trong chi phí; dưới đây cũng vậy.

5.2 Thứ tự bộ phận nhiều chiều

Định nghĩa 5.1. Thứ tự bộ phận d chiều: với hai điểm d chiều $(a_1, a_2, \dots, a_d)$ và $(b_1, b_2, \dots, b_d)$, nếu $\forall i (1 \leq i \leq d), a_i \leq b_i$, thì có

$$(a_1, a_2, \dots, a_d) \leq (b_1, b_2, \dots, b_d).$$

Ví dụ 5. Cho tờ giấy ô vuông kích thước $r \times c$. Với điểm có tọa độ (i, j) , chi phí sửa đổi để tăng hoặc giảm trọng số đi 1 là $w_{i,j}$, trọng số ban đầu là $d_{i,j}$. Hãy tìm tổng chi phí sửa đổi nhỏ nhất sao cho với mọi $(i_1, j_1) \leq (i_2, j_2)$, đều có $d_{i_1, j_1} \leq d_{i_2, j_2}$.

Có thể tiếp tục dùng tư tưởng chia đôi toàn cục. Xét một số tính chất của bài toán S ở mỗi tầng chia đôi:

1. Tọa độ y ứng với mỗi cột i (tọa độ x) là một đoạn liên tiếp $[l_i, r_i]$.
2. l_i, r_i đều là các dãy đơn điệu không tăng.

3. Nếu cột thứ i không có phần tử (tức $l_i > r_i$), thì $\forall j, k (j < i < k)$, có $l_j > r_k$.

Theo các tính chất trên, có thể thấy khi chia đôi ở mỗi tầng, ta chỉ cần xét ảnh hưởng giữa hai cột kề nhau. Có thể thiết kế trực tiếp trạng thái quy hoạch động: $g_{i,j}$ biểu thị tổng chi phí nhỏ nhất của i cột đầu trong trường hợp tọa độ y nhỏ nhất được chọn b ở cột thứ i là j .

Chuyển trạng thái là

$$g_{i,j} = \left(\min_{k=j}^{r_{i-1}} g_{i-1,k} \right) + \text{cost}_{i,j},$$

trong đó $\text{cost}_{i,j}$ là chi phí của quyết định này trên cột thứ i . Dễ thấy có thể dùng hậu tố nhỏ nhất để tối ưu, độ phức tạp thời gian là $O(rc \log r)$.

Ví dụ 6. Cho n điểm trên mặt phẳng, trong đó điểm thứ i , tức v_i , có tọa độ (x_i, y_i) . Chi phí sửa đổi để tăng hoặc giảm trọng số đi 1 là w_i , trọng số ban đầu là d_i . Hãy tìm tổng chi phí sửa đổi nhỏ nhất sao cho với mọi $v_i \leq v_j$, đều có $d_i \leq d_j$.

Khác với bài trước, trong bài toán S ở mỗi tầng không thể bỏ qua ảnh hưởng chi phí giữa các cột không kề nhau. Trạng thái cần duy trì chi phí nhỏ nhất $g_{i,j}$ khi trong i cột đầu, tọa độ y nhỏ nhất được chọn b là j .

Có thể thấy $g_{i,j}$ có hai loại chuyển trạng thái sau:

1. Ở cột thứ i , chọn vị trí đầu tiên $\geq j$, ký hiệu next_j , làm vị trí đầu tiên được chọn b . Khi đó

$$g_{i,j} = g_{i-1,j} + \text{cost}_{i,\text{next}_j}.$$

2. Ở cột thứ i , chọn vị trí j . Khi đó

$$g_{i,j} = \left(\min_{k \geq j} g_{i-1,k} \right) + \text{cost}_{i,j}.$$

Dễ thấy loại thứ nhất tương đương cộng đoạn, loại thứ hai tương đương truy vấn nhỏ nhất trên đoạn và sửa điểm; cả hai đều có thể duy trì bằng cây đoạn. Vì quá trình chia chọn giá trị của chia đôi toàn cục cần dựa vào phương án nghiệm tối ưu của bài toán S , nên cần dùng cây đoạn bền vững để hiện thực. Độ phức tạp thời gian là $O(n \log^2 n)$.

6 Thuật toán và mở rộng của bài toán L_∞

6.1 Chặt nhị phân đơn giản

Vì bài toán L_∞ yêu cầu cực tiểu hóa giá trị lớn nhất, ta có thể nghĩ đến phương pháp chặt nhị phân. Sau mỗi lần chặt, có thể xác định khoảng giá trị được chọn $[a_i, b_i]$ cho mỗi đỉnh v_i . Sau đó làm quy hoạch động trên DAG, tính giá trị nhỏ nhất của f_i tại mỗi đỉnh v_i nếu chỉ thỏa mọi điều kiện $f_j \leq f_i$ ($v_j \leq v_i$), rồi kiểm tra tính khả thi. Như vậy ta nhận được một cách làm tốt cho bài toán L_∞ tổng quát với độ phức tạp $O(m \log \max\{y_i\})$.

6.2 Mở rộng bài toán

Ví dụ 7. Cho tập đỉnh $V = \{v_1, v_2, \dots, v_n\}$ kích thước n và hàm chi phí (y, w) . Với mọi $i, j (1 \leq i < j \leq n)$, đều có $v_i \leq v_j$.

Yêu cầu quyết định một dãy giá trị đỉnh f , thỏa với mọi $v_i \leq v_j$ ($i, j \in [1, n]$), đều có $f_i \leq f_j$.

Yêu cầu với mỗi $k (1 \leq k \leq n)$, tính giá trị nhỏ nhất của

$$\max_{i=1}^k w_i |f_i - y_i|.$$

Nếu áp dụng cách chặt nhị phân trong mục 6.1, ta cần xử lý nhiều truy vấn; vì không thể tiền xử lý nhanh, cũng không thể dùng chia đôi toàn cục để tối ưu. Do đó ta cần tìm một số tính chất.

Định nghĩa 6.1. $\text{err}(v_i, r)$ biểu thị chi phí hồi quy khi dùng giá trị đỉnh r tại v_i :

$$\text{err}(v_i, r) = w_i |r - y_i|.$$

$\text{mean}(v_i, v_j)$ biểu thị trung bình có trọng số của v_i, v_j :

$$\text{mean}(v_i, v_j) = \frac{w_i y_i + w_j y_j}{w_i + w_j}.$$

Khi $v_i \leq v_j$ và $y_i > y_j$, $\text{mean_err}(v_i, v_j)$ là giá trị lớn hơn trong hai giá trị

$$\text{err}(v_i, \text{mean}(v_i, v_j)), \quad \text{err}(v_j, \text{mean}(v_i, v_j));$$

ngược lại, $\text{mean_err}(v_i, v_j) = 0$.

Bổ đề 6.1. $\text{mean_err}(v_i, v_j)$ là chi phí hồi quy nhỏ nhất khi chỉ xét v_i và v_j .

Chứng minh. Khi $v_i \leq v_j$ hoặc $y_i \leq y_j$, hiển nhiên chọn $f_i = y_i, f_j = y_j$ cho chi phí hồi quy bằng 0.

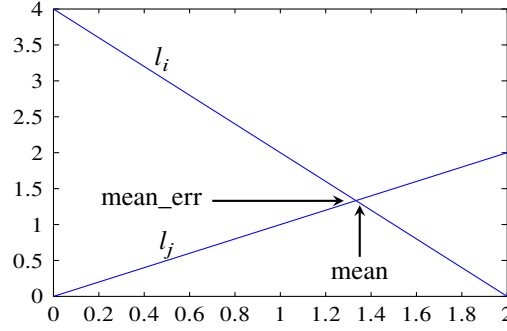


Figure 1: $y_i = 2, w_i = 2, y_j = 0, w_j = 1$.

Khi $v_i \leq v_j$ và $y_i > y_j$, theo Hình 1 (trong đó l_i là nhánh trái của hàm $y = \text{err}(v_i, x)$, còn l_j là nhánh phải của hàm $y = \text{err}(v_j, x)$), có thể thấy $\text{mean}(v_i, v_j)$ là giá trị trung bình L_∞ của tập $\{v_i, v_j\}$, và $\text{mean_err}(v_i, v_j)$ chính là chi phí hồi quy nhỏ nhất. ■

Bổ đề 6.2. Với bài toán L_∞ bất kỳ, chi phí hồi quy nhỏ nhất của toàn đồ thị là

$$\max\{\text{mean_err}(v_i, v_j)\}.$$

Chứng minh. Theo Bổ đề 6.1, dễ thấy chi phí hồi quy nhỏ nhất không nhỏ hơn $\max\{\text{mean_err}(v_i, v_j)\}$.

Ta chứng minh chi phí hồi quy nhỏ nhất không vượt quá $\max\{\text{mean_err}(v_i, v_j)\}$.

Với đỉnh v_k , có thể chọn giá trị đỉnh f_k là $\text{mean}(v_i, v_j)$ của một cặp v_i, v_j làm $\text{mean_err}(v_i, v_j)$ lớn nhất trong số các cặp thỏa $v_i \leq v_k \leq v_j$. Dưới đây chứng minh tính đúng đắn của cách chọn này theo ba bước.

Trước hết, nếu còn có một cặp v'_i, v'_j ($v'_i \leq v_k \leq v'_j$) thỏa $\text{mean_err}(v'_i, v'_j) = \text{mean_err}(v_i, v_j)$, thì nhất định

$$\text{mean}(v'_i, v'_j) = \text{mean}(v_i, v_j).$$

Nếu không, giả sử $\text{mean}(v'_i, v'_j)$ lớn hơn. Theo tính chất của đồ thị hàm, nếu $\text{mean_err}(v'_i, v'_j)$ không lớn hơn $\text{mean_err}(v_i, v_j)$, thì hệ số góc của l'_i phải không âm, mâu thuẫn với việc l'_i là nhánh trái của hàm err .

Tiếp theo, ta chứng minh $\text{err}(v_k, \text{mean}(v_i, v_j))$ không vượt quá $\text{mean_err}(v_i, v_j)$: nếu đồ thị $y = \text{err}(v_k, x)$ đi qua phần phía trên giao điểm của l_i, l_j , thì mean_err của v_k với v_i hoặc v_j sẽ lớn hơn $\text{mean_err}(v_i, v_j)$, mâu thuẫn với việc trước đó ta chọn một cặp lớn nhất.

Cuối cùng, ta chứng minh cách chọn giá trị này thỏa ràng buộc thứ tự bộ phận. Với quan hệ $v_a \leq v_b$, gọi C_a, D_a lần lượt là tập các tia tạo bởi nhánh trái của hàm err của mọi v_e thỏa $v_e \leq v_a$, và tập các tia tạo bởi nhánh phải của hàm err của mọi v_e thỏa $v_e \geq v_a$. Định nghĩa C_b, D_b tương tự. Để thấy C_b nhận được bằng cách thêm một số tia vào C_a , còn D_b nhận được bằng cách xóa một số tia khỏi D_a . Dùng quy nạp, có thể chuyển bài toán thành chứng minh rằng sau khi thêm một tia vào C_a hoặc xóa một tia khỏi D_a , trong mọi giao điểm, tọa độ x của điểm có tọa độ y lớn nhất sẽ không nhỏ đi. Xét bao lồi dưới tạo bởi C_a , giao điểm có tọa độ y lớn nhất nhất định nằm trên bao lồi; xóa một tia khỏi D_a chỉ làm tọa độ y lớn nhất giảm đi, và tọa độ x tương ứng dịch sang phải. Tương tự có thể chứng minh việc thêm một tia vào C_a cũng có tính chất như vậy. ■

Trong Ví dụ 7, ta có thể đặt

$$\text{pre}(v_i) = \max\{\text{mean_err}(v_j, v_i) \mid v_j \leq v_i\}.$$

Để thấy chỉ cần tính được mọi $\text{pre}(v_i)$ là có thể nhận được đáp án cho mọi k . Như vậy ta có một thuật toán $O(n^2)$.

Tiếp tục xét tối ưu quá trình tính $\text{pre}(v_i)$. Có thể theo tư duy tương tự phần chứng minh: duy trì bao lồi dưới tạo bởi nhánh trái của mọi hàm err ứng với $v_1 \sim v_i$, rồi truy vấn giao điểm giữa nhánh phải của hàm err của v_i và bao lồi.

Một cách duy trì khá đơn giản là xem đường thẳng dạng $y = kx + b$ như điểm (k, b) trên mặt phẳng, và duy trì bao lồi trên tạo bởi các điểm này. Khi đó mỗi lần chèn trở thành chèn nhị phân vị trí rồi xóa bỏ các điểm không còn nằm trên bao lồi; truy vấn trở thành chèn nhị phân để so sánh giao điểm giữa đường thẳng truy vấn với đường thẳng trong bao lồi và đoạn gấp khúc tương ứng trên bao lồi. Tất cả đều có thể duy trì đơn giản bằng set trong C++. Vì vậy ta nhận được cách làm tốt với độ phức tạp $O(n \log n)$.

Bài toán L_∞ còn có nhiều mở rộng phức tạp hơn; ở đây không trình bày từng cái. Nếu quan tâm, có thể xem tài liệu tham khảo [5].

7 Tổng kết

Bắt đầu từ mô tả bài toán hồi quy bảo toàn thứ tự L_p , bài viết lần lượt giới thiệu các cách làm cho $1 \leq p < \infty$ và $p = \infty$.

Trong phần $1 \leq p < \infty$, trước hết bài viết giới thiệu lời giải cho các tình huống đặc biệt từ góc nhìn tham lam và bảo trì đường gấp khúc; sau đó dùng tư tưởng chia đôi toàn cục hoàn toàn mới để giới thiệu cách làm cho bài toán tổng quát. Ba phương pháp đều có ưu và nhược điểm. Cách chia đôi toàn cục có thể giải bài toán tổng quát, đồng thời có thể tối ưu trên các cấu trúc thứ tự bộ phận đặc biệt, nhưng rất khó trả lời nhiều nhóm truy vấn khác nhau; còn cách tham lam có thể đạt độ phức tạp tuyến tính trên bài toán L_2 đặc biệt, và bảo trì đường gấp khúc có hằng số tốt hơn chia đôi toàn cục. Hơn nữa, cả hai cách này đều có thể đồng thời tính được đáp án của một số bài toán con, nhưng chúng chỉ giải được các trường hợp đặc biệt tương ứng.

Trong phần $p = \infty$, trước hết bài viết giới thiệu cách chặt nhị phân đơn giản; sau đó thông qua suy luận kết hợp hình học và đại số, rút ra kết luận tổng quát cho bài toán L_∞ , rồi vận dụng nó vào bài toán trên cấu trúc thứ tự bộ phận tuyến tính. Hai phương pháp cũng có ưu và nhược điểm riêng. Chặt nhị phân có thể giải bài toán trên cấu trúc thứ tự bộ phận bất kỳ, nhưng không thể đồng thời hỗ trợ giải nhiều bài

toán; còn vận dụng kết luận tổng quát tuy có thể đồng thời giải nhiều bài toán, nhưng rất khó tính nhanh trên thứ tự bộ phận tổng quát. Điều này nhắc ta phải chọn phương pháp phù hợp với đề bài.

Bài toán hồi quy bảo toàn thứ tự được giới thiệu trong bài này vẫn đáng được nghiên cứu sâu hơn. Hy vọng bài viết có thể đóng vai trò gợi mở, giúp độc giả tiếp tục nghiên cứu và tìm ra nhiều cách làm cũng như ứng dụng thú vị hơn.

Lời cảm ơn

- Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu.
- Cảm ơn thầy Li Shu của Trường Ngoại ngữ Nam Kinh đã quan tâm và hướng dẫn.
- Cảm ơn các huấn luyện viên đội tuyển quốc gia Zhang Ruizhe và Yu Linyun đã hướng dẫn.
- Cảm ơn Yang Jinzhao, Yang Qianlan và Xu Haike đã thảo luận các thuật toán này với tôi.
- Cảm ơn Wang Xiuhua và Xu Haike đã đọc duyệt bài viết này.
- Cảm ơn cha mẹ đã quan tâm và chăm sóc tôi.

Tài liệu tham khảo

1. Xu Haoran, *Bàn sơ lược về một số lời giải phi kinh điển cho bài toán cấu trúc dữ liệu*, tập luận văn đội tuyển quốc gia năm 2013.
2. Zhang Hengjie, *Một số kỹ thuật tối ưu DP*, tập luận văn đội tuyển quốc gia năm 2015.
3. Wikipedia, https://en.wikipedia.org/wiki/Isotonic_regression.
4. Quentin F. Stout, “Isotonic Regression via Partitioning”, *Algorithmica* 66 (2013), pp. 93–112.
5. Stout, Q. F., “Algorithms for L_∞ isotonic regression”, submitted (2011).

6 Bài 4: Báo cáo ra đề *Fim 4*

Wu Jinzhao, Trường Trung học số 2 Hàng Châu, Chiết Giang

Tóm tắt

Bài viết này giới thiệu đề *Fim 4*, một bài toán lấy bối cảnh khớp xương do tác giả ra trong đợt thi đối kháng của đội tuyển tập huấn. Đề khai thác sâu các tính chất liên quan tới chu kỳ của xương, đồng thời kết hợp tính chất đó với thuật toán KMP quen thuộc. Bài này đòi hỏi thí sinh hiểu khá sâu về xương, đồng thời cần năng lực cài đặt nhất định; đây là một đề tốt để kiểm tra chiều sâu tư duy của thí sinh.

Ngoài ra, cách chia điểm của đề hợp lý, dữ liệu mạnh, độ phân hóa cao, và có thể dẫn dắt thí sinh suy nghĩ tới lời giải chuẩn. Dù là thí sinh chỉ viết vét cạn, thí sinh đã suy nghĩ thêm một chút, hay thí sinh nghiên cứu sâu và tìm ra mọi tính chất, đều có thể nhận được phần điểm phù hợp.

Bài viết cũng cung cấp một hướng ra đề: với một thuật toán đã được biết rộng rãi, ta suy nghĩ đầy đủ về bản chất của nó, tìm những tính chất ít được dùng trong thuật toán gốc, rồi mở rộng thuật toán đó sang một bài toán tổng quát hơn.

1 Đại ý đề bài và phạm vi dữ liệu

1.1 Đề gốc

1.1.1 Đại ý đề gốc. Cho n xâu s_i và m số nguyên a_i nằm trong khoảng $1 \sim n$. Gọi xâu mẹ là

$$s_{a_1} + s_{a_2} + \dots + s_{a_m}.$$

Cần trả lời q truy vấn; mỗi truy vấn cho một xâu t_i , hỏi số lần xuất hiện của xâu này trong xâu mẹ.

Bảo đảm s_i và t_i đều chỉ gồm hai chữ cái a, b.

1.1.2 Phạm vi dữ liệu của đề gốc. Với mọi dữ liệu:

$$1 \leq n, m, q \leq 5 \times 10^4, \quad \sum_{1 \leq i \leq n} |s_i|, \quad \sum_{1 \leq i \leq q} |t_i| \leq 10^5.$$

- Subtask 1 (20 điểm):

$$\sum_{1 \leq i \leq m} |s_{a_i}| \leq 10^6.$$

- Subtask 2 (20 điểm):

$$\max |t_i| \leq \min |s_i|.$$

- Subtask 3 (30 điểm):

$$\max |t_i| \leq 200.$$

- Subtask 4 (30 điểm): không có tính chất đặc biệt.

Trên thực tế, lời giải tác giả đưa ra có thể giải một phiên bản mạnh hơn đề gốc. Đại ý của đề tăng cường được nêu dưới đây. Không khó thấy các truy vấn của đề gốc là một tập con thật sự của đề tăng cường, vì vậy chỉ cần giải đề tăng cường là đồng thời giải được đề gốc. Bài viết này cũng chủ yếu giới thiệu cách làm cho đề tăng cường.

1.2 Đề tăng cường

1.2.1 Đại ý đề tăng cường. Cho một cây Trie có n nút. Gọi xâu tạo bởi đường đi từ gốc đến nút thứ i là s_i .

Lại cho m số nguyên a_i nằm trong khoảng $1 \sim n$.

Cần trả lời q truy vấn; mỗi truy vấn cho một xâu t_i và hai số nguyên dương L_i, R_i . Đặt

$$S_i = s_{a_{L_i}} + s_{a_{L_i+1}} + \dots + s_{a_{R_i}},$$

hỏi số lần xuất hiện của t_i trong S_i .

Kích thước bảng chữ cái không vượt quá 10.

1.2.2 Phạm vi dữ liệu của đề tăng cường. Với mọi dữ liệu:

$$n \leq 10^6, \quad 1 \leq m, q \leq 5 \times 10^4, \quad \sum_{1 \leq i \leq q} |t_i| \leq 10^5.$$

- Subtask 1 (20 điểm):

$$\sum_{1 \leq i \leq m} |s_{a_i}| \leq 10^5.$$

- Subtask 2 (30 điểm):

$$\max |t_i| \leq 200.$$

- Subtask 3 (20 điểm):

$$\max |t_i| \leq \min |s_{a_i}|.$$

- Subtask 4 (30 điểm): không có tính chất đặc biệt.

2 Kiến thức chuẩn bị

Phần này giới thiệu sơ lược một số định nghĩa và thuật toán có thể cần dùng để giải bài này.

2.1 Khái niệm cơ bản về xâu

Xét một xâu $s_1 s_2 \dots s_n$ có độ dài n . Với $1 \leq i \leq j \leq n$, gọi $s_i s_{i+1} \dots s_j$ là một xâu con của s , ký hiệu $s[i : j]$. Ký hiệu $|s|$ là độ dài của s , tức $|s| = n$.

Với $1 \leq i \leq n$, gọi $s[i : n]$ là một hậu tố của s , ký hiệu $s[i :]$. Tương tự, với $1 \leq i \leq n$, gọi $s[1 : i]$ là một tiền tố của s , ký hiệu $s[: i]$.

Với $1 \leq i < n$, nếu $s[: i] = s[n - i + 1 :]$, thì gọi $s[: i]$ là một *border* của s . Đặc biệt, xâu rỗng cũng là một border của s . Border dài nhất của s được gọi là $\text{Lborder}(s)$.

Nếu $2|\text{Lborder}(s)| \geq n$, thì gọi s là xâu tuần hoàn; độ dài chu kỳ là $|s| - |\text{Lborder}(s)|$.

2.2 Một số ký hiệu

Quy ước xâu mẹ $S = s_{a_1} + s_{a_2} + \dots + s_{a_m}$. Ký hiệu lp_i là vị trí đầu trái của s_{a_i} trong S , và rp_i là vị trí đầu phải của s_{a_i} trong S . Gọi L là độ dài lớn nhất của các xâu truy vấn, và Σ là bảng chữ cái.

Định nghĩa $[cond]$ bằng 1 khi $cond$ đúng, và bằng 0 khi $cond$ sai.

Định nghĩa $c(T, S)$ là số lần xâu T xuất hiện trong xâu S .

Định nghĩa $\text{suf}(S)$ là tập mọi hậu tố của S , và $\text{pre}(S)$ là tập mọi tiền tố của S .

2.3 Automaton hậu tố

Automaton hậu tố của một xâu S là một automaton hữu hạn tất định (DFA) có thể và chỉ có thể chấp nhận mọi hậu tố của S , đồng thời có số trạng thái và số chuyển ít nhất. Dưới đây gọi tắt là SAM.

Với một xâu S , ta xây dựng SAM bằng phương pháp tăng dần trong thời gian $O(|S|)$. Cách xây dựng cụ thể và chứng minh độ phức tạp không liên quan đến chủ đề chính của bài viết nên không trình bày ở đây; có thể xem tài liệu tham khảo [5].

2.4 SAM on Trie

Với một cây Trie, định nghĩa tập xâu con của nó là tập các hậu tố của những xâu tạo bởi đường đi từ gốc đến một nút bất kỳ.

Ta xây dựng cho cây Trie một automaton hữu hạn có thể nhận biết mọi xâu con trên cây Trie, đồng thời cố gắng giảm số trạng thái và số chuyển của automaton; automaton đó được gọi là SAM on Trie.

Tính chất của SAM on Trie gần như hoàn toàn giống tính chất của SAM trên một dãy. Ví dụ: SAM on Trie có thể nhận biết mọi xâu con trong một Trie; cây parent của SAM on Trie là một cây hậu tố đảo

chiều, trong đó cha của mỗi nút là nút biểu diễn hậu tố dài nhất khác nhau về bản chất của xâu mà nút đó biểu diễn. “Khác nhau về bản chất” ở đây nghĩa là có số lần xuất hiện khác nhau trong Trie này.

Cách xây dựng SAM on Trie tương tự như trên đây. Mỗi lần mở rộng một nút, ta dùng nút tương ứng với cha của nó trên SAM làm last để tiếp tục mở rộng. Nếu xây dựng theo thứ tự FIFO, độ phức tạp thời gian là $O(n|\Sigma|)$, trong đó n là số nút của Trie và Σ là bảng chữ cái. Chứng minh độ phức tạp không liên quan đến chủ đề chính của bài viết nên không trình bày ở đây; có thể xem tài liệu tham khảo [1].

2.5 KMP

Dùng thuật toán KMP có thể tính số lần xâu t xuất hiện trong xâu s trong thời gian $O(|s| + |t|)$. Ngoài ra, nó còn có thể tính mọi $Lborder(t[: i])$ với $1 \leq i \leq n$. Quy trình cụ thể như sau:

Thuật toán 1. Thuật toán KMP.

```
j <- 0
np <- 0
for i <- 2 to |t| do
  while t[i] != t[j+1] and j != 0 do
    j <- p[j]
  end while
  if t[i] = t[j+1] then
    j <- j + 1
  end if
  p[i] <- j
end for
j <- 0
for i <- 1 to |s| do
  while s[i] != t[j+1] and j != 0 do
    j <- p[j]
  end while
  if s[i] = t[j+1] then
    j <- j + 1
  end if
  if j = |t| then
    pos[np] <- i
    np <- np + 1
    j <- p[j]
  end if
end for
```

2.6 Tìm nhanh tiền-hậu tố chung dài nhất của hai xâu

Ta cần giải bài toán sau: cho một Trie và một xâu T , cần trả lời nhanh truy vấn: với một nút nào đó trên Trie, tìm xâu dài nhất T' sao cho T' là tiền tố của T , đồng thời T' là hậu tố của xâu tạo bởi đường đi từ gốc của Trie đến nút này.

Trước hết xây dựng SAM của Trie đó. Xét từng tiền tố của T ; tại nút tương ứng với tiền tố này trên Trie, ta gán một thẻ có trọng số bằng độ dài tiền tố hiện tại.

Khi truy vấn, ta chỉ cần tìm tổ tiên trên cây parent có thể trọng số lớn nhất trong số các tổ tiên của nút SAM ứng với xâu tạo bởi đường đi từ gốc Trie đến nút đang xét. Chỉ cần dùng một cây LCT để duy trì cây parent là có thể hỗ trợ thao tác này trong thời gian $O(\log n)$.

3 Giới thiệu thuật toán

3.1 Thuật toán một

Với Subtask 1, chú ý rằng tổng độ dài xâu mẹ nằm trong phạm vi 10^5 , nên có thể trực tiếp dựng xâu mẹ.

Trước hết dựng SAM của xâu mẹ.

Sau đó dùng gộp cây đoạn bền vững để tiền xử lý tập Right của mỗi nút trên cây parent.

Với mỗi truy vấn, ta chỉ cần tìm nút tương ứng với xâu truy vấn t trên cây parent, rồi hỏi trong tập Right của nó có bao nhiêu phần tử nằm trong khoảng $[L + |t| - 1, R]$. Vì trước đó ta đã dùng cây đoạn bền vững để tiền xử lý tập Right của mỗi nút, chỉ cần truy vấn tổng trên khoảng này trong cây đoạn tương ứng với nút đó.

Độ phức tạp thời gian là

$$O\left(\sum |s_{a_i}| \log n + \sum |t_i| + q \log n\right).$$

Điểm kỳ vọng là 20 điểm.

3.2 Thuật toán hai

Xét Subtask 2; chú ý độ dài xâu truy vấn dài nhất chỉ không quá 200.

Khi một xâu truy vấn khớp trong xâu mẹ, nó hoặc xuất hiện bên trong một xâu từ điển đơn lẻ, hoặc bắc qua một số xâu từ điển. Ta xử lý riêng hai phần này.

3.2.1 Phần 1. Với mỗi xâu truy vấn t , cần tính

$$\sum_{i=L}^R c(t, S_i).$$

Đây là một bài toán khá thường gặp. Ta xây SAM on Trie cho Trie đã cho.

Tương tự thuật toán một, với mỗi nút trên SAM, ta dùng gộp cây đoạn bền vững để tiền xử lý số lần xuất hiện của xâu mà mỗi nút trên cây parent biểu diễn trong s_{a_i} .

Khi truy vấn, tìm nút tương ứng với xâu truy vấn t trên cây parent, rồi hỏi tổng trên khoảng $[L, R]$ trong cây đoạn tương ứng với nút này.

Độ phức tạp của phần này là

$$O(n|\Sigma| + n \log n + \sum |t_i|).$$

3.2.2 Phần 2. Nói một cách hình thức, tính số lần xuất hiện bắc qua nhiều xâu từ điển tức là cần tính

$$\sum_{i=L}^{R-1} c(t, S[\max(lp_i, rp_i - |t| + 2) : \min(rp_R, rp_i + |t| - 1)]).$$

Ta lấy mọi xâu

$$S[\max(lp_i, rp_i - L + 2) : rp_i + L - 1]$$

ra để xây Trie, rồi xây SAM cho Trie này. Với tiền tố

$$S[\max(lp_i, rp_i - L + 2) : rp_i + k - 1],$$

ta gán nhãn cho nó là k , và đặt chỉ số của nó là $rp_i + k - 1$.

Khi đó, việc thống kê

$$\sum_{i=L}^{R-1} c(t, S[\max(lp_i, rp_i - |t| + 2) : \min(rp_R, rp_i + |t| - 1)])$$

tương đương với thống kê trong cây con của nút biểu diễn t trên cây parent có bao nhiêu nút có nhãn nằm trong $[2, |t| - 1]$ và chỉ số nằm trong $[lp_L + |t| - 1, rp_R]$. Sau khi lấy thứ tự DFS trên cây parent, bài toán chuyển thành đếm điểm ba chiều.

Ta chú ý rằng chiều “nhân” của bài toán đếm điểm ba chiều này chỉ cỡ $O(L)$. Tổng số điểm cỡ $O(mL)$, số truy vấn cỡ $O(q)$. Ta có thể dễ dàng xử lý offline với chi phí thêm điểm $O(\log n)$ và truy vấn $O(L \log n)$. Vì vậy, tổng độ phức tạp của phần này là $O(mL \log n)$.

Tóm lại, ta có thể giải Subtask này trong độ phức tạp

$$O\left(n \log n + \sum |t_i| + mL \log n\right),$$

điểm kỳ vọng là 30 điểm.

3.3 Thuật toán ba

Với Subtask 3, chú ý rằng độ dài xâu truy vấn dài nhất cũng không vượt quá độ dài xâu từ điển ngắn nhất.

Điều này đem lại tính chất: một xâu truy vấn nhiều nhất chỉ bắc qua hai xâu từ điển.

Ta tiếp tục dùng ý tưởng của thuật toán hai: vẫn thống kê riêng phần nằm trong xâu từ điển và phần bắc qua xâu từ điển.

Đặt một tham số B . Với các truy vấn có độ dài không quá B , ta dùng thuật toán đã giới thiệu trong thuật toán hai; với truy vấn có độ dài lớn hơn B , ta dùng một thuật toán khác.

Vẫn theo thuật toán hai, xét các truy vấn có độ dài lớn hơn B . Định nghĩa

$$\tau(s, t) = \text{suf}(s) \cap \text{pre}(t).$$

Định lý 3.1. Gọi $\tau(s, t) = \{s[p_i :]\}$, trong đó p là một dãy tăng đơn điệu. Khi đó p có thể được biểu diễn thành $O(\log n)$ cấp số cộng nối tiếp nhau.

Chứng minh. $s[p_{i+1} :]$ là tiền tố của $s[p_i :]$, nên chỉ có hai trường hợp:

1. $|s[p_{i+1} :]| \leq |s[p_i :]|/2$: độ dài giảm một nửa. Tổng cộng chỉ có $O(\log n)$ lần giảm một nửa như vậy, không cần xử lý thêm.
2. $|s[p_{i+1} :]| > |s[p_i :]|/2$: khi đó $s[p_i :]$ nhất định là xâu tuần hoàn. Ta có thể nén các vị trí có dạng $p_k = p_i + (k - i)l$ thành một cấp số cộng, trong đó l là độ dài chu kỳ. Sau khi xử lý, $|s[p_j :]| < |s[p_i :]|/2$, với $j > i$ là chỉ số đầu tiên không còn nằm trong cấp số cộng. Trường hợp này cũng chỉ có $O(\log n)$ lần giảm một nửa.

■

Muốn tính số lần xuất hiện của mỗi xâu giữa s_{a_i} và $s_{a_{i+1}}$, ta có thể tính trước $\tau(s_{a_i}, t)$ và $\tau(t, s_{a_{i+1}})$, rồi gộp kết quả của hai phần. Chú ý miền giá trị của $O(\log n)$ đoạn cấp số cộng này nhất định đôi một không giao nhau, nên có thể gộp bằng cách tương tự merge sort; mỗi lần gộp tốn $O(\log^2 n)$.

Vấn đề còn lại là cách tính $\tau(s_{a_i}, t)$. Việc tính $\tau(t, s_{a_{i+1}})$ là đối xứng, dùng cùng cách làm.

Ta xây Trie cho các xâu từ điển, rồi xây SAM cho Trie này. Trong $O(m \log n)$ thời gian, với mỗi i tính

$$\max\{\text{LCP}(s_{a_i}[j:], t)\}.$$

Cách tính cụ thể đã được giới thiệu ở phần kiến thức chuẩn bị. Đây chính là p_1 cần tính. Sau đó dùng KMP tiền xử lý mảng next của t . Chú ý $s_{a_i}[p_i:]$ nhất định là một border của $s_{a_i}[p_1:]$, nên ta có thể dùng mảng next này để dựng toàn bộ mảng p .

Khi chọn $B = \sqrt{m} \log^{1.5} n$, độ phức tạp thời gian của toàn bộ thuật toán là nhỏ nhất, bằng

$$O(n|\Sigma| + n \log n + m\sqrt{m} \log^{1.5} n).$$

Điểm kỳ vọng là 20 điểm; kết hợp thuật toán một và hai có thể đạt 70 điểm.

3.4 Thuật toán bốn

Ta tiếp tục dùng thuật toán hai, và xét các truy vấn có độ dài lớn hơn B .

Trước hết giới thiệu hai bổ đề.

Bổ đề 3.1. Nếu p, q đều là chu kỳ của xâu S , và $p + q \leq |S|$, thì $\gcd(p, q)$ cũng là một chu kỳ của xâu S .

Chứng minh. Đặt $d = q - p$ ($p < q$). Khi đó từ $i - p > 0$ hoặc $i + q \leq |S|$ đều có thể suy ra $S_i = S_{i+d}$. ■

Bổ đề 3.2. Gọi l là độ dài chu kỳ. Với $i > 2l$, ta có $\text{next}_i = i - l$.

Chứng minh. Điều này tương đương với việc tiền tố độ dài $i > 2l$ có chu kỳ ngắn nhất là l . Dùng phản chứng: nếu tồn tại chu kỳ T ngắn hơn l , thì $T + l < i$. Theo Bổ đề 3.1, $\gcd(T, l)$ cũng là một chu kỳ của xâu này. Chú ý $\gcd(T, l) \mid l$, mâu thuẫn với việc l là chu kỳ ngắn nhất của toàn xâu. ■

Xét việc tối ưu thuật toán KMP. Ta đặt xâu truy vấn lên xâu mẹ và chạy KMP. Chú ý ta chỉ cần tính các khớp giữa các xâu.

Gọi i là con trở hiện tại của KMP trở vào xâu từ điển, j là con trở hiện tại của KMP trở vào xâu truy vấn, và L là độ dài xâu truy vấn.

Nếu tồn tại một k sao cho $i \in [lp_k, rp_k]$ và $i - j + 1 \in [lp_k, rp_k]$, thì phần đang khớp hiện tại nằm hoàn toàn trong một xâu, không cần được tính ở đây. Vì vậy ta trực tiếp đặt j bằng độ dài tiền tố dài nhất của xâu truy vấn sao cho tiền tố này là một hậu tố của s_{a_k} , rồi đặt i thành vị trí bắt đầu của hậu tố đó và tiếp tục chạy KMP.

Vì có thể tìm nhanh LCP của hai hậu tố, ta dễ dàng tính được vị trí mất khớp tiếp theo. Do đó, chỉ cần tối ưu quá trình nhảy theo next.

Sau khi mất khớp, nếu $\text{next}_j < j/2$, ta trực tiếp thực hiện $j := \text{next}_j$. Ngược lại:

- Nếu sau khi thực hiện $j := \text{next}_j$ vẫn mất khớp, theo Bổ đề 3.2, ta có thể nhảy thẳng đến vị trí $l + j \bmod l$, trong đó l là độ dài chu kỳ. Đây là Case (1).
- Nếu không, ta thống kê tiếp theo $S[i + 1:]$ còn khớp được bao nhiêu chu kỳ của $t[:j]$, rồi tìm vị trí mất khớp. Khi đó có hai khả năng: khả năng thứ nhất là $S[i + 1] = t_j$, tức Case (2), và ta tiếp tục khớp xuống dưới; khả năng thứ hai xử lý giống Case (1).

Còn một tình huống đặc biệt là đã khớp đến cuối xâu. Khi đó ta có thể tính $S[i + 1:]$ còn khớp được bao nhiêu chu kỳ của $t[:j]$; số chu kỳ khớp được chính là đóng góp vào đáp án. Sau đó xử lý giống trường hợp mất khớp.

Định lý 3.2. Thuật toán nêu trên có độ phức tạp thời gian $O(m \log n)$.

Chứng minh. Với Case (1), xét giá trị $j - (i - lp_k)$. Hiển nhiên khi $j - (i - lp_k) \leq 0$, ta sẽ đặt lại giá trị của i và j .

Trước hết, khi tìm LCP, giá trị $j - (i - lp_k)$ không đổi.

Trong quá trình nhảy next, nếu $\text{next}_j \leq j/2$, thực hiện $j := \text{next}_j$ sẽ làm j giảm ít nhất một nửa. Nếu $\text{next}_j > j/2$, khi thực hiện $j := l + j \bmod l$, ta có

$$l + j \bmod l \leq \left\lfloor \frac{2l}{3} \right\rfloor,$$

nên j giảm ít nhất một phần ba. Vì vậy mỗi thao tác Case (1) đều làm đại lượng đang xét giảm theo một tỉ lệ hằng số, và tổng độ phức tạp của Case (1) là $O(m \log n)$.

Với Case (2), trước hết đưa vào một định nghĩa về tầng. Đặt

$$b_i = [\text{next}_i \geq i/2],$$

và gọi đoạn liên tiếp toàn 1 thứ i từ trái sang phải trong dãy b là tầng thứ i .

Ta chứng minh thêm một tính chất về tầng.

Định lý 3.3. Gọi chu kỳ của tiền tố ở tầng thứ i là per_i . Khi đó

$$\frac{3}{2}|\text{per}_i| \leq |\text{per}_{i+1}|.$$

Chứng minh. Gọi pos_i là vị trí ngoài cùng bên phải của tầng thứ i . Hiển nhiên per_i và per_{i+1} đều là chu kỳ của $S[: \text{pos}_i]$.

Nếu $|\text{per}_i| + |\text{per}_{i+1}| \leq \text{pos}_i$, thì theo Bổ đề 3.2, $\gcd(|\text{per}_i|, |\text{per}_{i+1}|)$ cũng là một chu kỳ của $S[: \text{pos}_i]$. Chú ý $\gcd(|\text{per}_i|, |\text{per}_{i+1}|)$ là một ước của $|\text{per}_{i+1}|$, mâu thuẫn với việc per_{i+1} là chu kỳ ngắn nhất của tầng thứ $i + 1$. Do đó

$$|\text{per}_i| + |\text{per}_{i+1}| > \text{pos}_i.$$

Lại có $\text{pos}_i \geq 2|\text{per}_i|$; kết hợp hai bất đẳng thức thu được $\frac{3}{2}|\text{per}_i| \leq |\text{per}_{i+1}|$. ■

Với Case (2), theo Định lý 3.3 hiển nhiên tổng cộng chỉ có $O(\log n)$ tầng. Chú ý chỉ có hai loại thao tác: thao tác Case (1) mỗi lần chỉ lùi một tầng, còn thao tác Case (2) mỗi lần chỉ tăng một tầng. Vì vậy số thao tác Case (2) nhiều nhất chỉ là $m \log n$ lần.

Do đó tổng số lần mất khớp là $O(m \log n)$. Với phần tìm LCP, sau khi xây SAM on Trie của các xâu từ điển, chỉ cần hỏi LCA của các nút tương ứng trên SAM để nhận được LCP của chúng. Chú ý bài toán LCA có thể chuyển thành bài toán RMQ trên thứ tự Euler, nên có thể trả lời $O(1)$. Với phần đặt lại i, j , phần kiến thức chuẩn bị đã giới thiệu cách tìm tiền-hậu tố chung dài nhất của hai xâu trong $O(\log n)$ thời gian. Vì vậy tổng độ phức tạp thời gian cũng là $O(m \log n)$. ■

Tóm lại, chọn $B = \sqrt{m} \log n$ cho độ phức tạp thấp nhất:

$$O((q + m) \sqrt{m} \log n).$$

4 Cách sinh dữ liệu

Vì đây là bài toán liên quan đến khớp xâu, nếu chỉ sinh ngẫu nhiên trực tiếp thì gần như toàn bộ xâu sẽ không có chu kỳ, khiến vết cạn dễ dàng qua được. Do đó cần một cách xây dựng dữ liệu tinh tế hơn.

4.1 Cách xây dựng xâu từ điển

Ý tưởng cơ bản là trước hết sinh ngẫu nhiên một số xâu khá nhỏ sao cho chu kỳ của chúng càng nhỏ càng tốt. Sau đó lặp xâu nhỏ này nhiều lần để nhận được một xâu lớn. Tiếp theo cắt xâu lớn này thành nhiều xâu từ điển. Khi cắt cần bảo đảm xâu từ điển dài nhất không quá nhỏ, đồng thời số lượng xâu từ điển cũng không quá ít. Sau đó lại sinh thêm một số xâu từ điển là bội số của cùng một chu kỳ, và ngẫu nhiên thêm một số nhiều. Cuối cùng đưa các xâu đã sinh vào một Trie.

4.2 Cách xây dựng mảng a

Khi xây dựng mảng a , có thể dùng một xác suất nào đó để quyết định đoạn tiếp theo gồm các xâu có nhiều hay không có nhiều. Nhờ vậy vừa tránh được việc một xâu truy vấn chỉ bắc qua vài xâu kề nhau, vừa tránh được tình huống một xâu khớp thẳng đến cuối.

Ngoài ra, cũng phải bảo đảm $\sum_{i=1}^m |s_{a_i}|$ đủ lớn.

4.3 Cách xây dựng xâu truy vấn

Với xâu truy vấn, ta cũng đặt chu kỳ của nó bằng một trong các xâu nhỏ được sinh ngẫu nhiên lúc đầu, rồi thêm nhiều với xác suất khá thấp.

Về độ dài của xâu truy vấn, cần bảo đảm cả truy vấn dài lẫn truy vấn ngắn đều không quá ít. Tương tự, có ba cách sinh ngẫu nhiên: cách thứ nhất là toàn bộ đều là xâu khá dài, cách thứ hai là toàn bộ đều là xâu khá ngắn, cách thứ ba là độ dài tương đối đồng đều và cố gắng có càng nhiều độ dài khác nhau càng tốt.

4.4 Các cách xây dựng khác

Để bao phủ tốt hơn một số corner case, còn có một số dữ liệu dựng tay, chẳng hạn xâu toàn A, xâu ngẫu nhiên, v.v.

5 Duyệt cố và quá trình ra đề

Bài này do tôi và bạn Zhang Yubo cùng thảo luận và ra đề.

Bạn Zhang Yubo là người đầu tiên nghĩ ra đề bài ban đầu. Chúng tôi cho rằng một đề có ý nghĩa ngắn gọn như vậy chắc cũng sẽ có một lời giải đẹp và đơn giản. Trong quá trình suy nghĩ ban đầu, tôi và Zhang Yubo trước hết nghĩ tới thuật toán ba. Nhưng về sau chúng tôi phát hiện rằng nếu không có tính chất mỗi xâu truy vấn chỉ bắc qua hai xâu được khớp, thì chúng tôi không biết cách tính $\tau(t, S[i :])$ trong độ phức tạp tốt.

Sau đó chúng tôi nhìn lại các thuật toán khớp xâu đã học, hy vọng tìm được gợi ý từ đó. Cuối cùng chúng tôi phát hiện tính chất tốt rằng mảng next trong KMP và chu kỳ của xâu có quan hệ bổ sung cho nhau; điều này có thể tăng tốc quá trình khớp rất mạnh. Vì vậy chúng tôi tối ưu trên nền thuật toán KMP, thêm một số xử lý bổ sung, rồi nhận được thuật toán bốn.

Về cách đặt subtask, chúng tôi cũng cố ý đặt điểm cho các bộ phận của lời giải chuẩn và cho những ngã rẽ mà chính chúng tôi đã đi qua trong quá trình suy nghĩ bài này, nhằm dẫn dắt thí sinh suy nghĩ tới lời giải chuẩn.

Trong đợt thi đối kháng của đội tuyển tập huấn, do cân nhắc độ phức tạp cài đặt, chúng tôi đã giảm độ khó đề gốc qua ba phiên bản rồi mới đưa vào kỳ thi.

Với thuật toán ba, tôi và Zhang Yubo đều chỉ biết giải bài này dưới tính chất mỗi xâu truy vấn chỉ bắc qua hai xâu được khớp. Chúng tôi chưa có kết luận tốt về việc lời giải này có thể mở rộng tốt hơn hay không; hoan nghênh mọi người thảo luận với tôi.

6 Tổng kết

Đề bài khai thác sâu các tính chất liên quan đến chu kỳ của xâu, đồng thời áp dụng tính chất đó vào khớp xâu.

Trong bốn subtask của đề, Subtask 1 là thuật toán thô, độ khó về kiến thức, tư duy và mã nguồn đều rất thấp. Subtask 2 cần dùng kiến thức liên quan tới SAM on Trie, nhưng sau khi phân tích đề một chút cũng không khó nhận ra lời giải này. Subtask 3 đòi hỏi thí sinh hiểu tính chất của border, nên độ khó tăng thêm một chút. Subtask 4 ngoài việc nắm vững thuật toán truyền thống, còn yêu cầu thí sinh có năng lực đổi mới trên thuật toán truyền thống sau khi quan sát tính chất; đây là một kiểm tra khá cao đối với trình độ xử lý xâu của thí sinh. Dù là thí sinh chỉ viết vét cạn, thí sinh đã suy nghĩ thêm một chút, hay thí sinh nghiên cứu sâu và tìm ra mọi tính chất, đều có thể nhận được phần điểm phù hợp.

Trong kỳ thi thực tế, có 4 bạn đạt 20 điểm và 1 bạn qua được bài này. Sau kỳ thi, tôi trao đổi với một số bạn về suy nghĩ của họ đối với bài này, và phát hiện đa số bạn nghĩ tới thuật toán hai, một vài bạn nghĩ tới thuật toán ba, nhưng vì nhiều lý do chỉ viết được vét cạn cơ bản nhất. Có thể thấy đề này có độ phân hóa tốt.

Nhìn chung, đây là một bài vừa yêu cầu thí sinh nắm được một số tính chất thường gặp trong xâu, vừa yêu cầu họ thuần thục các thuật toán xâu cơ bản. Đồng thời, bài này cũng có độ khó cài đặt khá lớn; độ khó tổng thể gần với CTSC những năm gần đây.

Về cá nhân tôi, tôi rất thích ý tưởng liên quan tới thuật toán bốn. KMP là một thuật toán kinh điển. Sau khi đào sâu tính chất của KMP, chúng tôi áp dụng thuật toán KMP vào một bối cảnh mới như vậy, qua đó kiểm tra việc nắm vững và hiểu thuật toán cơ bản này của mọi người. Điều này nhắc ta rằng với một thuật toán quen thuộc, có lẽ ta vẫn có thể tiếp tục khai thác tính chất của nó, từ đó ra được một số đề thú vị.

7 Lời cảm ơn

- Cảm ơn bạn Zhang Yubo đã giúp đỡ rất nhiều cho việc viết bài này, đồng thời cảm ơn bạn đã luôn giúp đỡ và đồng hành cùng tôi trên con đường OI.
- Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu.
- Cảm ơn huấn luyện viên đội tuyển quốc gia Zhang Ruizhe đã hướng dẫn.
- Cảm ơn thầy Li Jian cùng cha mẹ đã quan tâm và chỉ dẫn trong nhiều năm.
- Cảm ơn Wang Xiuhuan, Wu Yue, Dong Weijun và Li Ningyuan đã đọc duyệt bài viết này.

Tài liệu tham khảo

1. Liu Yanyi, *Mở rộng automaton hậu tố trên cây Trie*, tập luận văn đội tuyển quốc gia năm 2015.
2. Wikipedia, *Knuth–Morris–Pratt algorithm*, https://en.wikipedia.org/wiki/KMP_algorithm.
3. Jin Ce, *Chuyên đề thuật toán xâu*, trại mùa đông NOI năm 2017.
4. Wang Jianhao, *Bàn sơ lược về một số phương pháp khớp xâu*, tập luận văn đội tuyển quốc gia năm 2015.

5. Zhang Tianyang, *Automaton hậu tố và ứng dụng*, tập luận văn đội tuyển quốc gia năm 2015.

7 Bài 5: Một số kỹ thuật và công cụ giải bài toán khối liên thông trên cây

Ren Xuandi, Trường Trung học số 1 Thiệu Hưng

Tóm tắt

Bài toán khối liên thông trên cây là một lớp bài toán được ưa chuộng trong thi tin học. Bài viết này giới thiệu một số kỹ thuật và công cụ thường dùng để giải lớp bài toán đó. Với các bài toán DP trên khối liên thông của cây, bài viết giới thiệu những kỹ thuật quen thuộc như chuyển trạng thái theo thứ tự DFS và phân trị theo điểm, “số đỉnh trừ số cạnh”, DP toàn cục bằng gộp cây đoạn, và bảo trì DP động bằng phân trị theo chuỗi, cùng các ứng dụng tương ứng. Với bài toán bảo trì khối liên thông cùng màu trên cây, bài viết khảo sát tư duy chung và giới thiệu một cấu trúc dữ liệu tổng quát dựa trên LCT, hỗ trợ đổi màu cả đường đi và bảo trì thông tin trên một thứ tự DFS bất kỳ.

1 Dẫn nhập

Cây là một cấu trúc rất đặc biệt. Mọi đồ thị con liên thông của một cây, tức một khối liên thông trên cây, cũng là một cây; giao của một số khối liên thông trên cây cũng là một cây. Trong quá trình học tập, tác giả gặp nhiều bài toán liên quan tới khối liên thông trên cây, và đại thể chia chúng thành hai nhóm lớn: bài toán DP đếm hoặc tìm giá trị tối ưu trên khối liên thông của cây, và bài toán bảo trì thông tin của các khối liên thông cùng màu trên cây.

Trong những năm gần đây, nhiều bài toán liên quan tới khối liên thông trên cây có rất nhiều cách giải, lại thay đổi theo từng đề, nên thường khiến người học khó biết bắt đầu từ đâu. Bài viết này sắp xếp và tổng kết một số phương pháp, tư duy thường gặp khi giải bài toán khối liên thông trên cây. Riêng với bài toán bảo trì thông tin khối liên thông cùng màu trên cây, bài viết giới thiệu một cấu trúc dữ liệu tổng quát hỗ trợ đổi màu theo đường đi và bảo trì thông tin trên một thứ tự DFS bất kỳ, nhằm giúp độc giả có một lộ trình rõ hơn khi gặp lớp bài toán này.

Bài viết chủ yếu gồm hai phần. Phần thứ nhất đi từ vài ví dụ, lần lượt giới thiệu ứng dụng của chuyển trạng thái theo thứ tự DFS và phân trị theo điểm, “số đỉnh trừ số cạnh”, DP toàn cục bằng gộp cây đoạn, và bảo trì DP động bằng phân trị theo chuỗi trong các bài toán DP trên khối liên thông của cây. Phần thứ hai xoay quanh bài toán khối liên thông cùng màu trên cây: thông qua hai ví dụ kinh điển, bài viết trình bày tư duy chung của lớp bài toán này, rồi giới thiệu cấu trúc dữ liệu Link Cut Memphis, có thể đổi màu theo đường đi và bảo trì thông tin trên thứ tự DFS.

2 Định nghĩa và quy ước liên quan

Một đồ thị vô hướng $G = \{V, E\}$ được gọi là cây khi và chỉ khi giữa hai đỉnh bất kỳ có đúng một đường đi đơn. Mọi đồ thị vô hướng liên thông và không có chu trình đều là cây.

Một khối liên thông trên cây $T = \{V, E\}$ là một cây $T' = \{V', E'\}$ thỏa $V' \subseteq V$, $E' \subseteq E$ và T' liên thông. Dễ thấy cấu trúc của T' cũng là một cây.

Các bài toán DP trên khối liên thông của cây trong bài viết này thường có một trong hai dạng sau:

- Bài toán đếm: cho một cây T , hỏi có bao nhiêu khối liên thông trên cây thỏa tính chất A .
- Bài toán tối ưu: cho một cây T và một hàm giá trị, hỏi giá trị tốt nhất trong mọi khối liên thông trên cây T thỏa một tính chất nào đó.

Các bài toán bảo trì thông tin khối liên thông cùng màu trên cây trong bài viết này thường có dạng: cho một cây T , mỗi đỉnh có hai màu đen hoặc trắng. Định nghĩa một khối liên thông trên cây là hợp lệ khi và chỉ khi mọi đỉnh trong đó có cùng màu. Truy vấn hỏi thông tin của khối liên thông hợp lệ cực đại chứa đỉnh x .

Ngoài hai dạng này còn có thể có một số bài toán ít gặp hơn liên quan tới khối liên thông trên cây; chúng không nằm trong phạm vi thảo luận của bài viết này.

3 Bài toán DP trên khối liên thông của cây

3.1 Thuật toán thô

Với một bài toán DP trên khối liên thông của cây nói chung, cách thô phổ biến là đặt dp_x là số phương án hoặc đáp án tối ưu khi chọn một khối liên thông trong cây con của x và khối đó chứa x . Giá trị dp_x có thể thu được bằng cách gộp các giá trị dp của con x , tức duyệt mọi con và quyết định trong cây con đó bỏ trống hay chọn một khối liên thông chứa gốc.

Nếu gộp thông tin của hai con tốn $O(1)$, chẳng hạn đếm số khối liên thông khác nhau về bản chất trên cây, độ phức tạp tổng là $O(n)$. Nếu gộp hai con tốn $O(size_a \cdot size_b)$, chẳng hạn đếm số khối liên thông khác nhau về bản chất gồm đúng k đỉnh, thì hai đỉnh bất kỳ sẽ phát sinh độ phức tạp 1 tại LCA của chúng; độ phức tạp tổng là $O(n^2)$.

Thuật toán thô này đều dùng được cho bài toán đếm và bài toán tối ưu.

3.2 Chuyển theo thứ tự DFS và phân trị theo điểm

Với một lớp bài toán DP trên khối liên thông của cây, nếu việc gộp thông tin không đủ hiệu quả nhưng thêm thông tin của một đỉnh đơn lại khá hiệu quả, ta thường có thể chuyển trạng thái theo thứ tự DFS, biến thao tác gộp cây con thành thao tác thêm một đỉnh. Mô hình thường gặp là ba lô: giả sử giới hạn thể tích là m , gộp hai ba lô tốn $O(m^2)$, còn thêm một vật phẩm tốn $O(m)$.

Trước hết giả sử khối liên thông được chọn bắt buộc chứa gốc. Lấy thứ tự DFS của toàn cây; cấu trúc của một khối liên thông chứa gốc nhất định có dạng toàn cây sau khi xóa đi một số cây con đôi một không giao nhau. Trong thứ tự DFS, những phần bị xóa chính là một số đoạn liên tiếp.

Đặt dp_i là thông tin sau khi đã xét i đỉnh đầu tiên trong thứ tự DFS. Nếu chọn đỉnh thứ i , ta chuyển sang dp_{i+1} . Nếu không chọn, ta chuyển sang dp_j , trong đó j là đầu phải của đoạn DFS ứng với cây con của đỉnh thứ i , cộng thêm 1; điều này biểu thị việc xóa cả cây con đó.

Như vậy mỗi bước chỉ cần thêm một vật phẩm vào ba lô, thay vì gộp hai ba lô. Nếu thêm một đỉnh tốn $O(m)$, độ phức tạp tổng là $O(nm)$.

Với bài toán mà khối liên thông được chọn không nhất thiết chứa gốc, chỉ cần dùng phân trị theo điểm: mỗi lần chọn trọng tâm làm gốc, tính số phương án bắt buộc chứa trọng tâm, rồi xóa trọng tâm và đệ quy trên từng khối liên thông còn lại. Độ phức tạp tổng là $O(n \log n \cdot m)$.

Ví dụ 1. Nguồn: *Tsinghua Training 2016*, bài con Cây con liên thông.

Đề bài. Cho một cây, mỗi đỉnh có một màu. Cho ba màu u, v, w , hỏi có bao nhiêu khối liên thông trên cây sao cho số đỉnh mang màu u, v, w lần lượt là a, b, c .

Cách làm. Cách thô: DP từ dưới lên, dùng $dp_{x,i,j,k}$ biểu thị số phương án sau khi xử lý cây con gốc x , trong đó số lượng ba màu lần lượt là i, j, k . Khi chuyển trạng thái và gộp hai cây con, ta liệt kê số đỉnh

của ba màu trong từng cây con; độ phức tạp là $O(na^2b^2c^2)$.

Chú ý việc gộp hai cây con ở đây là một phép tích chập ba chiều. Nếu dùng FFT ba chiều để tối ưu, độ phức tạp có thể giảm thành $O(nabc \log abc)$, nhưng hằng số và lượng mã đều khá lớn.

Xét dùng thuật toán phân trị theo điểm nói trên, kết hợp với chuyển theo thứ tự DFS. Độ phức tạp là $O(nabc \log n)$, hằng số rất nhỏ, và cũng có thể mở rộng thuận tiện sang bài toán tối ưu.

3.3 “Số đỉnh trừ số cạnh”

Với một cây không rỗng, số đỉnh trừ số cạnh luôn bằng 1. Ta có thể dùng đẳng thức này để thống kê số khối liên thông trên cây thỏa một tính chất nào đó: liệt kê từng đỉnh hoặc từng cạnh, tính số khối liên thông hợp lệ chứa đỉnh hoặc cạnh đó, rồi lấy đáp án theo đỉnh trừ đáp án theo cạnh. Khi đó mỗi khối liên thông hợp lệ và không rỗng được tính đúng một lần.

Kỹ thuật này thường dùng trong trường hợp cần xét giao của một số khối liên thông trên cây, vì giao của một số khối liên thông trên cây vẫn là một cây, có thể rỗng. Số phương án để giao của nhiều khối liên thông không rỗng không dễ tính trực tiếp; nhưng nếu chỉ xét một đỉnh hoặc một cạnh, điều kiện trở thành mỗi khối liên thông đều phải chứa đỉnh hoặc cạnh đó. Các khối liên thông độc lập với nhau, nên bài toán trở nên tương đối dễ tính.

Ví dụ 2: Tập hoàn hảo. *Nguồn: thi đối kháng đội tuyển tập huấn 2018, Liang Yancheng.*

Đề bài. Cho một cây; mỗi đỉnh có trọng lượng và giá trị, mỗi cạnh có trọng số. Xét việc chọn một tập đỉnh S sao cho tổng trọng lượng các đỉnh không vượt quá M và các đỉnh đó tạo thành một khối liên thông trên cây. Những tập S có tổng giá trị lớn nhất được gọi là tập hoàn hảo.

Nếu hai đỉnh x, y thỏa $\text{dist}(x, y) \cdot v_y \leq \text{Max}$, thì x có thể kiểm tra y . Hỏi có bao nhiêu cách chọn k tập trong tất cả các tập hoàn hảo sao cho trong giao của chúng tồn tại một đỉnh x có thể kiểm tra mọi đỉnh thuộc các tập đó.

Đáp án lấy modulo 523. Giới hạn:

$$n \leq 60, \quad m \leq 10000, \quad k \leq 10^9.$$

Cách làm. Tập các đỉnh có thể kiểm tra mọi đỉnh trong một tập là một khối liên thông trên cây. Giao của một số khối liên thông trên cây vẫn là một khối liên thông trên cây. Xét dùng kỹ thuật “số đỉnh trừ số cạnh”: thống kê số tập hoàn hảo có thể được một đỉnh x nào đó kiểm tra, rồi trừ đi số tập hoàn hảo có thể được đồng thời hai đầu mút của một cạnh kiểm tra.

Xét cách tính số tập hoàn hảo có thể được đỉnh x kiểm tra. Những tập này chắc chắn không được chứa bất kỳ đỉnh nào có khoảng cách tới x không hợp lệ. Sau khi xóa các đỉnh không hợp lệ, ta làm DP trên cây còn lại; bài toán tương đương hỏi có bao nhiêu khối liên thông trên cây có tổng trọng lượng không vượt quá M và tổng giá trị lớn nhất. Đây là bài toán ba lô tối ưu kinh điển: chỉ cần vừa ghi giá trị tốt nhất trong ba lô, vừa ghi số phương án đạt giá trị đó. Lấy x làm gốc và chuyển theo thứ tự DFS, độ phức tạp là $O(nm)$.

Cuối cùng còn phải xử lý tổ hợp modulo, không liên quan nhiều đến chủ đề bài viết nên không triển khai. Độ phức tạp tổng là $O(n^2m + \text{độ phức tạp tính tổ hợp modulo})$.

3.4 DP toàn cục bằng gộp cây đoạn

Có một lớp bài toán đếm khối liên thông trên cây như sau: mỗi đỉnh cần bảo trì một mảng DP kích thước m ; khi gộp hai cây con, thao tác là gộp theo từng vị trí tương ứng trong m vị trí; khi thêm một đỉnh, chỉ

cần sửa một vị trí nào đó. Dù số trạng thái DP là $O(nm)$, phần lớn trạng thái chỉ lặp lại việc gộp từ trạng thái của con. Mặt khác, thao tác “thêm một đỉnh” có quy mô $O(n)$. Nếu ta có thể hoàn thành nhanh việc gộp theo từng vị trí, lớp bài toán này sẽ được giải quyết tốt.

Xét dùng gộp cây đoạn: với mỗi đỉnh, dùng một cây đoạn để bảo trì m giá trị DP của nó. Khi thêm một đỉnh, ta sửa trong cây đoạn của nó; khi gộp hai cây con, ta dùng gộp cây đoạn để hoàn thành chuyển trạng thái hàng loạt. Tổng số nút được chèn vào các cây đoạn là $O(n \log n)$, còn độ phức tạp của gộp cây đoạn không vượt quá tổng độ phức tạp chèn.

Ví dụ 3. Nguồn: THUWC 2018.

Đề bài. Cho một cây, mỗi đỉnh có một màu. Hỏi có bao nhiêu khối liên thông trên cây chứa không quá 2 màu.

Cách làm. Đặt $f(x, c)$ là số phương án chọn một khối liên thông chứa x trong cây con của x , sao cho hai màu là col_x và c .

Xét màu của một con y của x , không khó nhận được chuyển trạng thái: nếu y cùng màu với x , nhân trực tiếp mọi $f(y, \sim) + 1$ vào $f(x, \sim)$; ngược lại, nhân $f(y, \text{col}_x) + 1$ vào $f(x, \text{col}_y)$.

Loại chuyển thứ hai tổng cộng chỉ cần chuyển $O(n)$ trạng thái. Nút thất nằm ở loại chuyển thứ nhất: mỗi khi có một con cùng màu, ta phải tốn độ phức tạp bằng số màu trong cây con.

Quan sát rằng các màu khác nhau độc lập với nhau. Vì vậy có thể dùng cây đoạn để bảo trì mọi giá trị DP tại một đỉnh; khi gộp cây con, thực hiện gộp cây đoạn là được.

Độ phức tạp là $O(n \log n)$.

3.5 Bảo trì DP động bằng phân trị theo chuỗi

DP động là bài toán hỗ trợ sửa đầu vào của DP, và sau mỗi lần sửa phải nhanh chóng tính lại giá trị DP; nó thường xuất hiện trong bài toán đếm. Trong luận văn đội tuyển tập huấn năm 2017, Chen Junkun đề xuất một thuật toán có tính mở rộng để giải bài toán DP động trên cây: phân trị theo chuỗi. Ở đây ta xét cách áp dụng phân trị theo chuỗi vào bài toán DP động trên khối liên thông của cây.

Cụ thể, ta cần hỗ trợ sửa thông tin của đỉnh và sau mỗi lần sửa lại hỏi số khối liên thông trên cây.

Tư tưởng của phân trị theo chuỗi là phân rã nặng-nhẹ trên cây, rồi tính DP theo thứ tự các chuỗi nặng từ dưới lên. Với một chuỗi nặng, thông tin DP của các chuỗi nặng con rẽ ra từ nó đã được tính xong; ta có thể gắn các thông tin đó lên chuỗi nặng hiện tại, khi đó bài toán trở thành DP động trên một dãy. Trên dãy, ta thường có thể dùng cấu trúc dữ liệu để hỗ trợ sửa thông tin của một vị trí và nhanh chóng bảo trì giá trị DP.

Khi sửa thông tin của một đỉnh, theo cấu trúc phân trị này chỉ có $O(\log n)$ chuỗi nặng trên tổ tiên bị ảnh hưởng. Ta lần lượt đi từ dưới lên, sửa và bảo trì giá trị DP trong cấu trúc dữ liệu của từng chuỗi nặng.

Nói chung, đặt $G(x)$ là đáp án khi chọn một khối liên thông chứa gốc trong cây con của x . Với một chuỗi nặng, giả sử các đỉnh trên đó theo thứ tự độ sâu tăng dần là $x_1, x_2, \dots, x_k$. Với mỗi đỉnh x_i , ta đã tính được các giá trị $G()$ của mọi con nhẹ; gộp các thông tin này với thông tin của bản thân x_i sẽ thu được đóng góp khi chọn đỉnh x_i trên chuỗi nặng. Gọi giá trị này là $F(x_i)$.

Nếu khối liên thông được chọn giao với chuỗi nặng này, giao đó chắc chắn là một đoạn, và đóng góp là tích các $F(x)$ trong đoạn. Do đó cần tính tổng tích $F(x)$ trên mọi đoạn của chuỗi nặng. Chỉ cần dùng cây đoạn hoặc cây cân bằng để bảo trì tích thông tin trong đoạn, tổng tích của mọi tiền tố và hậu tố, là có thể thuận tiện hỗ trợ sửa một vị trí và tính lại DP trong thời gian $O(\log n)$.

Đóng góp của chuỗi nặng này cho chuỗi nặng cha chính là số phương án chọn một tiền tố trên chuỗi nặng. Chỉ cần dùng thông tin tiền tố do cây đoạn hoặc cây cân bằng bảo trì để cập nhật giá trị $G()$ của đỉnh cha.

Đến đây khung thuật toán đã khá rõ: phân rã nặng-nhẹ trên cây, tính DP theo thứ tự các chuỗi nặng từ dưới lên. Khi sửa thông tin của một đỉnh, với mỗi chuỗi nặng trên tổ tiên của nó, tính lại bằng cây đoạn hoặc cây cân bằng số phương án chọn một tiền tố, rồi dùng số phương án này cập nhật thông tin của cha chuỗi nặng.

Nếu thông tin cần bảo trì có phép trừ, sau khi sửa thông tin của một chuỗi nặng con, chỉ cần trừ thông tin cũ và cộng thông tin mới. Nếu không, với mỗi đỉnh còn cần mở một cây đoạn hoặc cây cân bằng kích thước $O(\text{số con nhẹ})$ để nhanh chóng bảo trì thông tin của các con nhẹ. Tuy vậy độ phức tạp không đổi.

Ví dụ 4: Trò chơi cắt cây. Nguồn: SDOI 2017 Round 2 Day 1.

Đề bài. Cho một cây, mỗi đỉnh có một trọng số. Có hai loại thao tác:

1. Change $x\ y$: đổi trọng số của đỉnh số x thành y .
2. Query k : hỏi có bao nhiêu cây con liên thông không rỗng có xor trọng số các đỉnh đúng bằng k .

In đáp án modulo 10007. Giới hạn:

$$n, Q \leq 30000, \quad 0 \leq A_i, y, k < 128.$$

Cách làm. Trước hết chuyển toàn bộ giá trị xor thành giá trị điểm của FWT; trong quá trình tính toán, mọi thao tác đều dùng giá trị điểm, cuối cùng mới FWT ngược lại. Đặt m là giá trị trọng số lớn nhất; độ phức tạp gộp thông tin là $O(m)$.

Xét áp dụng thuật toán phân trị theo chuỗi. Đặt $G(x, \sim)$ là giá trị điểm khi chọn một khối liên thông chứa gốc trong cây con của x . Với một đỉnh trên chuỗi nặng, tính tích các $G(y, \sim) + 1$ của mọi con nhẹ y , rồi dùng cấu trúc dữ liệu bảo trì số phương án chọn một đoạn; tiếp đó dùng số phương án chọn một tiền tố để cập nhật $G(fa, \sim)$ của cha chuỗi nặng.

Vì thông tin được bảo trì là tích, khi bỏ đóng góp cũ của một con nhẹ có thể gặp vấn đề chia cho 0. Một cách xử lý là bảo trì tích của các giá trị khác 0 modulo 10007 và số mũ của 10007.

Dùng phân rã chuỗi trên cây, độ phức tạp là $O(qm \log^2 n)$.

Nếu dùng LCT để bảo trì cấu trúc phân trị theo chuỗi, khi Access ta động cập nhật quan hệ chuỗi nặng-nhẹ và bảo trì đáp án. Độ phức tạp có thể đạt $O(qm \log n)$; hơn nữa, cách này truy vấn thông tin của một số cấu trúc con thuận tiện hơn, chẳng hạn đáp án trong cây con của một đỉnh, hoặc đáp án của các khối liên thông có giao với một đường đi $u \sim v$, đồng thời còn hỗ trợ các thao tác Link/Cut.

3.6 Tiểu kết

Bài toán DP trên khối liên thông của cây thường được chia thành bài toán đếm và bài toán tối ưu.

Nếu độ phức tạp gộp hai trạng thái DP cao, còn độ phức tạp thêm một đỉnh thấp, chẳng hạn phần lớn bài toán ba lô, ta thường có thể dùng chuyển trạng thái theo thứ tự DFS thay cho chuyển trạng thái từ dưới lên thô. Nếu khối liên thông cần tìm không nhất thiết chứa gốc, có thể trả thêm $O(\log n)$ bằng phân trị theo điểm: mỗi lần lấy trọng tâm làm gốc, tính thông tin của khối liên thông bắt buộc chứa gốc, rồi đệ quy trên từng khối liên thông.

Kỹ thuật “số đỉnh trừ số cạnh” thường dùng trong bài toán đếm khi chọn nhiều khối liên thông trên cây và yêu cầu giao không rỗng. Bằng cách làm $O(n)$ lần DP bắt buộc chứa một đỉnh hoặc một cạnh,

ta có thể khiến các khối liên thông khác nhau độc lập với nhau. Đặc biệt, nếu đề yêu cầu các khối liên thông đôi một có giao, điều này cũng tương đương với việc giao của tất cả chúng không rỗng.

Với những đề mà phần lớn thời gian nằm ở việc gộp thông tin tương ứng của hai cây con, có thể xét dùng cây đoạn để lưu thông tin DP, rồi dùng gộp cây đoạn để hoàn thành nhanh các chuyển trạng thái hàng loạt.

Với bài toán cần hỗ trợ sửa thông tin đỉnh rồi đếm lại khối liên thông trên cây, có thể dùng phân rã chuỗi trên cây hoặc LCT để bảo trì DP động. Bản chất là tận dụng điều kiện phép gộp thông tin thỏa tính kết hợp. Với mỗi chuỗi nặng, ta tính tổng đáp án khi chọn một đoạn trên chuỗi, rồi dùng tổng đáp án khi chọn một tiền tố để cập nhật đáp án của chuỗi nặng cha.

4 Bài toán bảo trì khối liên thông cùng màu trên cây

4.1 Tư duy chung

Xét một cây hai màu đen-trắng. Với mỗi khối liên thông cùng màu cực đại, đỉnh nông nhất của nó, tức LCA của toàn khối liên thông, là duy nhất. Đỉnh này có thể được tìm nhanh bằng phân rã chuỗi trên cây hoặc LCT: tìm đỉnh áp chót trên đường từ một đỉnh tới tổ tiên gần nhất có màu khác. Ta có thể bảo trì thông tin tại đỉnh nông nhất đó.

Khi sửa màu của đỉnh nông nhất, hình dạng khối liên thông có thể thay đổi rất lớn: các con vốn cùng màu với nó trở thành khác màu, còn các con vốn khác màu trở thành cùng màu. Nếu trực tiếp bảo trì “thông tin của khối liên thông trong cây con của x có cùng màu với x ”, ta phải tốn $O(\text{số con})$ thời gian.

Cách giải quyết là đồng thời ghi thông tin của cả hai màu tại mỗi đỉnh: nếu đỉnh này là đen hoặc trắng, thông tin của khối liên thông màu tương ứng lấy nó làm đỉnh nông nhất là gì. Khi sửa màu của đỉnh, thông tin của bản thân nó không cần sửa; chỉ cần thêm hoặc xóa thông tin của đỉnh này tại cha nó.

Nếu số màu không phải hai mà là một hằng số nhỏ k , ta có thể bảo trì k cấu trúc. Trong cấu trúc thứ i , coi màu thứ i là đen, còn các màu khác là trắng. Như vậy ta hỗ trợ được nhiều màu với chi phí nhân thêm k .

Ví dụ 5. Nguồn: SPOJ QTREE6.

Đề bài. Cho một cây, mỗi đỉnh có một trong hai màu đen-trắng. Cần hỗ trợ hai thao tác:

1. Hỏi kích thước khối liên thông cùng màu chứa đỉnh u .
2. Lật màu đỉnh u , tức đen thành trắng và trắng thành đen.

Giới hạn $n, q \leq 10^5$.

Cách làm. Xét bảo trì hai giá trị $B(x)$ và $W(x)$ tại mỗi đỉnh, lần lượt biểu thị: nếu đỉnh x là đen hoặc trắng, kích thước khối liên thông cùng màu có màu đó và lấy x làm đỉnh nông nhất.

Với thao tác hỏi, giả sử hỏi đỉnh u và u màu trắng. Chỉ cần tìm đỉnh nông nhất v của khối liên thông trắng chứa u , rồi xuất $W(v)$.

Xét thao tác lật màu u . Giả sử đổi từ trắng sang đen. Chỉ cần tìm đỉnh đen đầu tiên v trên đường từ u đi lên, trừ $W(u)$ khỏi mọi giá trị $W()$ trên đoạn $(u, v]$; rồi tìm đỉnh trắng đầu tiên v trên đường từ u đi lên, cộng $B(u)$ vào mọi giá trị $B()$ trên đoạn $(u, v]$. Khi đó bảo trì đã hoàn tất.

“Đỉnh trắng hoặc đen đầu tiên trên đường từ u đi lên” có thể tìm bằng cấu trúc dữ liệu bảo trì số đỉnh của hai màu trên đoạn đường. Nếu dùng phân rã chuỗi trên cây cộng cây đoạn, độ phức tạp là $O(n \log^2 n)$; nếu dùng LCT, độ phức tạp là $O(n \log n)$.

Ví dụ 6. Nguồn: *SPOJ QTREE7*.

Đề bài. Cho một cây, mỗi đỉnh có một trong hai màu đen-trắng và có một trọng số. Cần hỗ trợ ba thao tác:

1. Hỏi trọng số lớn nhất trong khối liên thông cùng màu chứa đỉnh u .
2. Lật màu đỉnh u .
3. Sửa trọng số đỉnh u .

Giới hạn $n, q \leq 10^5$.

Cách làm. Khác với bài trước, bài này cần bảo trì giá trị lớn nhất, nên không thể trực tiếp trừ đóng góp như ở bài trước.

Quan sát bản chất của thông tin được bảo trì trong bài trước: nếu một đỉnh là trắng, thông tin của nó được cộng vào $W()$ của cha; nếu là đen, thông tin của nó được cộng vào $B()$ của cha.

Điều này tương đương với việc có một cây đen và một cây trắng. Đỉnh đen nối với cha nó trong cây đen, đỉnh trắng nối với cha nó trong cây trắng. Trong mỗi cây, mỗi khối liên thông ngoài đỉnh nông nhất ra đều có màu tương ứng. Khi hỏi thông tin của khối liên thông cùng màu chứa một đỉnh, ta vẫn tìm tổ tiên cùng màu nông nhất, rồi hỏi thông tin của cả cây con của đỉnh đó.

Xét dùng LCT. Tại mỗi đỉnh bảo trì một *multiset*, ghi trọng số lớn nhất trong cây con của mọi con nhẹ. Trên *splay*, đồng thời bảo trì trọng số lớn nhất của chuỗi nặng.

Khi Access chuyển cạnh nhẹ-nặng, ta cập nhật thông tin trong *multiset*: chèn giá trị của con nặng cũ và xóa giá trị của con nặng mới. Khi sửa trọng số, chỉ cần Access trước là có thể bảo trì thuận tiện.

Sửa màu một đỉnh, chẳng hạn đổi u từ trắng sang đen: nếu u không phải gốc, chỉ cần cắt cạnh giữa u và cha trong cây trắng, rồi nối cạnh giữa u và cha trong cây đen.

Mỗi lần chuyển cạnh nhẹ-nặng có một độ phức tạp $O(\text{số con nhẹ})$ của *multiset*. Theo chứng minh độ phức tạp của Top Tree, cách làm này có độ phức tạp $O(n \log n)$.

4.2 Bảo trì thông tin bất kỳ trên thứ tự DFS

Với thông tin phức tạp hơn, chỉ cần thông tin đó thỏa tính kết hợp và có thể gộp nhanh, nói cách khác có thể được cấu trúc dữ liệu bảo trì trên thứ tự DFS, thì đều có thể áp dụng cách làm phía trên.

Quan sát hình thái của cây đen và cây trắng nói trên, ta thấy chúng hoàn toàn nhất quán với cây gốc, và trong toàn bộ quá trình không cần dùng thao tác đổi gốc. Nói cách khác, có thể bảo trì trực tiếp theo thứ tự DFS của cây gốc.

Xét dùng một cây cân bằng hỗ trợ tách đoạn, như Splay hoặc Treap. Khi Link/Cut trong LCT, ta lấy đoạn cây con ra khỏi cây cân bằng rồi ghép vào thứ tự DFS của cha, hoặc tách cây con khỏi cha.

Mỗi thao tác trên cây cân bằng tốn $O(\log n)$, nên độ phức tạp tổng là $O(n \log n)$.

4.3 Đổi màu theo đường đi

Với thao tác đổi màu theo đường đi, mục này giới thiệu một cấu trúc dữ liệu dựa trên LCT do Tao Yuanzheng đề xuất.

Nếu đổi toàn bộ đỉnh trên một đường đi thành đen hoặc trắng và coi thao tác phủ màu đường đi là một thao tác LCT, ta có thể chuyển bài toán thành $O(n \log n)$ lần đảo màu một chuỗi cùng màu.

4.3.1 Định nghĩa mới của cây đen/trắng. Phỏng theo cách làm phía trên, ta vẫn bảo trì cây đen và cây trắng, nhưng cần sửa nhẹ định nghĩa của chúng. Lấy một đỉnh trắng u làm ví dụ:

1. u và cha v của nó nối với nhau trong cây trắng.
2. Nếu u và v nối với nhau bằng cạnh nặng trong cây trắng, ta nối chúng trong cây đen.

Hình trong bản gốc minh họa một hình thái khả dĩ của cây đen và cây trắng bằng hai cây trên các đỉnh $1, \dots, 8$, trong đó cạnh liền nét biểu thị cạnh nặng và cạnh đứt nét biểu thị cạnh nhẹ.

Trong định nghĩa mới, một khối liên thông bất kỳ trong cây đen có thể xem là hợp bởi hai phần:

1. Một số cây con liên thông tạo bởi các khối liên thông màu đen; gọi phần này là *cây khối*.
2. Một chuỗi tạo bởi các đỉnh trắng tương ứng với một chuỗi nặng trong cây trắng; gọi phần này là *cây chuỗi*.

Có thể thấy dưới chuỗi trắng có thể treo một số cây khối đen, nhưng dưới cây khối đen chắc chắn không có đỉnh trắng, vì trong cây đen một đỉnh trắng không thể nối với cha của nó.

Trong định nghĩa mới, khối liên thông cùng màu trong cây có màu tương ứng vẫn là một đoạn liên tiếp trên thứ tự DFS, nên có thể dùng cây cân bằng để bảo trì. Khi truy vấn, ta vẫn tìm tổ tiên cùng màu nông nhất rồi hỏi thông tin cây con. Vấn đề còn lại là bảo trì hình thái cây đen và cây trắng khi đổi màu theo đường đi.

4.3.2 Thao tác Expose. Định nghĩa thao tác Expose là một biến thể của thao tác Access. Gọi v là tổ tiên cùng màu nông nhất của u . Hiệu quả của $\text{Expose}(u)$ là: đổi đường đi giữa u và v thành các cạnh nặng, đồng thời cắt cạnh nặng phía dưới u . Khi thay đổi cạnh nhẹ-nặng, ta thực hiện thao tác Link/Cut tương ứng trong cây khác màu.

Xét việc đảo màu chuỗi từ u đến tổ tiên cùng màu nông nhất v , giả sử u là đen. Trước hết $\text{Expose}(u)$; khi đó trong cây đen, chuỗi từ u tới đỉnh đầu chuỗi đã được nối thông, nên có thể trực tiếp coi phần chuỗi này là cây chuỗi trắng. Còn trong cây trắng, phần từ chuỗi $u \sim v$ trở lên là cây khối trắng; ta trực tiếp coi phần chuỗi này là cây khối và ghép vào dưới cây khối của cha. Đồng thời cần cắt cạnh giữa v và cha trong cây đen, rồi nối một cạnh nhẹ giữa v và cha trong cây trắng.

Hình trong bản gốc xét ví dụ đổi chuỗi đen $4 \sim 6$ thành trắng. Sau $\text{Expose}(6)$, chỉ cần cắt cạnh ảo $(2, 4)$ trong cây đen, nối cạnh ảo $(2, 4)$ trong cây trắng, rồi trực tiếp đổi phần này thành trắng.

4.3.3 Sắp xếp quy trình. Giả sử cần đổi màu đường đi $u \sim v$ thành màu c . Trước hết thảo luận trường hợp tại LCA để chuyển thành một số lần đổi màu đường đi tới gốc.

Bắt đầu từ u , liên tục tìm đoạn cùng màu đang cần đảo trên đường tới gốc. Gọi đỉnh nông nhất của đoạn hiện tại là v .

Dùng thao tác $\text{Expose}(u)$ để nối thông chuỗi $u \sim v$, đồng thời bảo trì hình thái tương ứng trong cây còn lại.

Nối hoặc cắt cạnh giữa v và cha của nó, rồi đặt u bằng cha của v .

Lặp quá trình này cho tới khi u leo tới gốc.

4.3.4 Phân tích độ phức tạp. Số đoạn cùng màu trên đường đi, tức số lần Expose chuyển cạnh nhẹ-nặng, giống thao tác Access của LCT: theo nghĩa khấu hao là $O(n \log n)$. Mỗi lần chuyển cạnh nhẹ-nặng đều phải thực hiện một thao tác Link/Cut trong cây còn lại; khi Link/Cut, cần dùng cây cân bằng để bảo trì tốt thứ tự DFS trong $O(\log n)$. Vì vậy độ phức tạp tổng là $O(n \log^2 n)$.

Ví dụ 7. Nguồn: BZOJ 3914.

Đề bài. Cho một cây vô căn gồm n đỉnh. Mỗi đỉnh có hai màu, ban đầu mọi đỉnh đều màu đen. Mỗi cạnh có trọng số dương.

Cần bảo trì m thao tác:

1. 1 u : hỏi đường kính của khối liên thông cùng màu chứa u .
2. 2 $u \ v \ c$: phủ màu c lên đường đi $u \sim v$.

Giới hạn $n, m \leq 10^5$.

Cách làm. Tìm đường kính trên cây có một cách làm $O(n)$ cho một lần: tùy ý chọn một đỉnh xuất phát, chạy BFS một lần để tìm một đỉnh xa nhất u , rồi từ u chạy BFS một lần để tìm một đỉnh xa nhất v . Khi đó $u \sim v$ là một đường kính. Đáng tiếc, cách này khó bảo trì hiệu quả.

Có một cách khác bảo trì được trên thứ tự DFS: thứ tự duyệt Euler. Tức trong quá trình DFS, mỗi lần đi xuôi hoặc đi ngược qua một cạnh đều ghi lại, lần lượt viết thành ngoặc trái và ngoặc phải. Khi đó độ dài của dãy ngoặc sau khi rút gọn giữa hai đỉnh chính là khoảng cách giữa hai đỉnh đó.

Chẳng hạn, với cây ví dụ trong bản gốc có các cạnh $1 - 2$, $1 - 5$, $2 - 3$ và $2 - 4$, thứ tự DFS là

[1 [2 [3] [4]] [5]].

Khi đó $\text{dist}(2, 4)$ bằng độ dài của dãy ngoặc giữa 2 và 4 sau khi rút gọn, tức 1; còn $\text{dist}(3, 4) = 2$.

Dùng cặp (A, B) để biểu diễn dãy ngoặc sau khi rút gọn:

$\underbrace{]]] \dots]}_{A} \underbrace{[[[\dots []}_{B}$.

Đường kính của cây chính là chọn một xâu con sao cho $A + B$ sau khi rút gọn là lớn nhất.

Khi gộp hai xâu, ở giữa sẽ sinh ra $\min(B_1, A_2)$ cặp ngoặc khớp nhau. Do đó

$$(A_1, B_1) + (A_2, B_2) = (A_1 + \max(A_2 - B_1, 0), B_2 + \max(B_1 - A_2, 0)),$$

và

$$A_0 + B_0 = \max(A_1 + B_1 + B_2 - A_2, A_1 - B_1 + A_2 + B_2).$$

Dùng splay để bảo trì trên đoạn thứ tự DFS các thông tin: giá trị lớn nhất của $A + B$, giá trị lớn nhất của $A + B$ trên hậu tố, giá trị lớn nhất của $A - B$ trên hậu tố, giá trị lớn nhất của $B - A$ trên tiền tố, giá trị lớn nhất của $A + B$ trên tiền tố, cùng với A, B của cả đoạn. Các thông tin này đều có thể tính qua lại.

Với thao tác đổi màu theo đường đi, áp dụng trực tiếp LCM là được. Độ phức tạp là $O(n \log^2 n)$.

5 Lời cuối

Sau khi thảo luận với bạn Zhao Yuyang từ THPT trực thuộc Đại học Sư phạm An Huy, tác giả được biết còn có một cách làm khác: từ góc nhìn Rake và Compress, bảo trì trên Top Tree các thông tin liên quan tới đỉnh biên của *cluster* và hai màu. Cách làm này cũng giải được bài toán bảo trì khối liên thông cùng màu trên cây có đổi màu theo đường đi, và độ phức tạp đạt $O(n \log n)$. Ở đây không giới thiệu thêm; bạn đọc quan tâm có thể tự tham khảo các bài viết liên quan.

6 Triển vọng

Bài viết đã nhắc tới khá nhiều phương pháp và kỹ thuật để giải bài toán khối liên thông trên cây, nhưng trong đại dương thuật toán rộng lớn, chúng vẫn chỉ là một góc nhỏ. Hy vọng trong tương lai sẽ có thêm nhiều thuật toán liên quan tới bài toán khối liên thông trên cây được phát hiện và sáng tạo, và cũng hy vọng sẽ có thêm nhiều bài toán khối liên thông trên cây thú vị xuất hiện trong các kỳ thi tin học.

7 Lời cảm ơn

- Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu.
- Cảm ơn thầy Chen Heli và thầy Dong Yehua của Trường Trung học số 1 Thiệu Hưng đã quan tâm và hướng dẫn.
- Cảm ơn huấn luyện viên đội tuyển quốc gia Zhang Ruizhe đã hướng dẫn và giúp đỡ.
- Cảm ơn anh Chen Junkun của Đại học Thanh Hoa đã đề xuất nhiều thuật toán và kỹ thuật thực dụng cho bài toán quy hoạch động trên cây.
- Cảm ơn anh Tao Yuanzheng của Đại học Bắc Kinh và anh Ren Zhizhou của Đại học Thanh Hoa đã đóng góp vào nghiên cứu cấu trúc dữ liệu LCM.
- Cảm ơn bạn Zhao Yuyang từ THPT trực thuộc Đại học Sư phạm An Huy đã gợi mở cho tác giả về thuật toán Top Tree.
- Cảm ơn các anh Hong Huadun, Sun Yaofeng, Feng Zhe, Sun Yican, Ye Jianing của Đại học Bắc Kinh và các bạn Trường Trung học số 1 Thiệu Hưng đã giúp đỡ tác giả.
- Cảm ơn anh Sun Yican của Đại học Bắc Kinh đã đọc duyệt bài viết này.
- Cảm ơn các thầy cô và bạn học khác từng giúp đỡ, gợi mở cho tác giả.
- Cảm ơn cha mẹ đã quan tâm, ủng hộ và chăm sóc chu đáo.

8 Tài liệu tham khảo

1. Chen Junkun, *Cây tròn-vuông bình thường và quy hoạch động (động) kỳ diệu*, WC 2018.
2. Chen Junkun, *Báo cáo ra đề Đề thị con kỳ diệu và các mở rộng*, tập luận văn đội tuyển quốc gia năm 2017.
3. Tao Yuanzheng, “Mở rộng cây động”, linkcutmemphis.99293.html.
4. Ren Zhizhou, *Một thuật toán bảo trì khối liên thông cùng màu trên cây dựa trên LCT*, trao đổi học viên WC 2016.
5. Zhao Yuyang, *Bàn sơ lược về bảo trì thông tin trên cây động*.
6. Stephen Alstrup, Jacob Holm, Kristian de Lichtenberg, Mikkel Thorup, “Maintaining Information in Fully-Dynamic Trees with Top Trees”, 2003.
7. Robert E. Tarjan, Renato F. Werneck, “Self-Adjusting Top Trees”, 2005.

8 Bài 6: Báo cáo ra đề *Jellyfish* và khảo cứu mở rộng

Liang Yancheng, Trường Trung học Trần Hải, Ninh Ba

Tóm tắt

Jellyfish là một bài do tác giả ra trong đợt bài tập vòng một của đội tuyển, với tên gốc “Sửa? Sửa!”. Bài viết này trước hết giới thiệu thông tin đề bài và cách giải, sau đó dựa trên bài này để bàn một số bài toán tích chập đa thức và chuỗi lũy thừa tập hợp dưới góc nhìn phép nhân chia để trị.

1 Dẫn nhập

Trong những năm gần đây, các bài toán về đa thức xuất hiện ngày càng nhiều. Loại bài này linh hoạt, đa dạng, vừa có độ khó tư duy, vừa kiểm tra nền tảng toán học; một số bài còn kiểm nghiệm khả năng hiện thực của thí sinh. Vì vậy chúng ngày càng được người ra đề ưa chuộng.

Trong các bài toán đa thức, “tích chập” giữ một vị trí quan trọng. Khi giải bài toán tích chập, ta thường dùng tư tưởng biến đổi để tăng tốc. Các phép biến đổi quen thuộc như biến đổi Fourier nhanh (FFT), biến đổi Walsh nhanh (FWT) và biến đổi Mobius nhanh (FMT) đều là các ví dụ điển hình của cách dùng biến đổi để xử lý tích chập.

Phép nhân chia để trị là một phương pháp giải tích chập từ một góc nhìn khác. Thuật toán Karatsuba³ là phép nhân chia để trị kinh điển nhất; nó nhân hai đa thức bậc N trong thời gian xấp xỉ $O(N^{1.59})$. Vì độ phức tạp không bằng FFT nên nó không quá thường gặp, nhưng nó cung cấp một góc nhìn mới. Thực tế, khi dùng tư tưởng nhân chia để trị cho một số tích chập dưới các quy tắc phép toán bit, ta thường thu được hiệu quả khá tốt.

Mục 2 và 3 của bài viết giới thiệu bài *Jellyfish*, một bài dùng tư tưởng nhân chia để trị để giải tích chập, cùng lời giải của nó; đồng thời giới thiệu thuật toán Karatsuba và một cải tiến. Mục 4 đưa ra một số ví dụ ứng dụng của phép nhân chia để trị.

Để tiện trình bày, ta quy ước một số ký hiệu.

Ký hiệu 1.1. Với một biểu thức Boolean x ,

$$[x] = \begin{cases} 1, & x = \text{true}, \\ 0, & x = \text{false}. \end{cases}$$

Ký hiệu 1.2. Với một số thực x , $[x]$ biểu thị số nguyên lớn nhất không vượt quá x .

2 Thông tin đề bài

2.1 Tóm tắt đề

- Cho N điểm then chốt trong không gian 20 chiều. Với mỗi điểm, xem các tọa độ là một số cơ số d , rồi đổi sang thập phân và ký hiệu bằng a_i ; gọi a_i là mã vị trí của điểm đó. Giá trị tọa độ ở mỗi chiều của điểm then chốt thuộc tập $\{0, \dots, d-1\}$.
- Bây giờ chọn ngẫu nhiên một điểm then chốt làm điểm xuất phát, đi ngẫu nhiên K bước, mỗi bước đi tới một điểm kề với điểm hiện tại. Hai điểm kề nhau khi và chỉ khi chúng chỉ khác nhau đúng một chiều với độ lệch tọa độ bằng 1. Hãy tính kỳ vọng số điểm then chốt khác nhau đã đi qua.

2.2 Giới hạn dữ liệu, thời gian và bộ nhớ

$$1 \leq N \leq 200000, \quad 0 \leq K \leq 500, \quad d \in \{2, 4\}, \quad 0 \leq a_i \leq 250000.$$

³https://en.wikipedia.org/wiki/Karatsuba_algorithm

Có 25% dữ liệu thỏa $1 \leq N \leq 3000$. Ngoài ra có 15% dữ liệu thỏa $d = 2$. Ngoài ra có 20% dữ liệu thỏa $1 \leq K \leq 4$, trong đó 10% thỏa $d = 2$.

Giới hạn thời gian: 4 giây trên TUOJ.

Giới hạn bộ nhớ: 512 MB.

3 Lời giải đề bài

3.1 Phân tích ngắn

Gọi $e_{x,y}$ là xác suất xuất phát từ điểm x và đi qua điểm y trong K bước; ở đây x, y có thể trùng nhau. Khi đó đáp án cần tìm là

$$\sum_{i=1}^N \sum_{j=1}^N e_{i,j}.$$

Theo tính tuyến tính của kỳ vọng, ta có thể độc lập tính tất cả các $e_{x,y}$, rồi cộng lại để nhận đáp án.

Gọi $\text{ways}_{x,y}$ là số phương án xuất phát từ điểm x và đi qua điểm y trong K bước. Khi đó

$$e_{x,y} = \frac{\text{ways}_{x,y}}{N \times 40^N}.$$

Vì vậy ta chỉ cần tính $\text{ways}_{x,y}$ cho từng cặp x, y , cộng lại rồi chia cho $N \times 40^N$.

3.2 Lời giải vét cạn

3.2.1 Vét cạn đơn giản. Trước hết xét cách liệt kê một điểm x , rồi trực tiếp liệt kê ở từng bước trong K bước tiếp theo sẽ di chuyển theo chiều nào, chiều đó tăng 1 hay giảm 1. Khi lần đầu đi qua một điểm nào đó thì cập nhật đáp án.

Độ phức tạp thời gian là $O(N40^K)$.

3.2.2 Cải tiến 1. Chú ý độ phức tạp của thuật toán trên là $O(N40^K)$. Nếu có thể làm K nhỏ đi một chút thì thời gian sẽ cải thiện đáng kể.

Xét dùng tư tưởng tìm kiếm chia đôi. Giả sử tại bước thứ s ta đi từ x tới y . Có thể thay quá trình này bằng cách trước hết đi từ x trong $\lfloor s/2 \rfloor$ bước tới một điểm p , rồi đi ngược từ y trong $\lfloor s/2 \rfloor$ bước cũng tới điểm p .

Như vậy, trước hết ta liệt kê tổng số bước s , ghi lại mọi điểm có thể đạt được khi đi ngược $\lfloor s/2 \rfloor$ bước từ mọi y , sau đó mỗi lần liệt kê điểm xuất phát x và $\lfloor s/2 \rfloor$ bước tiếp theo để thống kê đáp án.

Cách thống kê này có thể bị đếm trùng, nhưng vì nó chỉ nhắm tới trường hợp K rất nhỏ, ta chỉ cần thảo luận đơn giản là có thể trừ phần trùng lặp.

Độ phức tạp thời gian giảm xuống $O(NK40^{K/2})$, đủ qua 20% dữ liệu.

3.2.3 Cải tiến 2. Chú ý rằng với cặp điểm (x, y) , ta không quan tâm vị trí cụ thể của x, y , mà chỉ quan tâm vị trí tương đối của chúng.

Vì vậy trước hết liệt kê $O(N^2)$ cặp điểm (x, y) , rồi xem x là gốc tọa độ và đưa vị trí tương ứng của y vào một đa tập S . Sau đó bài toán có thể xem là: xuất phát từ gốc tọa độ, với mỗi điểm trong đa tập S , tính số phương án đi K bước và đi qua điểm đó.

Nếu dùng băm để bảo trì S và truy vấn số lần xuất hiện của một điểm trong S , độ phức tạp thời gian có thể đạt $O(N^2 + 40^K)$.

3.3 Phần tiền xử lý

Cải tiến 2 của thuật toán vét cạn cho ta một hướng đi: nếu có thể nhanh chóng tiền xử lý đáp án cho mọi vị trí tương đối có thể có, thì ta có thể tính đáp án cuối cùng trong $O(N^2)$.

Nói cách khác, ta cần giải bài toán sau:

Cho một điểm then chốt, hỏi xuất phát từ gốc tọa độ $(0, 0, \dots, 0)$, đi K bước thì có bao nhiêu phương án đi qua điểm then chốt này.

Ta xét dùng quy hoạch động để tiền xử lý đáp án cho mọi trường hợp có thể. Để tiện, giả sử $d = 4$; trường hợp $d = 2$ thực ra có thể được bao hàm bởi $d = 4$.

Bổ đề 3.1. Sau khi hoán đổi hai chiều bất kỳ của một điểm then chốt, số phương án xuất phát từ gốc tọa độ và đi qua điểm then chốt đó trong K bước không đổi.

Chứng minh. Xét một điểm then chốt

$$P = (a_1, \dots, u, \dots, v, \dots, a_n)$$

và điểm tương ứng sau khi hoán đổi u, v ,

$$P' = (a_1, \dots, v, \dots, u, \dots, a_n).$$

Xây dựng một ánh xạ $A(x)$ thỏa

$$A(x) = \begin{cases} v, & x = u, \\ u, & x = v, \\ x, & x \neq u, x \neq v. \end{cases}$$

Ta dùng (t_i, d_i) để biểu thị bước thứ i đi theo chiều thứ t_i , và lượng thay đổi trên chiều đó là d_i . Nếu ta đi qua P bằng $(t_1, d_1), \dots, (t_K, d_K)$, thì có thể đi qua P' bằng $(A(t_1), d_1), \dots, (A(t_K), d_K)$. Chiều ngược lại cũng tương tự. Vì vậy các phương án đi qua P và đi qua P' tương ứng một-một, và số phương án bằng nhau. ■

Bổ đề 3.2. Sau khi lấy đối của một chiều nào đó của một điểm then chốt, số phương án xuất phát từ gốc tọa độ và đi qua điểm then chốt đó trong K bước không đổi.

Chứng minh. Giả sử lấy đối chiều thứ k . Chỉ cần định nghĩa hàm $B(x)$:

$$B(x) = \begin{cases} -1, & x = k, \\ 1, & x \neq k. \end{cases}$$

Sau đó thay (t_i, d_i) bằng $(t_i, B(t_i)d_i)$ là được; các phần còn lại dùng cùng phương pháp chứng minh như bổ đề trước. ■

Xây dựng một mảng phụ trợ b , trong đó b_i biểu thị điểm then chốt này có bao nhiêu chiều mà trị tuyệt đối của tọa độ bằng i . Theo Bổ đề 3.1 và Bổ đề 3.2, nếu hai điểm then chốt có cùng mảng b , đáp án của chúng bằng nhau.

Đồng thời, vì $b_0 + b_1 + b_2 + b_3 = 20$, ta có thể dùng dp_{b_1, b_2, b_3} để biểu thị số phương án xuất phát từ gốc tọa độ, đi K bước và đi qua một điểm then chốt có mảng

$$b = \{20 - b_1 - b_2 - b_3, b_1, b_2, b_3\}.$$

Trước khi tính mảng dp , ta cần một số chuẩn bị.

- Trước hết, cần một mảng $f_{i,j,k}$, biểu thị số phương án đi từ gốc tọa độ trong không gian j chiều, đi k bước và tới $(i, i, \dots, i)$.
- Khi đó có thể liệt kê trên chiều thứ j đã đi x lần theo hướng -1 và $x + i$ lần theo hướng $+1$, nhận được chuyển tiếp

$$f_{i,j,k} = \sum_x \binom{k}{x+x+i} \binom{x+x+i}{x} f_{i,j-1,k-x-(x+i)}.$$

Tiền xử lý tất cả các giá trị f mất $O(pdK^2)$, trong đó p là số chiều của không gian; ở bài này $p = 20$. Bây giờ xét một mảng b và cần tính mảng dp tương ứng.

- Trước hết tính một mảng h , trong đó $h_{i,j}$ biểu thị trong $b_0 + \dots + b_i$ chiều, số phương án xuất phát từ gốc tọa độ, đi j bước và tới điểm

$$(0, \dots, 0 \text{ (} b_0 \text{ số } 0), 1, \dots, 1 \text{ (} b_1 \text{ số } 1), \dots, i, \dots, i \text{ (} b_i \text{ số } i)).$$

- Liệt kê k là số bước đi trên các chiều ứng với b_i , ta có

$$h_{i,j} = \sum_k \binom{j}{k} h_{i-1,j-k} f_{i,b_i,k}.$$

- Gọi g_i là số phương án đi i bước và vừa đúng tới điểm then chốt này, tức lần đầu tới điểm then chốt là ở bước thứ i . Dùng bao hàm-loại trừ: lấy tổng số phương án $h_{d,i}$ trừ các phần không hợp lệ.
- Với phần không hợp lệ, chắc chắn trước đó đã đi qua điểm then chốt. Liệt kê lần đầu trước đó đi tới điểm then chốt là bước thứ j , nhận được

$$g_i = h_{d,i} - \sum_{j < i} g_j f_{0,20,i-j}.$$

- Khi đó

$$dp_{b_1,b_2,b_3} = \sum_i g_i \cdot 40^{K-i}.$$

Toàn bộ phần tiền xử lý có độ phức tạp thời gian $O(p^3 d K^2)$, nhưng hằng số rất nhỏ nên có thể tính trong thời gian ngắn.

Nếu sau khi tiền xử lý, ta liệt kê $O(N^2)$ cặp điểm và dùng mảng dp để tính đáp án, có thể qua 25% dữ liệu.

3.4 Phân tích thêm

Khi đã có mảng dp , ta chỉ cần tính một mảng lưu chênh lệch khoảng cách giữa mọi cặp trong N điểm, cộng số phương án của tất cả các cặp, rồi chia cho $N \times 40^N$.

Định nghĩa 3.1. Định nghĩa toán tử $\oplus$ là hàm chênh lệch khoảng cách của hai mã vị trí trong cơ số d . Cụ thể:

$$\overline{(a_1 b_1 c_1 \dots)_d} \oplus \overline{(a_2 b_2 c_2 \dots)_d} = \overline{(|a_1 - a_2| |b_1 - b_2| |c_1 - c_2| \dots)_d}.$$

Viết hình thức:

$$x \oplus y = \sum_{k=0}^{\infty} \left(\left\lfloor \frac{x}{d^k} \right\rfloor - \left\lfloor \frac{y}{d^k} \right\rfloor \right) \bmod d \cdot d^k.$$

Định nghĩa 3.2. Với hai dãy f, g có độ dài N , định nghĩa $f \oplus g = a$, trong đó dãy a thỏa

$$a_i = \sum_{j=0}^{N-1} \sum_{k=0}^{N-1} [j \oplus k = i] f_j g_k.$$

Có thể thấy, với mảng a ban đầu, nếu ta xây dựng mảng phụ trợ a' sao cho a'_i biểu thị trong mảng a có bao nhiêu mã vị trí bằng i , rồi tính $a' \oplus a'$, ta nhận được đúng “mảng chênh lệch khoảng cách” cần tìm.

Khi $d = 2$, $\oplus$ tương đương với xor, nên có thể dùng biến đổi Walsh nhanh để qua phần dữ liệu $d = 2$ trong giới hạn thời gian.

Khi $d = 4$, ta có thể xét mở rộng cơ số d thành cơ số $2d - 1$, để mỗi chiều có phạm vi $[-(d - 1), d - 1]$. Như vậy ta có thể bỏ trị tuyệt đối trong hàm chênh lệch khoảng cách, và trực tiếp đổi hàm chênh lệch khoảng cách thành phép trừ hai số.

Sau khi mở rộng cơ số d thành $2d - 1$, độ dài dãy mới mở rộng thành

$$M = (\max a_i)^{\log_d(2d-1)}.$$

Dùng biến đổi Fourier nhanh để thu được mảng chênh lệch khoảng cách có độ phức tạp thời gian

$$O(M \log M) = O((\max a_i)^{\log_d(2d-1)} \log \max a_i) \approx O((\max a_i)^{1.40} \log \max a_i).$$

Cách này khá khó qua bài trong giới hạn thời gian đã cho, nên ta xét dùng phép nhân chia để trị.

3.5 Bài toán nhân đa thức

Xét bài toán nhân đa thức kinh điển: cho hai dãy a, b có độ dài N , hãy tìm dãy c thỏa

$$c_i = \sum_{j=0}^i a_j b_{i-j}.$$

Đặt

$$A(x) = \sum a_k x^k, \quad B(x) = \sum b_k x^k, \quad C(x) = \sum c_k x^k.$$

Khi đó bài toán tương đương với tìm hệ số của đa thức $C(x)$ nhận được từ tích hai đa thức $A(x), B(x)$.

3.5.1 Thuật toán Karatsuba. Với bài toán nhân đa thức, thông thường ta dùng biến đổi Fourier nhanh để đổi a, b sang dạng giá trị tại điểm, lấy được giá trị tại điểm của c , rồi nội suy để thu được dãy c .

Thuật toán Karatsuba cung cấp một hướng khác: dùng chia để trị để giải nhân đa thức. Quy trình cơ bản như sau; để tiện, giả sử N là lũy thừa của 2.

- Tách $A(x)$ thành hai đa thức $A_0(x), A_1(x)$ thỏa $A(x) = A_0(x) + A_1(x)x^{N/2}$.
- Tương tự, tách $B(x)$ thành hai đa thức $B_0(x), B_1(x)$ thỏa $B(x) = B_0(x) + B_1(x)x^{N/2}$.
- Khi đó

$$C(x) = A(x)B(x) = A_0(x)B_0(x) + (A_0(x)B_1(x) + A_1(x)B_0(x))x^{N/2} + A_1(x)B_1(x)x^N.$$

- Tiếp theo cần tính nhanh hạng $A_0(x)B_1(x) + A_1(x)B_0(x)$.
- Chú ý tổng của ba hạng khá dễ tính:

$$A_0B_0 + A_0B_1 + A_1B_0 + A_1B_1 = (A_0 + A_1)(B_0 + B_1).$$

- Vì vậy chỉ cần tính $(A_0(x) + A_1(x))(B_0(x) + B_1(x))$, ta nhận được

$$A_0B_1 + A_1B_0 = (A_0 + A_1)(B_0 + B_1) - A_0B_0 - A_1B_1.$$

- Do đó chỉ cần giải ba bài toán con có kích thước bằng một nửa ban đầu: $A_0(x)B_0(x)$, $A_1(x)B_1(x)$, và $(A_0(x) + A_1(x))(B_0(x) + B_1(x))$.
- Từ đó có thể khôi phục $C(x)$.

Phân tích đơn giản độ phức tạp thời gian cho ta

$$T(N) = 3T\left(\frac{N}{2}\right) + O(N) = O(N^{\log_2 3}) \approx O(N^{1.59}).$$

Với $N = 100000$, bậc tối ưu $O2$, thuật toán này chạy trên UOJ khoảng 0.7 giây. Nếu xét nhân đa thức theo mô-đun, thời gian chạy xấp xỉ 1.5 ~ 2 lần thuật toán NTT ba mô-đun.⁴

3.5.2 Cải tiến thuật toán Karatsuba. Khi giải nhân đa thức, thuật toán Karatsuba tách đa thức thành hai phần để tăng tốc độ tính.

Vậy nếu ta tách đa thức thành $\gamma + 1$ phần, trong đó γ là hằng số, liệu có nhận được độ phức tạp thời gian tốt hơn không?

Xét hai đa thức bậc $(\gamma + 1)n - 1$ có hệ số lần lượt là $a_0, \dots, a_{(\gamma+1)n-1}$ và $b_0, \dots, b_{(\gamma+1)n-1}$. Ta cắt a, b thành $\gamma + 1$ đoạn độ dài n , ký hiệu $A_0, \dots, A_\gamma$ và $B_0, \dots, B_\gamma$. Đặt $p = x^n$, và

$$A(p) = \sum_{k=0}^{\gamma} A_k p^k, \quad B(p) = \sum_{k=0}^{\gamma} B_k p^k.$$

Khi đó

$$C(p) = A(p)B(p), \quad C_i = \sum_{j=0}^i A_j B_{i-j}.$$

Vì

$$C(p) = \sum_{k=0}^{2\gamma} C_k p^k$$

là một đa thức bậc 2γ theo p , ta xét lấy $2\gamma + 1$ giá trị tại điểm rồi khôi phục $C(p)$.

Khi lấy $p = p_0$ và thế vào, ta nhận được một dãy mới a' có độ dài n , trong đó

$$a'_i = \sum_{k=0}^{\gamma} a_{i+kn} p_0^k.$$

Tương tự thu được b' . Khi đó cần nhân hai đa thức có hệ số là a' và b' , nên kích thước bài toán giảm còn $\frac{1}{\gamma+1}$ ban đầu.

Ngoài phần đệ quy, mọi phần còn lại đều có thể hoàn thành trong $O(N)$. Do đó độ phức tạp là

$$T(N) = (2\gamma + 1)T\left(\frac{N}{\gamma + 1}\right) + O(N) = O\left(N^{\log_{\gamma+1}(2\gamma+1)}\right).$$

Từ đó ta chứng minh được định lý sau.⁵

⁴Thuật toán nhanh về mặt này có thể xem trong luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2015 của Mao Xiao, *Xét lại biến đổi Fourier nhanh*.

⁵The Art of Computer Programming, tập 2, Mục 4.3.3, Định lý A.

Định lý 3.1. Với mọi $\varepsilon > 0$, tồn tại một hằng số $c(\varepsilon)$ không phụ thuộc vào N và một thuật toán nhân sao cho

$$T(N) < c(\varepsilon)N^{1+\varepsilon}.$$

Thực tế, thuật toán Karatsuba có thể xem là trường hợp đặc biệt $\gamma = 1$.

3.6 Lời giải cuối cùng

Quay lại bài toán ban đầu. Ta thử áp dụng tư tưởng của thuật toán Karatsuba vào bài này. Xét cách tính $a = f \oplus g$, trong đó f, g có độ dài N .

Trong thuật toán Karatsuba, ta tăng tốc bằng cách tách dãy thành hai phần. Vậy trong cơ sở d , ta xét tách dãy theo chữ số cao nhất ở cơ sở d . Dùng a_i để biểu thị phần của dãy a có chữ số cao nhất trong cơ sở d bằng i . Tách riêng a, f, g , ta thấy với trường hợp $d = 4$:

- $a_0 = f_0 \oplus g_0 + f_1 \oplus g_1 + f_2 \oplus g_2 + f_3 \oplus g_3$.
- $a_1 = f_0 \oplus g_1 + f_1 \oplus g_0 + f_1 \oplus g_2 + f_2 \oplus g_1 + f_2 \oplus g_3 + f_3 \oplus g_2$.
- $a_2 = f_0 \oplus g_2 + f_2 \oplus g_0 + f_1 \oplus g_3 + f_3 \oplus g_1$.
- $a_3 = f_0 \oplus g_3 + f_3 \oplus g_0$.

Để đơn giản hóa các công thức trên, trước hết tách các hạng lẻ và chẵn:

$$x' = (f_0 + f_1 + f_2 + f_3) \oplus (g_0 + g_1 + g_2 + g_3),$$

$$y' = (f_0 - f_1 + f_2 - f_3) \oplus (g_0 - g_1 + g_2 - g_3).$$

Sau đó đặt $x = (x' + y')/2$, $y = (x' - y')/2$. Khi đó

$$x = \sum_{i=0}^3 \sum_{j=0}^3 [i \equiv j \pmod{2}] f_i \oplus g_j, \quad y = \sum_{i=0}^3 \sum_{j=0}^3 [i \not\equiv j \pmod{2}] f_i \oplus g_j.$$

Nói cách khác, $x = a_0 + a_2$, $y = a_1 + a_3$.

Bây giờ xét trực tiếp tính a_3 . Từ

$$u' = (f_0 + f_3) \oplus (g_0 + g_3), \quad v' = (f_0 - f_3) \oplus (g_0 - g_3),$$

đặt $u = (u' + v')/2$, $v = (u' - v')/2$, ta có

$$u = f_0 \oplus g_0 + f_3 \oplus g_3, \quad v = f_0 \oplus g_3 + f_3 \oplus g_0.$$

Do đó $a_3 = v$, $a_1 = y - a_3$.

Bây giờ ta chỉ cần tính một trong hai giá trị a_0, a_2 . Quan sát u đã tính ở trên, ta có thể xét tính $f_1 \oplus g_1$ và $f_2 \oplus g_2$. Khi đó

$$a_0 = u + f_1 \oplus g_1 + f_2 \oplus g_2,$$

và $a_2 = x - a_0$.

Vì vậy ta tách một bài toán kích thước N thành 6 bài toán con kích thước $N/4$ để tính đệ quy, nhận được độ phức tạp thời gian

$$T(N) = 6T\left(\frac{N}{4}\right) + O(N) = O(N^{\log_4 6}) \approx O(N^{1.29}).$$

Trong bài toán gốc, $N = \max a_i$, nên có thể ước lượng độ phức tạp là

$$O((\max a_i)^{1.29}).$$

Tóm lại, ta đã giải trọn vẹn bài toán.

4 Phép nhân chia để trị và tích chập bit

Qua bài *Jellyfish*, có thể thấy khi áp dụng tư tưởng nhân chia để trị vào tích chập bit, ta thường nhận được hiệu quả tốt.

Tiếp theo tác giả dùng một số ví dụ để trình bày thêm các ứng dụng của phép nhân chia để trị trong tích chập bit.⁶

4.1 Tích chập dưới quy tắc and, or

Định nghĩa 4.1. Cho hai dãy a, b có chỉ số là các số nhị phân n bit, định nghĩa $a \cdot b = c$, trong đó dãy c thỏa

$$c_i = \sum_{j=0}^{2^n-1} \sum_{k=0}^{2^n-1} [j \text{ and } k = i] a_j b_k.$$

Ta gọi c là tích chập của hai dãy a, b dưới quy tắc and.

Bắt chước thuật toán Karatsuba, tách các dãy a, b theo bit cao nhất thành hai dãy a_0, a_1 và b_0, b_1 , đồng thời tách c thành c_0, c_1 . Khi đó:

- $c_0 = a_0 \cdot b_0 + a_0 \cdot b_1 + a_1 \cdot b_0$.
- $c_1 = a_1 \cdot b_1$.
- Từ $c_0 + c_1 = (a_0 + a_1) \cdot (b_0 + b_1)$, suy ra

$$c_0 = (a_0 + a_1) \cdot (b_0 + b_1) - c_1.$$

Như vậy chỉ cần tính đệ quy $(a_0 + a_1) \cdot (b_0 + b_1)$ và $a_1 \cdot b_1$, độ phức tạp thời gian là

$$T(n) = 2T(n-1) + O(2^n) = O(n2^n).$$

Tích chập dưới quy tắc or về bản chất giống and. Chỉ cần hoán đổi 0, 1 trong biểu diễn nhị phân của chỉ số trên cơ sở lời giải and là thu được lời giải cho quy tắc or.

4.2 Tích chập dưới quy tắc xor nhị phân

Định nghĩa 4.2. Cho hai dãy a, b có chỉ số là các số nhị phân n bit, định nghĩa $a \cdot b = c$, trong đó dãy c thỏa

$$c_i = \sum_{j=0}^{2^n-1} \sum_{k=0}^{2^n-1} [j \text{ xor } k = i] a_j b_k.$$

Ta gọi c là tích chập của hai dãy a, b dưới quy tắc xor.

Tương tự, tách ba dãy. Khi đó:

- $c_0 = a_0 \cdot b_0 + a_1 \cdot b_1$.
- $c_1 = a_0 \cdot b_1 + a_1 \cdot b_0$.
- Từ $c_0 + c_1 = (a_0 + a_1) \cdot (b_0 + b_1)$, suy ra

$$c_1 = (a_0 + a_1) \cdot (b_0 + b_1) - c_0.$$

⁶Nội dung Mục 4.1 và 4.2 cũng được thảo luận trong luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2015 của Lu Kaifeng, *Tính chất, ứng dụng và thuật toán nhanh của chuỗi lũy thừa tập hợp*.

Nếu làm như vậy, cần tính đệ quy $a_0 \cdot b_0$, $a_1 \cdot b_1$, và $(a_0 + a_1) \cdot (b_0 + b_1)$, nên độ phức tạp thời gian là

$$T(n) = 3T(n-1) + O(2^n) = O(3^n).$$

Tất nhiên, có thể xây dựng các đa thức để đệ quy tính, từ đó nhận lời giải tốt hơn:

- $x = (a_0 + a_1) \cdot (b_0 + b_1)$.
- $y = (a_0 - a_1) \cdot (b_0 - b_1)$.
- Khi đó $c_0 = (x + y)/2$, $c_1 = (x - y)/2$.

Như vậy chỉ cần tính đệ quy $(a_0 + a_1) \cdot (b_0 + b_1)$ và $(a_0 - a_1) \cdot (b_0 - b_1)$, độ phức tạp thời gian là

$$T(n) = 2T(n-1) + O(2^n) = O(n2^n).$$

4.3 Tích chập dưới quy tắc xor cơ số K

Định nghĩa 4.3. Định nghĩa phép xor trong cơ số K , ký hiệu xor_K , là kết quả cộng từng chữ số trong cơ số K mà không nhớ. Viết hình thức:

$$x \text{ xor}_K y = \sum_{i=0}^{\infty} \left(\left\lfloor \frac{x}{K^i} \right\rfloor + \left\lfloor \frac{y}{K^i} \right\rfloor \bmod K \right) K^i.$$

Định nghĩa 4.4. Cho hai dãy a, b có độ dài N , định nghĩa $a \cdot b = c$, trong đó dãy c thỏa

$$c_i = \sum_{j=0}^{N-1} \sum_{k=0}^{N-1} [j \text{ xor}_K k = i] a_j b_k.$$

Ta gọi c là tích chập của hai dãy a, b dưới quy tắc xor cơ số K .

4.3.1 Cách giải. Thông thường, ta dùng biến đổi Fourier nhanh nhiều chiều, mỗi chiều có kích thước K , để giải bài toán này. Tương tự, ta cũng có thể dùng phép nhân chia để trị để giải bài toán nhân đa thức nhiều chiều với mỗi chiều có kích thước K .

Tách dãy a, b theo chữ số đầu tiên trong cơ số K thành K dãy $(a_0, a_1, \dots, a_{K-1})$ và $(b_0, b_1, \dots, b_{K-1})$; tách dãy c tương tự. Khi đó

$$c_i = \sum_{j=0}^{K-1} \sum_{k=0}^{K-1} [j + k \equiv i \pmod{K}] a_j \cdot b_k.$$

Xây dựng $2K - 1$ dãy $d_0, d_1, \dots, d_{2K-2}$ thỏa

$$d_i = \sum_{j=0}^{K-1} \sum_{k=0}^{K-1} [j + k = i] a_j \cdot b_k.$$

Khi đó

$$c_i = \sum_j [j \equiv i \pmod{K}] d_j.$$

Vì vậy ta chuyển sang tìm dãy d . Xây dựng các đa thức

$$A(x) = \sum a_k x^k, \quad B(x) = \sum b_k x^k, \quad D(x) = \sum d_k x^k.$$

Khi đó chỉ cần tính $D(x) = A(x)B(x)$. Có thể dùng thuật toán Karatsuba để biến nó thành $t(K)K^{1.59}$ bài toán con và giải đệ quy, trong đó $t(K)$ là hệ số hằng.

Giả sử N là lũy thừa của K , độ phức tạp thời gian có thể ước lượng là

$$T(N) \approx t(K)K^{1.59}T\left(\frac{N}{K}\right) + t(K)K^{1.59}O\left(\frac{N}{K}\right) = O\left(N^{\log_K(t(K)K^{1.59})}\right) = O\left(N^{1.59+\log_K t(K)}\right).$$

Khi K tương đối lớn, $\log_K t(K)$ có thể bỏ qua; độ phức tạp xấp xỉ $O(N^{1.59})$.⁷

4.3.2 Một tối ưu. Vì thực ra ta chỉ cần tìm dãy c , không cần tìm toàn bộ dãy d , nên khi K là số chẵn ta có thể dùng một số tối ưu.

Chú ý trong lần tách đầu tiên của thuật toán Karatsuba, ta làm như sau:

$$D(x) = A_0(x)B_0(x) + (A_0(x)B_1(x) + A_1(x)B_0(x))x^{K/2} + A_1(x)B_1(x)x^K.$$

Xét các d có chỉ số đồng dư theo mô-đun K ; chúng được cộng vào cùng một c_i . Vì vậy ta có thể lấy $D(x)$ theo mô-đun x^K , tức là

$$D(x) = A_0(x)B_0(x) + A_1(x)B_1(x) + (A_0(x)B_1(x) + A_1(x)B_0(x))x^{K/2}.$$

Xây dựng hai đa thức $u(x), v(x)$ thỏa

$$u(x) = (A_0(x) + A_1(x))(B_0(x) + B_1(x)),$$

$$v(x) = (A_0(x) - A_1(x))(B_0(x) - B_1(x)).$$

Khi đó

$$D(x) = \frac{u(x) + v(x)}{2} + \frac{u(x) - v(x)}{2}x^{K/2}.$$

Như vậy ta đã giảm $t(K)$ xuống còn $2/3$ ban đầu; khi K nhỏ, hiệu quả khá rõ.

4.4 Tích chập dưới quy tắc bit phức tạp hơn

Ngay cả với các quy tắc bit phức tạp hơn, phép nhân chia để trị cũng thường cho ta một thuật toán khá tốt.

Xét ví dụ sau.

Định nghĩa 4.5. Định nghĩa toán tử $x * y$ là kết quả lấy lũy thừa từng chữ số của hai số thập phân x, y mà không nhớ. Viết hình thức:

$$x * y = \sum_{k=0}^{\infty} \left(\left\lfloor \frac{x}{10^k} \right\rfloor \left\lfloor \frac{y}{10^k} \right\rfloor \bmod 10 \right) 10^k.$$

Đặc biệt, đặt $0^0 = 0$.

Ta định nghĩa tích chập dưới quy tắc $*$ như sau.

Định nghĩa 4.6. Cho hai dãy a, b có độ dài N , định nghĩa $a \cdot b = c$, trong đó dãy c thỏa

$$c_i = \sum_{j=0}^{N-1} \sum_{k=0}^{N-1} [j * k = i] a_j b_k.$$

Ta gọi c là tích chập của a, b dưới quy tắc $*$.

⁷Ở đây cũng có thể dùng phương pháp nhắc tới trong Mục 3.5.2 để cải thiện độ phức tạp thời gian.

4.4.1 Cách giải. Có thể thấy toán tử $*$ hầu như không có tính chất tốt nào: giao hoán, kết hợp hay phân phối đều không thỏa.

Tuy nhiên ta vẫn có thể dùng tư tưởng nhân chia để trị để giảm độ phức tạp xuống dưới $O(N^2)$.

Xét hai số $x, y \in [0, 10)$. Ta có thể gộp một số cặp x, y có cùng giá trị $x^y \bmod 10$. Chẳng hạn khi $x = 0, 1, 5, 6$, với mọi số nguyên dương y , giá trị x^y đều không đổi. Dựa trên tư tưởng “gộp”, ta có thể thử giảm độ phức tạp thời gian.

Trước hết vẫn tách các dãy a, b, c theo chữ số cao nhất trong cơ số 10, mỗi dãy thành 10 dãy.

Để tiện mô tả, ta quy ước một số ký hiệu.

Ký hiệu 4.1. Đặt $U = \{0, 1, \dots, 9\}$, $V = \{1, 2, \dots, 9\}$.

Ký hiệu 4.2. Với hai tập $S, T \subseteq U$, ký hiệu

$$g_{S,T} = \sum_{i \in S} \sum_{j \in T} a_i \cdot b_j.$$

Có thể thấy

$$g_{S,T} = \left(\sum_{i \in S} a_i \right) \cdot \left(\sum_{j \in T} b_j \right).$$

Nghĩa là với S, T đã cho, chỉ cần làm đệ quy một bài toán con có kích thước bằng $1/10$ bài toán ban đầu để tính $g_{S,T}$.

Ký hiệu 4.3. Với $x, y \in V$, đặt

$$F(x, y) = \{i \mid i \in V, x^i \equiv y \pmod{10}\},$$

$$h_{x,y} = \sum_{i \in F(x,y)} a_x \cdot b_i.$$

Hiển nhiên $h_{x,y} = g_{\{x\}, F(x,y)}$, nên chỉ cần một lần đệ quy để tính $h_{x,y}$.

Chú ý rằng với một x cố định, không có quá nhiều y thỏa $F(x, y) \neq \emptyset$. Xét lần lượt từng $x \in V$:

- Khi $x = 1$, chỉ cần tính $h_{x,1}$.
- Khi $x = 2$ hoặc $x = 8$, chỉ cần tính $h_{x,2}, h_{x,4}, h_{x,6}, h_{x,8}$.
- Khi $x = 3$ hoặc $x = 7$, chỉ cần tính $h_{x,1}, h_{x,3}, h_{x,7}, h_{x,9}$.
- Khi $x = 4$, chỉ cần tính $h_{x,4}, h_{x,6}$.
- Khi $x = 5$, chỉ cần tính $h_{x,5}$.
- Khi $x = 6$, chỉ cần tính $h_{x,6}$.
- Khi $x = 9$, chỉ cần tính $h_{x,1}, h_{x,9}$.

Có thể thấy chỉ có 23 cặp có thứ tự $x, y \in V$ thỏa $F(x, y) \neq \emptyset$. Ta trực tiếp đệ quy vét cạn 23 bài toán con để tính các $h_{x,y}$ ở trên.

Bây giờ giả sử đã có $d_1, \dots, d_9$ thỏa

$$d_k = \sum_{x \in V} h_{x,k}.$$

Tiếp theo xét trường hợp $x = 0$ hoặc $y = 0$:

- Khi $x = 0$, $x^y \equiv 0 \pmod{10}$.

- Khi $x \neq 0$ và $y = 0$, $x^y \equiv 1 \pmod{10}$.

Thêm hai trường hợp trên vào dãy d , ta nhận được mảng c cuối cùng:

- $c_0 = g_{\{0\}, U}$.
- $c_1 = d_1 + g_{V, \{0\}}$.
- Với các c_k khác, $c_k = d_k$.

Như vậy chỉ cần đệ quy 25 bài toán con kích thước $1/10$ bài toán ban đầu là có thể thu được dãy c . Độ phức tạp thời gian là

$$T(N) = 25T\left(\frac{N}{10}\right) + O(N) = O(N^{\log_{10} 25}) \approx O(N^{1.40}).$$

4.4.2 Một số tối ưu đơn giản. Trong cách làm trên, ta tính $h_{x,y}$ đơn giản theo x , và thu được một thuật toán.

Thực ra khi tìm dãy d , ta cộng các $h_{x,y}$ theo y . Vì vậy một số $h_{x,y}$ có thể được gộp, chẳng hạn:

- $h_{2,4} + h_{8,4} = g_{\{2,8\}, \{2,6\}}$.
- $h_{2,6} + h_{8,6} = g_{\{2,8\}, \{4,8\}}$.
- $h_{3,9} + h_{7,9} = g_{\{3,7\}, \{2,6\}}$.
- $h_{3,1} + h_{7,1} = g_{\{3,7\}, \{4,8\}}$.

Như vậy giảm được 4 lần đệ quy, độ phức tạp thời gian trở thành

$$T(N) = 21T\left(\frac{N}{10}\right) + O(N) = O(N^{\log_{10} 21}) \approx O(N^{1.32}).$$

Ngoài ra, ta thấy

- $h_{2,2} + h_{8,2} + h_{2,8} + h_{8,8} = g_{\{2,8\}, \{1,3,5,7,9\}}$.
- $h_{2,2} + h_{8,2} - h_{2,8} - h_{8,8} = (a_2 - a_8) \cdot (b_1 - b_3 + b_5 - b_7 + b_9)$.

Vì vậy chỉ cần hai lần đệ quy là lần lượt tính được $h_{2,2} + h_{8,2}$ và $h_{2,8} + h_{8,8}$, giảm thêm hai lần đệ quy. Độ phức tạp thời gian tiếp tục giảm thành

$$T(N) = 19T\left(\frac{N}{10}\right) + O(N) = O(N^{\log_{10} 19}) \approx O(N^{1.28}).$$

4.5 Hàm có nhiều dãy làm biến

Tích chập thông thường lấy hai dãy làm biến. Nếu có ba dãy hoặc nhiều hơn làm biến, rất khó tăng tốc bằng một kiểu biến đổi nào đó. Nhưng dùng phép nhân chia để trị, ta vẫn có thể tìm ra một thuật toán tốt hơn vết cặn trực tiếp.

Xét ví dụ sau.

Định nghĩa 4.7. Cho một hàm $f(x, y, z)$ có miền xác định $\{(x, y, z) \mid x, y, z \in \{0, 1\}\}$, miền giá trị $\{0, 1\}$, và thỏa $f(0, 0, 0) = 0$.

Định nghĩa hàm $g(x, y, z)$ là kết quả áp dụng f theo từng bit của x, y, z . Viết hình thức:

$$g(x, y, z) = \sum_{k=0}^{\infty} 2^k f(x_k, y_k, z_k),$$

trong đó x_i biểu thị bit thứ i , tính từ thấp lên cao, trong biểu diễn nhị phân của x .

Định nghĩa 4.8. Cho ba dãy a, b, c có độ dài N , định nghĩa hàm $H(a, b, c)$. Đặt $d = H(a, b, c)$, khi đó

$$d_i = \sum_{j=0}^{N-1} \sum_{k=0}^{N-1} \sum_{l=0}^{N-1} [g(j, k, l) = i] a_j b_k c_l.$$

Đặt

$$S = \{(x, y, z) \mid f(x, y, z) = 0\}, \quad T = \{(x, y, z) \mid f(x, y, z) = 1\}.$$

Sau khi tách các dãy a, b, c, d theo bit cao nhất, hiển nhiên có

$$d_0 = \sum_{x, y, z, (x, y, z) \in S} H(a_x, b_y, c_z),$$

$$d_1 = \sum_{x, y, z, (x, y, z) \in T} H(a_x, b_y, c_z).$$

Ta biết $S \cap T = \emptyset$ và $|S| + |T| = 8$, nên chắc chắn $\min(|S|, |T|) \leq 4$. Vì vậy ta quyết định tính vết cận đệ quy d_0 hay d_1 theo kích thước của $|S|$ và $|T|$, rồi dùng

$$d_0 + d_1 = H(a_0 + a_1, b_0 + b_1, c_0 + c_1)$$

để thu được hạng còn lại.

Vì $\min(|S|, |T|) \leq 4$, nhiều nhất cần tính đệ quy $4 + 1 = 5$ lần. Độ phức tạp thời gian là

$$T(N) = 5T\left(\frac{N}{2}\right) + O(N) = O(N^{\log_2 5}) \approx O(N^{2.32}).$$

5 Tổng kết

Jellyfish là một bài toán khá tổng hợp, kết hợp toán học, quy hoạch động và đa thức; bài có độ khó tư duy và độ khó hiện thực nhất định. Thí sinh cần dùng kiến thức về kỳ vọng để đơn giản hóa bài toán, rồi dùng quy hoạch động và phép nhân chia để trị để tối ưu thuật toán.

Tác giả lần đầu tiếp xúc với tư tưởng dùng phép nhân chia để trị để giải một số bài tích chập dưới các quy tắc bit đặc biệt trong kỳ thi mô phỏng nội bộ của đàn anh Huang Zhihuan. Sau đó tác giả nghiên cứu thêm loại bài này và nhận thấy phép nhân chia để trị thường đạt hiệu quả tốt. Vì vậy tác giả chọn *Jellyfish* làm đề tự chọn trong đợt bài tập vòng một của đội tuyển để trao đổi, thảo luận với mọi người, hy vọng bài viết có thể gợi mở phần nào.

6 Lời cảm ơn

Cảm ơn CCF đã cung cấp nền tảng học tập và giao lưu.

Cảm ơn cha mẹ đã nuôi dưỡng và giáo dục.

Cảm ơn nhà trường đã bồi dưỡng, thầy Ying Ping'an đã chỉ dạy, và các bạn học đã giúp đỡ.

Cảm ơn đàn anh Huang Zhihuan của Đại học Bắc Kinh đã hướng dẫn chúng tôi.

Cảm ơn Ren Xuandi của Trường Trung học số 1 Thiệu Hưng và Liu Chengao của THPT trực thuộc Đại học Công nghiệp Tây Bắc đã trao đổi, thảo luận, gợi mở cho tác giả, đồng thời đọc duyệt bài viết này.

Tài liệu tham khảo

1. Tư liệu về thuật toán Karatsuba trên Wikipedia tiếng Anh: https://en.wikipedia.org/wiki/Karatsuba_algorithm.

2. Lu Kaifeng, *Tính chất, ứng dụng và thuật toán nhanh của chuỗi lũy thừa tập hợp*, luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2015.
3. Mao Xiao, *Xét lại biến đổi Fourier nhanh*, luận văn ứng viên đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2015.
4. Donald E. Knuth, *The Art of Computer Programming*, Nhà xuất bản Công nghiệp Quốc phòng.

9 *Leafy Tree* và cây cân bằng có trọng số được hiện thực bằng nó

Wang Siqi, Trường Trung học số 7 Thành Đô

Tóm tắt

Cây nhị phân và cây cân bằng là các cấu trúc dữ liệu quan trọng trong thi tin học. Chúng có công dụng rất lớn khi cần duy trì tập hợp hoặc dãy.

Bài viết này giới thiệu *Leafy Tree*, một cấu trúc dữ liệu có thể hiện thực phần lớn các cấu trúc dữ liệu dựa trên cây nhị phân. Bài viết mô tả cách dùng nó để hiện thực cây tìm kiếm nhị phân và cây cân bằng có trọng số, đồng thời so sánh hai cấu trúc dữ liệu này với các cấu trúc dữ liệu khác.

1 Lời nói đầu

Trong những năm gần đây, các bài toán cấu trúc dữ liệu trong thi tin học dần được khai thác nhiều hơn. Rất nhiều bài cần dùng cây nhị phân, chẳng hạn cây đoạn, cây cân bằng, cây lệch trái, v.v. để duy trì thông tin; trong đó cây cân bằng là một nội dung khảo sát quan trọng.

Các cây cân bằng thường gặp gồm Splay, Treap, v.v. Chúng ít nhiều đều có một số vấn đề, như hằng số lớn, lượng mã lớn, khó khả tri hóa. Để xử lý các vấn đề này, bài viết đưa vào một cấu trúc cây nhị phân mới, *Leafy Tree*, và một chiến lược cân bằng mới, cân bằng có trọng số. Khi kết hợp hai thứ này, phần lớn vấn đề của cây cân bằng có thể được khắc phục.

Mục 2 giới thiệu *Leafy Tree*. Mục 3 mô tả cách dùng *Leafy Tree* để hiện thực cây tìm kiếm nhị phân. Mục 4 đưa vào cây cân bằng có trọng số và mô tả cách dùng *Leafy Tree* để hiện thực nó.

2 *Leafy Tree* là gì

Leafy Tree là một cây nhị phân mà mỗi nút hoặc là lá, hoặc có đúng hai con. Toàn bộ thông tin được lưu trên các lá; thông tin ở mỗi nút không phải lá là kết quả gộp thông tin của các con. Cấu trúc của nó giống cây đoạn, đồng thời cũng có thể xem nó như cây tái cấu trúc Kruskal của một cây tìm kiếm nhị phân. Ở đây, cây tái cấu trúc Kruskal không phải là cây tái cấu trúc Kruskal của cây khung nhỏ nhất, mà là thao tác tương tự trên một cây tìm kiếm nhị phân: mỗi lần gộp hai nút và biến cạnh ở giữa chúng thành một nút trong đồ thị mới, cuối cùng thu được một cây.

Đây là một cấu trúc dữ liệu thuần hàm, có thể hiện thực các cấu trúc dữ liệu hàm khác một cách thuận tiện và hiệu quả, đồng thời cũng dễ hiện thực khả tri.

3 Dùng *Leafy Tree* hiện thực cây tìm kiếm nhị phân

3.1 Định nghĩa

Xét việc dùng một *Leafy Tree* để duy trì một tập hợp. Mỗi nút lá lưu một giá trị trong tập hợp, mỗi nút không phải lá lưu giá trị của con phải, tức giá trị lớn nhất trong cây con. Đồng thời, giá trị lớn nhất của con trái ở mỗi nút không phải lá không lớn hơn giá trị nhỏ nhất của con phải; nói cách khác, trong thứ tự duyệt giữa của cây này, các giá trị ở nút lá là có thứ tự. Ta gọi cây như vậy là cây tìm kiếm nhị phân được hiện thực bằng *Leafy Tree*.

3.2 Thao tác cơ bản

Để tiện mô tả, ta dùng $A.left$, $A.right$, $A.value$ để biểu thị con trái, con phải và giá trị của nút A . Với nút lá X , định nghĩa $X.left = X.right = \text{null}$. Đồng thời định nghĩa hàm $\text{NEWNODE}(x)$ để tạo một nút mới có giá trị x , đặt hai con của nút mới là null , và $\text{DELETENODE}(x)$ để thu hồi nút x .

3.2.1 Truy vấn. Quá trình tìm giá trị x trong cây con của nút T như sau.

Nếu T là nút lá, so sánh x với giá trị của T ; nếu hai giá trị bằng nhau thì tìm kiếm thành công, ngược lại thất bại. Nếu không, nếu x không lớn hơn giá trị của cây con trái của T , tìm trong cây con trái; ngược lại tìm trong cây con phải.

Thuật toán 1. Truy vấn trong cây tìm kiếm nhị phân bằng *Leafy Tree*.

```
Find(T, x):
  if T.left = null then
    if T.value = x then
      return True
    else
      return False
  end if
else
  if x <= T.left.value then
    return Find(T.left, x)
  else
    return Find(T.right, x)
  end if
end if
```

3.2.2 Chèn. Quá trình chèn giá trị x vào cây con của nút T như sau.

Nếu T là nút lá, so sánh x với giá trị của T . Nếu x lớn hơn giá trị của T , tạo hai nút mới A, B để lần lượt lưu giá trị của T và x , rồi đặt con trái và con phải của T là A và B ; khi x không lớn hơn giá trị của T , thao tác tương tự. Nếu không, nếu x không lớn hơn giá trị của cây con trái của T , chèn x vào cây con trái; ngược lại chèn vào cây con phải.

Thuật toán 2. Chèn trong cây tìm kiếm nhị phân bằng *Leafy Tree*.

```
Insert(T, x):
  if T.left = null then
```



```

A <- newNode(T.value)
B <- newNode(x)
if A.value > B.value then
    swap(A, B)
end if
T.left <- A
T.right <- B
T.value <- B.value
else
    if x <= T.left.value then
        Insert(T.left, x)
    else
        Insert(T.right, x)
    end if
    T.value <- T.right.value
end if

```

3.2.3 Xóa. Quá trình xóa giá trị x trong cây con của nút T như sau.

Nếu T là nút lá, so sánh x với giá trị của T . Nếu hai giá trị bằng nhau, việc xóa thành công: đặt mọi thuộc tính của cha T thành thuộc tính của nút anh em của T , rồi xóa nút anh em của T và chính T ; nếu không, việc xóa thất bại. Nếu T không phải lá, nếu x không lớn hơn giá trị của cây con trái của T , xóa trong cây con trái; ngược lại xóa trong cây con phải.

Thuật toán 3. Xóa trong cây tìm kiếm nhị phân bằng *Leafy Tree*. Hàm Remove nhận nút T và giá trị cần xóa x , trả về việc xóa có thành công hay không. Hàm này yêu cầu T không phải nút lá.

```

Remove(T, x):
    if x <= T.left.value then
        if T.left.size = 1 then
            if T.left.value = x then
                temp <- T.right
                T = T.right
                deleteNode(temp)
                deleteNode(T.left)
                return True
            else
                return False
            end if
        else
            return Remove(T.left, x)
        end if
    else
        if T.right.size = 1 then
            if T.right.value = x then
                temp <- T.left
                T = T.left
                deleteNode(temp)
                deleteNode(T.right)
            end if
        else
            return Remove(T.right, x)
        end if
    end if
end if

```

```

        return True
    else
        return False
    end if
else
    temp <- Remove(T.right, x)
    T.value <- T.right.value
    return temp
end if
end if

```

Tuy mã giả ở đây khá dài, trong hiện thực thực tế, nếu bảo đảm giá trị cần xóa tồn tại và dùng mảng để lưu hai con của mỗi nút, có thể tiết kiệm rất nhiều mã.

3.3 So sánh với cây tìm kiếm nhị phân

Có thể thấy, nhờ tính chất chỉ nút lá duy trì thông tin, thao tác chèn xóa trên *Leafy Tree* khá thuận tiện. Nút được chèn hoặc xóa mỗi lần thực chất đều là nút lá, giảm rất nhiều phần thảo luận rườm rà trong cây tìm kiếm nhị phân thông thường về kiểu của nút được chèn hoặc xóa: nút đó là lá, có một con, hay có hai con. Đồng thời, do mỗi nút không phải là duy trì thông tin đoạn và định vị bằng thông tin đoạn của con trái, về bản chất nó giống cách cây tìm kiếm nhị phân định vị bằng cách so sánh với giá trị ở gốc cây. Vì vậy, một hiện thực gọn hơn vẫn đạt cùng hiệu quả.

4 Dùng *Leafy Tree* hiện thực cây cân bằng có trọng số

Cây cân bằng có trọng số (*Weight Balanced Tree*, còn gọi là cây $BB[\alpha]$, hay cây cân bằng trọng lượng) là một cây tìm kiếm nhị phân lưu kích thước cây con. Nghĩa là một nút chứa các trường: giá trị (*value*), con trái (*left*), con phải (*right*) và kích thước cây con (*size*).

Trọng số *weight* của một nút phụ thuộc vào kích thước cây con của nó hoặc bằng kích thước cây con của nó. Cụ thể, cần thỏa

$$\text{weight}[x] = \text{weight}[x.\text{left}] + \text{weight}[x.\text{right}].$$

Nếu một nút x thỏa

$$\min(\text{weight}[x.\text{left}], \text{weight}[x.\text{right}]) \geq \alpha \cdot \text{weight}[x],$$

thì gọi nút đó là α -cân bằng theo trọng số; hiển nhiên $0 < \alpha \leq \frac{1}{2}$. Chiều cao h của một cây cân bằng có trọng số chứa n phần tử thỏa

$$h \leq \log_{\frac{1}{1-\alpha}} n = O(\log n).$$

4.1 Định nghĩa

Trong cây cân bằng có trọng số được hiện thực bằng *Leafy Tree*, kích thước cây con là số nút lá trong cây con, còn giá trị của một nút không phải lá là giá trị của con phải của nó. Tức là với một nút lá x , $x.\text{size} = 1$; với một nút không phải lá x ,

$$x.\text{size} = x.\text{left}.\text{size} + x.\text{right}.\text{size}, \quad x.\text{value} = x.\text{right}.\text{value}.$$

Trọng số của một nút chính là kích thước cây con của nó, tức là $\text{weight}[x] = x.\text{size}$.

4.2 Cách hiện thực

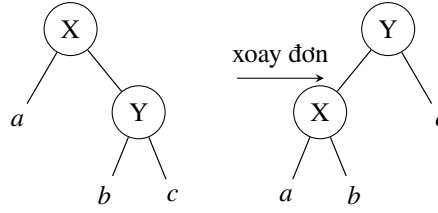
Cây cân bằng có trọng số có hai cách hiện thực: tái cấu trúc và xoay. Ở đây chỉ giới thiệu cân bằng bằng phép xoay.

Với một cây thỏa α -cân bằng theo trọng số, sau khi chèn hoặc xóa một nút, các nút không còn thỏa α -cân bằng chắc chắn nằm trên một dây chuyền. Ta xét xử lý lần lượt các nút trên dây chuyền này.

Giả sử trong một cây con T của cây này, mọi nút ngoài nút gốc đều thỏa α -cân bằng theo trọng số. Ta có thể dùng một phép xoay đơn hoặc xoay kép để làm T thỏa α -cân bằng theo trọng số.

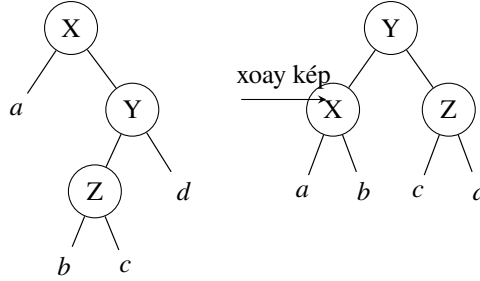
Định nghĩa độ cân bằng của x là

$$\frac{\text{weight}[x.\text{left}]}{\text{weight}[x]}.$$



Như hình trên, xét một phép xoay đơn. Gọi độ cân bằng của X, Y trước khi xoay là ρ_1, ρ_2 , sau khi xoay là γ_1, γ_2 . Ta có

$$\gamma_1 = \frac{\rho_1}{\rho_1 + (1 - \rho_1)\rho_2}, \quad \gamma_2 = \rho_1 + (1 - \rho_1)\rho_2.$$



Như hình trên, xét một phép xoay kép. Gọi độ cân bằng của X, Y, Z trước khi xoay là ρ_1, ρ_2, ρ_3 , sau khi xoay là $\gamma_1, \gamma_2, \gamma_3$. Ta có

$$\gamma_1 = \frac{\rho_1}{\rho_1 + (1 - \rho_1)\rho_2\rho_3}, \quad \gamma_2 = \rho_1 + (1 - \rho_1)\rho_2\rho_3, \quad \gamma_3 = \frac{\rho_2(1 - \rho_3)}{1 - \rho_2\rho_3}.$$

Giả sử $\rho_1 < \alpha$. Trong trường hợp chỉ chèn hoặc xóa một nút, ρ_1 nhỏ nhất là $\frac{\alpha}{2-\alpha}$, đạt được khi cây con a vốn có 2 nút và nay xóa một nút trong đó.

Trong các ràng buộc đã cho, có thể chứng minh rằng khi $\alpha \leq 1 - \frac{\sqrt{2}}{2}$, bằng hai phép nói trên, các nút có độ cân bằng bị thay đổi đều có độ cân bằng mới nằm trong $[\alpha, 1 - \alpha]$. Khi chứng minh có thể còn cần chặn thêm ρ_1 và phân loại trong phạm vi nhỏ. Do khuôn khổ có hạn, phần chứng minh cụ thể không trình bày ở đây.

Thao tác cụ thể là: khi

$$\rho_2 < \frac{1 - 2\alpha}{1 - \alpha},$$

thực hiện một lần xoay đơn; ngược lại thực hiện một lần xoay kép.

4.3 Thao tác cơ bản

Để tiện mô tả, ngoài *left* và *right*, ta còn dùng $A.child[0]$, $A.child[1]$ để biểu thị con trái và con phải của nút A . Với nút lá X , định nghĩa $X.child[0] = X.child[1] = \text{null}$. Đồng thời định nghĩa hàm $\text{NEWNODE}(x)$ để tạo nút mới có giá trị x , đặt hai con của nút mới là null , và $\text{DELETENODE}(x)$ để thu hồi nút x .

4.3.1 Gộp thông tin. Sau khi một nút trải qua thao tác, cần tính lại kích thước cây con và giá trị của nó.

Thuật toán 4. Gộp thông tin trong cây cân bằng có trọng số bằng *Leafy Tree*.

$\text{Pushup}(T)$:

```
if not T.left = null then
  T.size <- T.left.size + T.right.size
  T.value <- T.right.value
end if
```

4.3.2 Xoay đơn. Thao tác này xoay Y đơn lên vị trí của X . Ở đây, để không thay đổi con trở trở tới X , sau khi xoay ta hoán đổi vị trí của chúng. Trong thuật toán sau, $\oplus$ biểu thị xor.

Thuật toán 5. Xoay đơn trong cây cân bằng có trọng số bằng *Leafy Tree*. Hàm Rotate nhận nút T và biến bool d , biểu thị xoay $child[d]$ lên vị trí của T .

$\text{Rotate}(T, d)$:

```
temp <- T.child[d xor 1]
T.child[d xor 1] <- T.child[d]
T.child[d] <- T.child[d xor 1].child[d]
T.child[d xor 1].child[d] <- T.child[d xor 1].child[d xor 1]
T.child[d xor 1].child[d xor 1] <- temp
Pushup(T.child[d xor 1])
Pushup(T)
```

4.3.3 Duy trì cân bằng. Sau khi một nút trải qua thao tác, cần dùng một lần xoay đơn hoặc xoay kép để duy trì cân bằng.

Thuật toán 6. Duy trì cân bằng trong cây cân bằng có trọng số bằng *Leafy Tree*.

$\text{Maintain}(T)$:

```
if not T.left = null then
  if T.left.size < T.size * alpha then
    d <- 1
  else if T.right.size < T.size * alpha then
    d <- 0
  else
    return
  end if
  if T.child[d].child[d xor 1].size
    >= T.child[d].size * (1 - 2 * alpha) / (1 - alpha) then
    Rotate(T.child[d], d xor 1)
```

```

    end if
    Rotate(T, d)
end if

```

4.3.4 Chèn. Thao tác này tương tự cây tìm kiếm nhị phân được hiện thực bằng *Leafy Tree*.

Thuật toán 7. Chèn trong cây cân bằng có trọng số bằng *Leafy Tree*.

```

Insert(T, x):
  if T.size = 1 then
    T.left <- newNode(x)
    T.right <- newNode(T.value)
    if x > T.value then
      swap(T.left, T.right)
    end if
  else
    Insert(T.child[x > T.left.value], x)
  end if
  Pushup(T)
  Maintain(T)

```

4.3.5 Xóa. Thao tác này tương tự cây tìm kiếm nhị phân được hiện thực bằng *Leafy Tree*.

Thuật toán 8. Xóa trong cây cân bằng có trọng số bằng *Leafy Tree*. Hàm Remove nhận nút T và giá trị cần xóa x . Hàm này yêu cầu T không phải nút lá.

```

Remove(T, x):
  d <- x > T.left.value
  if T.child[d].size = 1 then
    if T.child[d].value = x then
      deleteNode(T.child[d])
      temp <- T.child[d xor 1]
      T.left = T.child[d xor 1].left
      T.right = T.child[d xor 1].right
      T.value = T.child[d xor 1].value
      deleteNode(temp)
    else
      return
    end if
  else
    Remove(T.child[d])
  end if
  Pushup(T)
  Maintain(T)

```

4.3.6 Truy vấn. Thao tác này giống cây tìm kiếm nhị phân được hiện thực bằng *Leafy Tree*. Mã giả xem thuật toán 1.

4.4 Tốc độ chạy

Ở đây tác giả chọn một số mã chạy nhanh cho bài LOJ #104, *Cây cân bằng thông thường*, để kiểm thử.

Do kết quả kiểm thử từ các bộ sinh dữ liệu khác nhau gần như giống nhau, ở đây chỉ trình bày kết quả trên dữ liệu ngẫu nhiên thuần túy.⁸ Kết quả này có thể có sai số nhất định, và mã cũng không bảo đảm có hằng số tốt nhất, nhưng có thể phản ánh đại khái tốc độ chạy của các loại cây cân bằng này.

Hình trong bản gốc so sánh tỉ lệ giữa thời gian chạy thực tế của từng cây cân bằng và thời gian chạy thực tế của cây cân bằng có trọng số, trên trục hoành $\log_{10}(n)$. Có thể thấy, tuy cây cân bằng có trọng số chậm hơn cây đỏ-đen và SBT, nó nhanh hơn Treap và Splay, đồng thời có tốc độ gần cây thay thế.

4.5 Thao tác khác

Ngoài các thao tác cơ bản như chèn và xóa, cây cân bằng có trọng số còn có thể hiện thực thao tác gộp, tách, v.v. với độ phức tạp tốt, cũng như hiện thực khả trì.

4.5.1 Gộp. Thao tác gộp là: cho hai cây A và B , bảo đảm giá trị lớn nhất trong A không lớn hơn giá trị nhỏ nhất trong B , cần gộp chúng thành một cây mới. Độ phức tạp của thao tác này là

$$O\left(\log \frac{\max(A.size, B.size)}{\min(A.size, B.size)}\right).$$

Trước hết, giả sử $A.size \geq B.size$. Nếu

$$B.size \geq \frac{\alpha}{1-\alpha} \cdot A.size,$$

ta có thể trực tiếp tạo một nút mới, với con trái là A và con phải là B .

Ngược lại, nếu

$$A.left.size \geq \alpha \cdot (A.size + B.size),$$

có thể đệ quy gộp con phải của A với B , rồi gộp kết quả với con trái của A .

Nếu không, có thể đệ quy gộp con trái của A với con trái của con phải của A , đồng thời gộp con phải của con phải của A với B , cuối cùng gộp hai kết quả này với nhau. Do khuôn khổ có hạn, phần chứng minh độ phức tạp không trình bày ở đây.

4.5.2 Tách. Thao tác tách là: cho một cây T , cần tách phần không lớn hơn k và phần lớn hơn k thành hai cây độc lập.

Thao tác này có thể được thực hiện trực tiếp bằng đệ quy, rồi gọi thao tác gộp nói trên với các kết quả trả về. Độ phức tạp là $O(\log T.size)$. Do khuôn khổ có hạn, phần chứng minh độ phức tạp không trình bày ở đây.

4.5.3 Khả trì hóa. Với mỗi thao tác, có thể sao chép toàn bộ các nút bị sửa đổi và lần lượt tạo mới chúng, tức path copying. Số nút bị sửa đổi trong một lần chèn hoặc xóa hiển nhiên là $O(\log size)$.

Trong cây cân bằng có trọng số được hiện thực bằng *Leafy Tree*, số nút bị sửa đổi trong một thao tác đơn lẻ là rất nhỏ theo kỳ vọng, và không cần tạo thêm các nút dư thừa. Vì vậy nó dùng ít bộ nhớ, hằng số cũng nhỏ, và có ưu thế lớn so với các cây cân bằng khác, chẳng hạn Treap không xoay.

⁸Mã kiểm thử và dữ liệu gốc có thể tải tại <https://mcfx.us/ctsc>.

5 Tổng kết

Bài viết này giới thiệu *Leafy Tree* và cây cân bằng có trọng số được hiện thực bằng nó.

So với cây tìm kiếm nhị phân, *Leafy Tree* có thao tác chèn xóa dễ hiện thực hơn, nhưng dùng gấp đôi số nút.

Cây cân bằng có trọng số có hằng số nhỏ, hiện thực tương đối đơn giản, đồng thời cũng có thể hiện thực khả tri một cách thuận tiện. So với cây đỏ-đen có hằng số nhỏ hơn, nó không cần ghi thêm thông tin phụ, vì kích thước nút có thể dùng để cắt đoạn hoặc tìm phần tử nhỏ thứ K , và dễ hiện thực hơn nhiều. So với Splay dễ hiện thực, lượng mã về cơ bản tương đương, nhưng hằng số nhỏ hơn và có thể khả tri hóa. So với Treap, nó không cần ghi thông tin phụ và có hằng số nhỏ hơn. So với cây thay thế, độ phức tạp của nó không phải dạng khẩu hao, và có thể hiện thực khả tri.

Leafy Tree cân bằng có trọng số, kết hợp hai ý tưởng trên, có thể hiện thực phần lớn chức năng của cây cân bằng bằng lượng mã khá ngắn, hằng số nhỏ, và có thể khả tri hóa.

Ngoài ra, *Leafy Tree* còn có thể hiện thực nhiều cấu trúc dữ liệu dựa trên cây nhị phân, như các cây cân bằng khác, cây đoạn, heap, v.v. Nhưng do khuôn khổ có hạn, bài viết không thể lần lượt giới thiệu tất cả.

Lời cảm ơn

Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu.

Cảm ơn các thầy Zhang Junliang và Lin Yang của Trường Trung học số 7 Thành Đô đã quan tâm và chỉ dẫn trong nhiều năm.

Cảm ơn bạn Li Xinlong của Trường Trung học số 7 Thành Đô và bạn Zhao Yuyang của THPT trực thuộc Đại học Sư phạm An Huy đã cung cấp ý tưởng cho bài viết này.

Cảm ơn các thầy cô và bạn học khác đã giúp đỡ tôi.

Tài liệu tham khảo

1. Nievergelt, J. and Reingold, E. M. (1972), *Binary search trees of bounded balance*. In *Proceedings of the Fourth Annual ACM Symposium on Theory of Computing*. ACM, pp. 137–142.
2. Wikipedia, *Weight-balanced tree*.
3. Wikipedia, *Binary search tree*.
4. LOJ #104, *Cây cân bằng thông thường*, <https://loj.ac/problem/104>.

10 Chủ đề chính

Từ mục lục và phần mở đầu của các bài, tập năm 2018 bao phủ các nhóm chủ đề sau:

- hàm sinh, xác suất, kỳ vọng và các bài toán gieo xúc xắc;
- cấu trúc dữ liệu chuỗi, cây hậu tố, suffix automaton, KMP và tính chu kỳ của chuỗi;
- hồi quy bảo toàn thứ tự, chia đôi toàn cục và tối ưu trên quan hệ thứ tự bộ phận;
- quy hoạch động và cấu trúc dữ liệu cho khối liên thông trên cây;
- tích chập, DFT, FFT, FWT, FMT và các bài toán đa thức;

- cây nhị phân, *Leafy Tree*, Splay, Treap, tìm kiếm theo ngón tay và cây cân bằng có trọng số;
- tổng hàm số học, hàm nhân tính, tổng trên số nguyên tố và các biến thể số học đặc biệt;
- matroid, tối ưu tổ hợp, cây khung phương sai nhỏ nhất và đồ thị Euler;
- các báo cáo ra đề cho *Số nút của cây hậu tố*, *Fim 4*, *Jellyfish*, *Tiểu H thích tô màu*, *Hàng đợi hoàn hảo* và *Cây khung phương sai nhỏ nhất*.

Ghi chú về nguồn

Tệp PDF có 210 trang. pdftotext trích xuất được Unicode đúng cho phần đầu sách, mục lục và các trang thân bài đã kiểm tra, nên không cần render mục lục bằng ảnh để đọc lại. Bản dịch đã bổ sung toàn văn bài đầu tiên của Yang Maolong, tương ứng trang in 1–10 và trang PDF 3–12; bài thứ hai của Chen Jianglun, tương ứng trang in 11–22 và trang PDF 13–24; bài thứ ba của Gao Ruiquan, tương ứng trang in 23–33 và trang PDF 25–35; bài thứ tư của Wu Jinzhao, tương ứng trang in 34–44 và trang PDF 36–46; bài thứ năm của Ren Xuandi, tương ứng trang in 45–57 và trang PDF 47–59; bài thứ sáu của Liang Yancheng, tương ứng trang in 58–74 và trang PDF 60–76; cùng bài thứ bảy của Wang Siqi, tương ứng trang in 75–86 và trang PDF 77–88. Một số công thức dài, ví dụ mẫu và hình minh họa được đối chiếu thêm bằng ảnh render từ PDF vì bố cục công thức và chú thích hình có chỗ bị méo trong văn bản trích xuất. Hình đường gấp khúc trong bài thứ ba được dựng lại bằng TikZ từ trang PDF 33. Bài thứ tư không có hình; ở bài thứ năm, các hình cây của cấu trúc LCM và ví dụ đường kính được kiểm tra bằng render trang PDF 53–55 và được diễn giải trong văn bản vì chúng chỉ minh họa cấu trúc, không chứa dữ liệu đề bài riêng. Bài thứ sáu không có bảng, hình hay mẫu code; các công thức dài về $\oplus$, Karatsuba, xor_K và phép toán $*$ được đối chiếu bằng ảnh render các trang PDF 60–76. Bài thứ bảy có hai sơ đồ xoay cây được dựng lại bằng TikZ từ trang PDF 80; biểu đồ tốc độ chạy trên trang PDF 84 được kiểm tra bằng render và diễn giải trong văn bản, vì dữ liệu chính của biểu đồ là so sánh định tính giữa các cây cân bằng.